

Designing with the Nios II Processor and Qsys

Objectives

- Students will be able to:
 - Describe the Nios[®] II softcore processor
 - Use Qsys to create complex embedded systems
 - Create and debug software for the Nios II processor
 - Create Nios II Custom Peripherals and attach them to the auto-generated Qsys Interconnect
 - Append Custom Instructions to Nios II instruction set
 - Program an Altera FPGA on an Altera development board
 - Program Flash memory on a development board
 - Design Avalon master, slave and streaming peripherals
 - Perform an RTL system simulation in ModelSim[®]

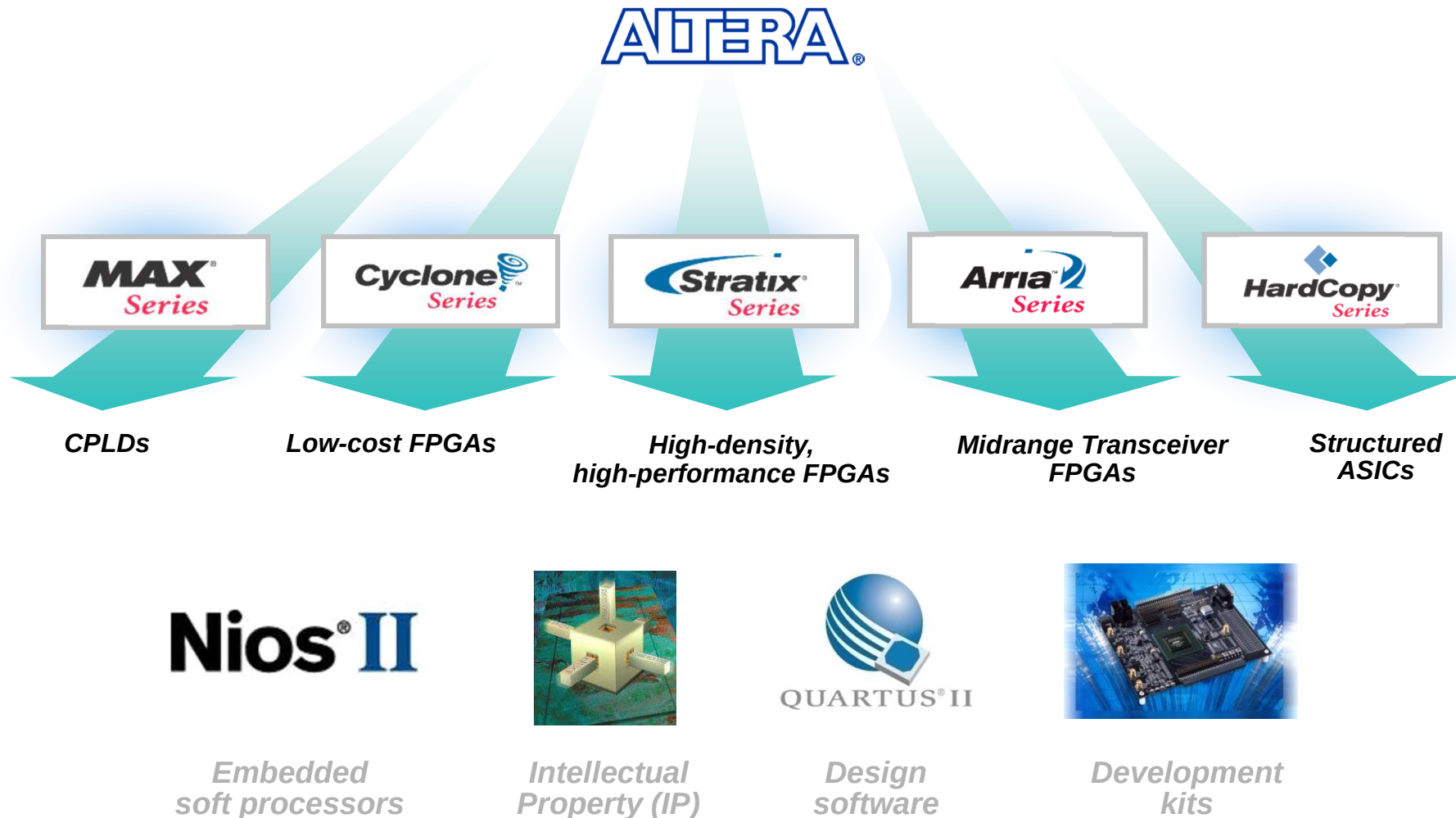
Agenda

- Introduction to Altera
- Nios II processor system hardware development
- Software development and debug tools
- Board bring-up tools
- Qsys Interconnect
- Importing user-defined Custom Peripherals into Qsys
- Nios II processor Custom Instructions
- Working with Altera development boards
- Developing systems on a programmable chip
- Creating Custom Components for Qsys

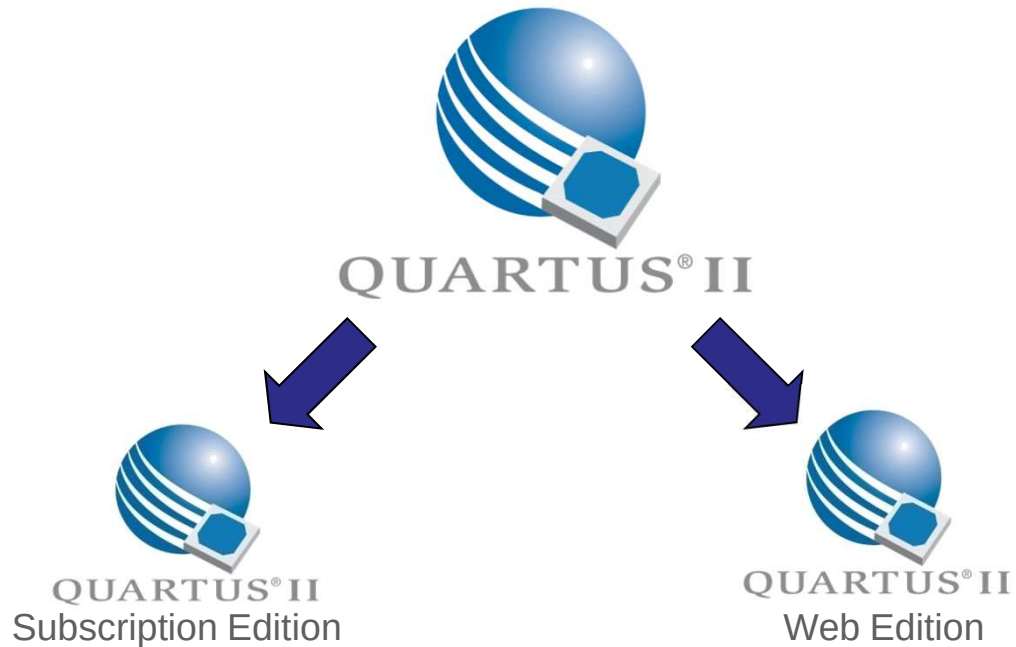
Designing with the Nios II Processor and Qsys

Nios II Processor Systems and
Altera Embedded Design Tools

A Complete Solutions Portfolio



Quartus II Software – Two Editions



Devices Supported	All	Selected Devices
Features	100%	95%
Distribution	Internet & DVD	Internet & DVD
Price	Paid	Free

[Feature Comparison available on Altera web site](#)

Qsys: Next-Generation SOPC Builder

- Lets you build systems with or without a Nios II processor via a convenient GUI
- Offers
 - Easy-to-use GUI similar to SOPC Builder
 - Higher performance interconnect based on a network-on-chip (NoC) architecture
 - Support for hierarchical design, enabling system scalability

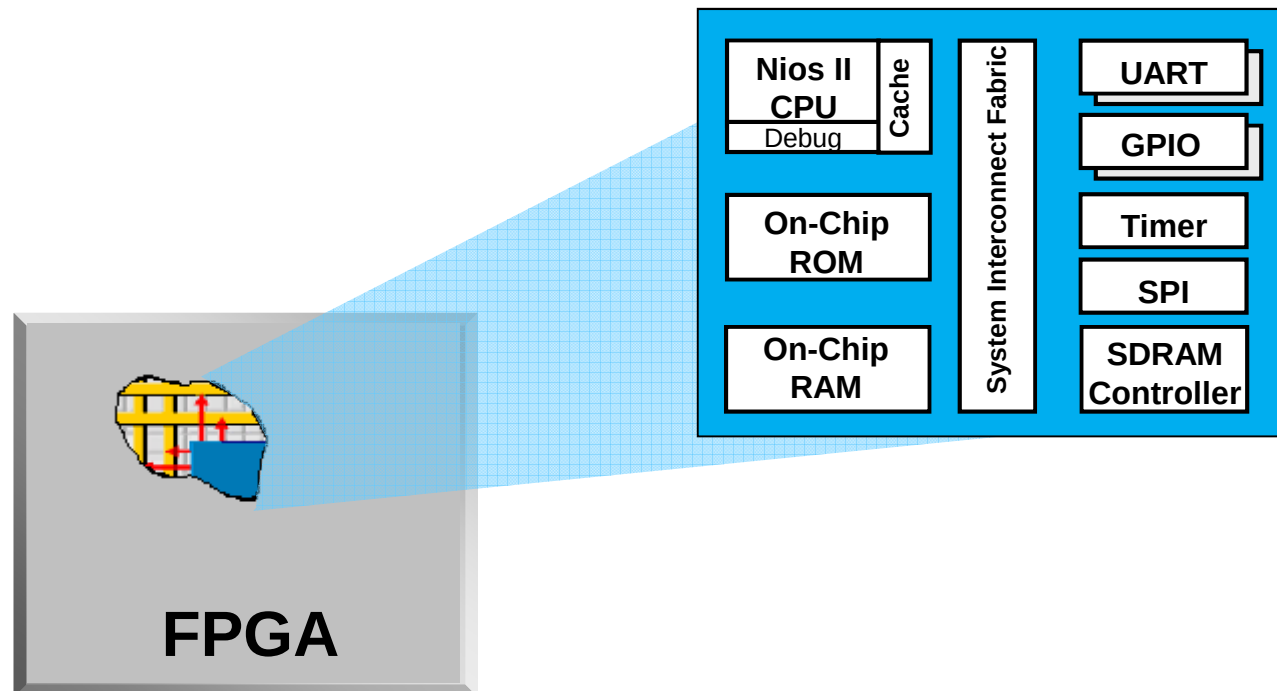


Designing with the Nios II Processor and Qsys

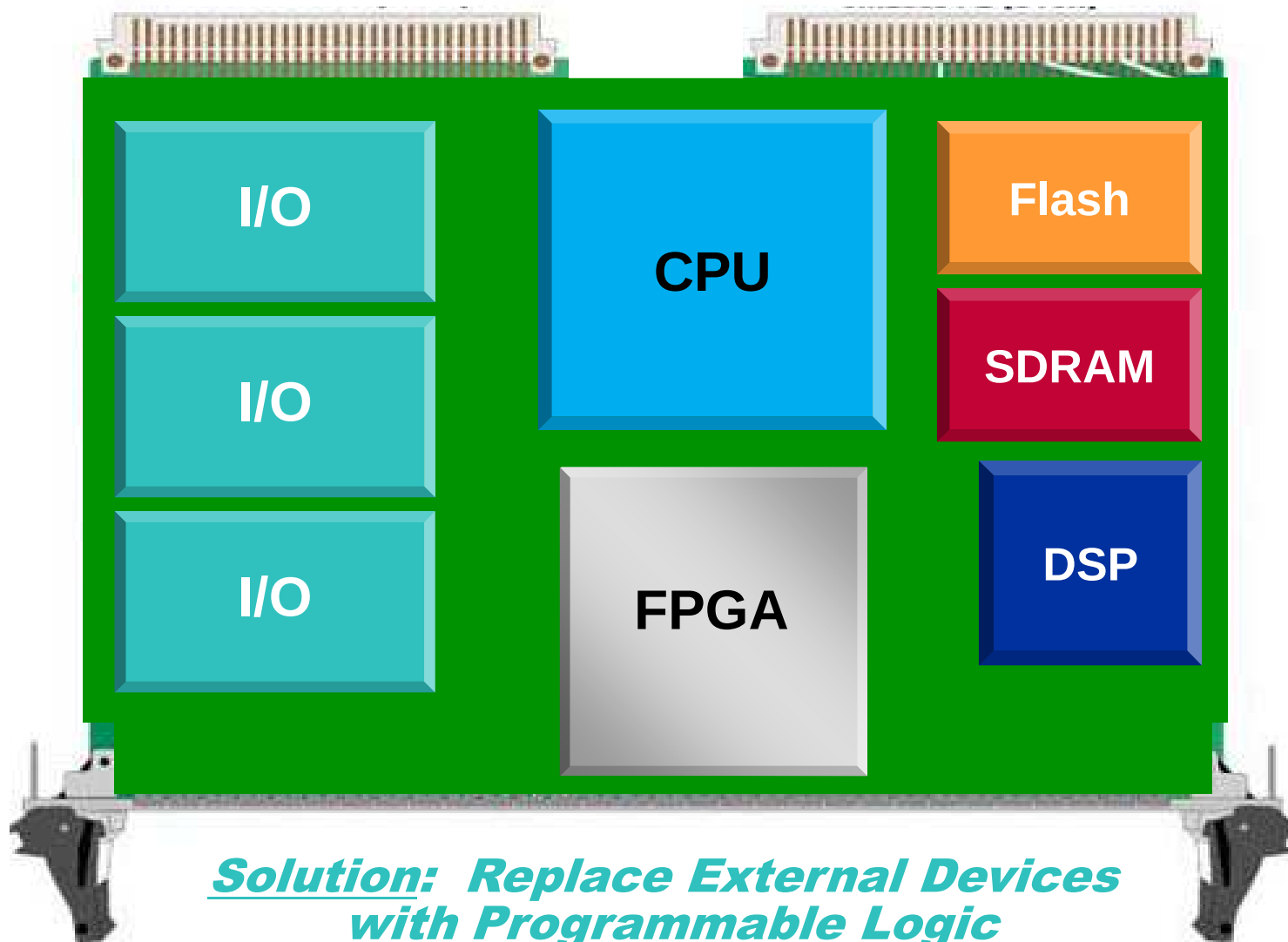
Nios II Processor-Based Systems
- Hardware Development

What is the Nios II Processor

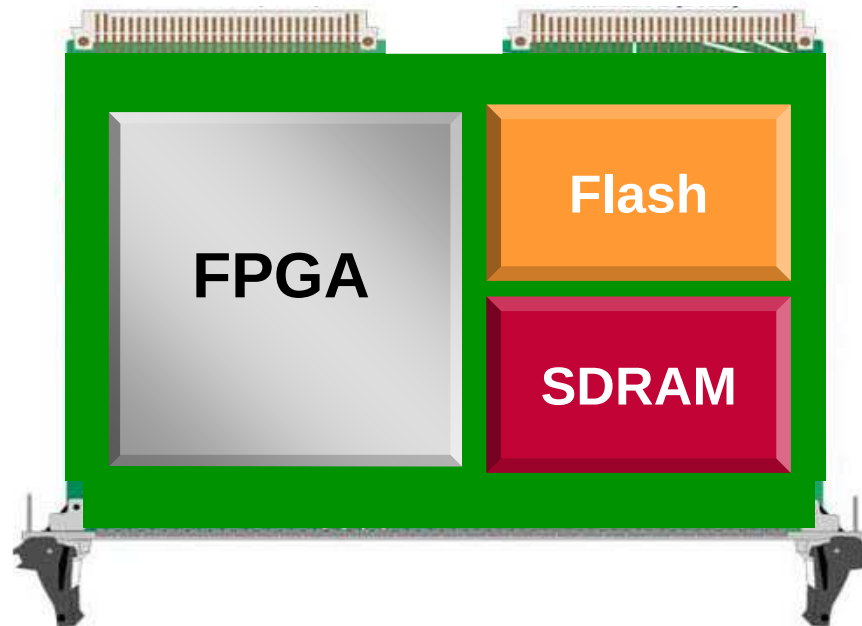
- Second Generation Soft-Core 32 Bit RISC Microprocessor
 - Nios II Processor + all peripherals written in HDL
 - Can be targeted for all Altera FPGAs
 - Synthesis using Quartus® II integrated synthesis engine



Problem: Reduce Cost, Complexity, and Power

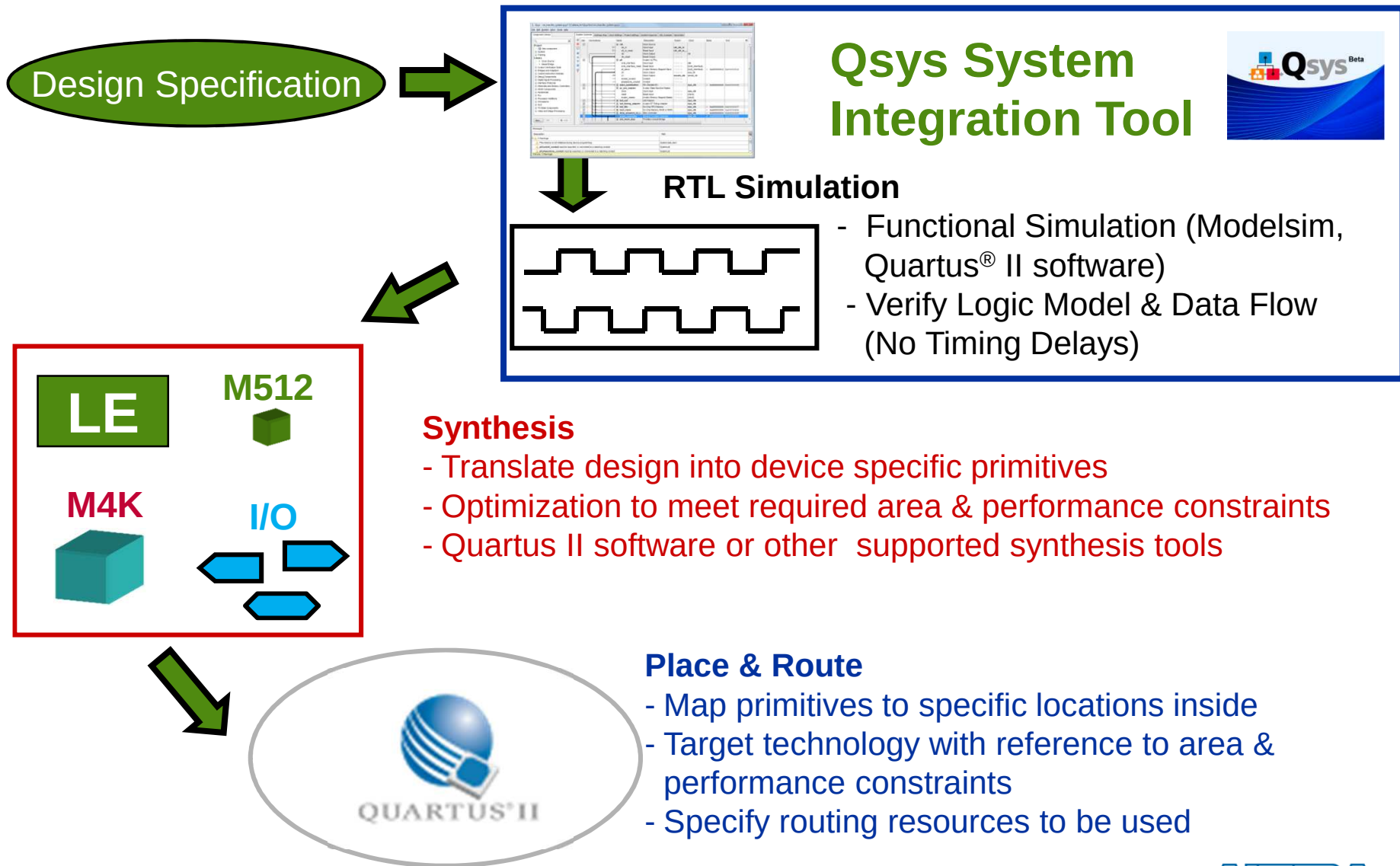


System on a Programmable Chip (SOPC)



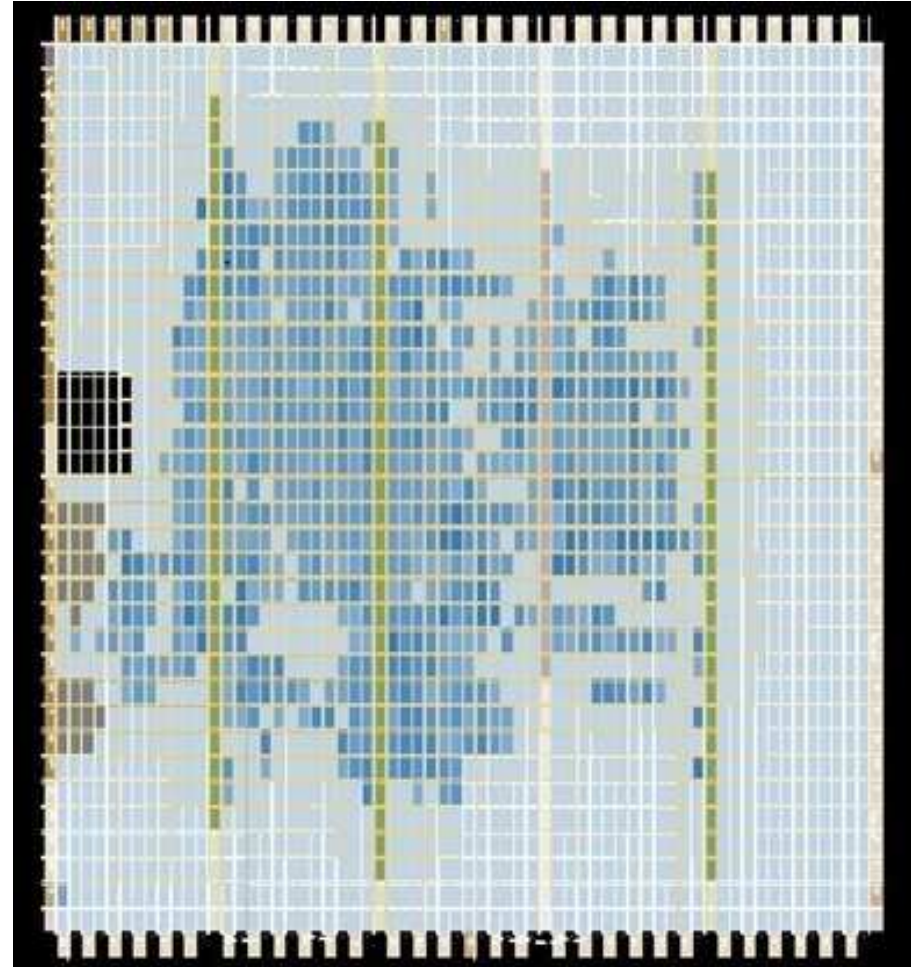
***CPU is a Critical Control Function
Required for System-Level Integration***

FPGA Hardware Design Flow

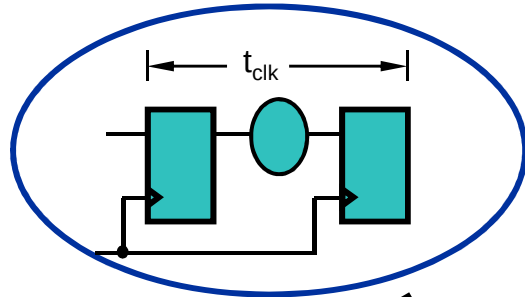


Compiled Qsys System Inside FPGA

- Use Quartus II software Integrated Synthesis and Place & Route engines to implement system in FPGA logic

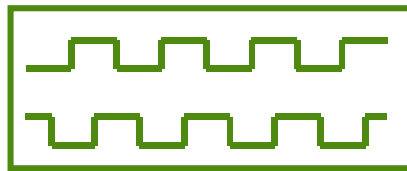


FPGA Hardware Design Flow (cont.)



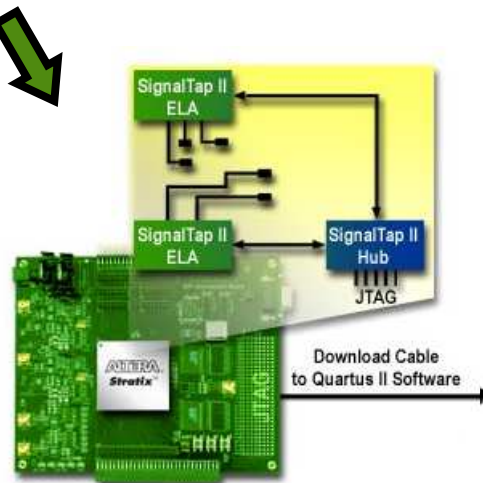
Timing Analysis

- Verify performance specifications were met
- TimeQuest static timing analysis



Gate Level Simulation

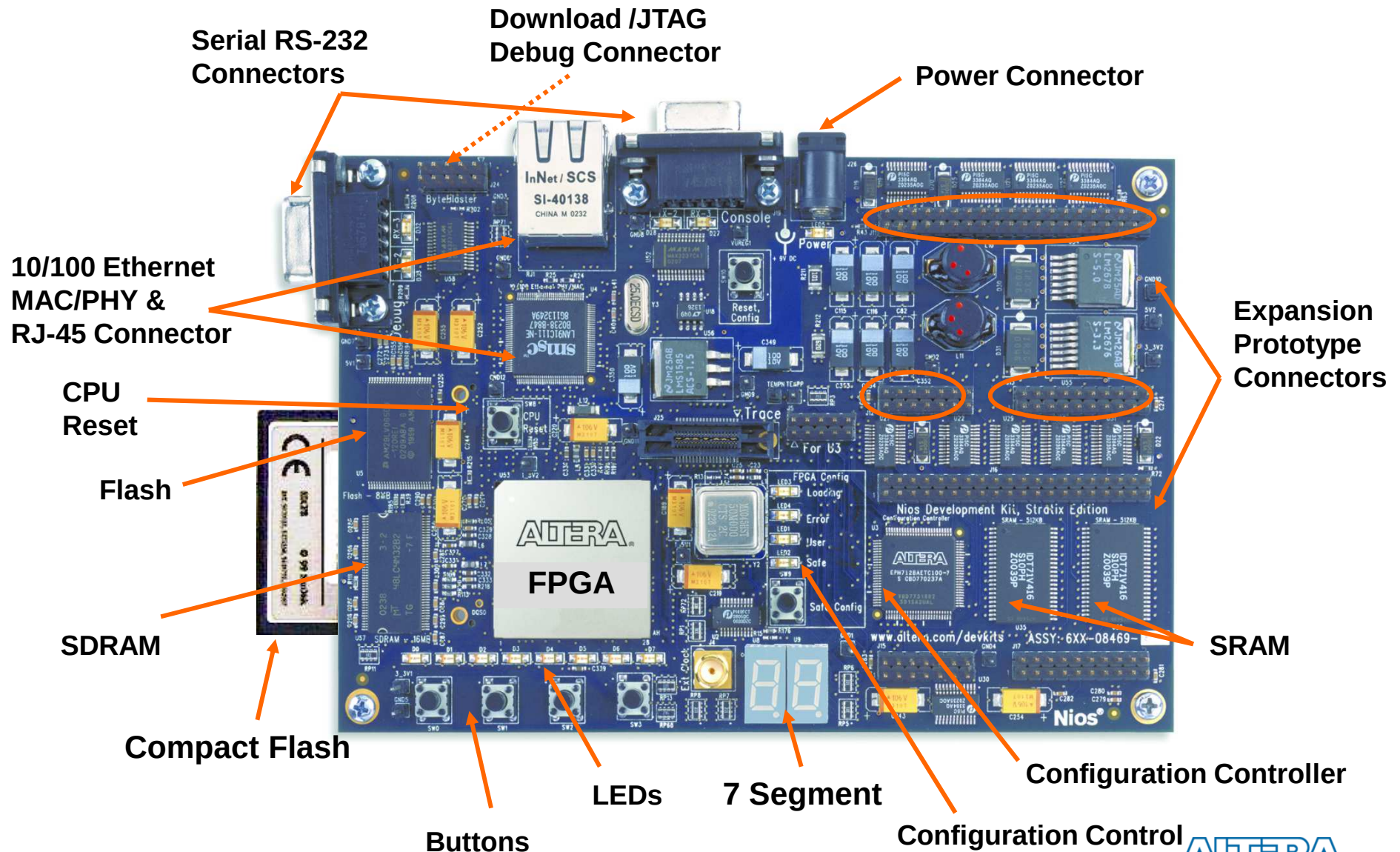
- Timing simulation
- Verify design will work in target technology



Test FPGA on PC Board

- Program & test device on board
- Use Quartus II tools (e.g. Signaltap[®] II logic analyzer) for debugging

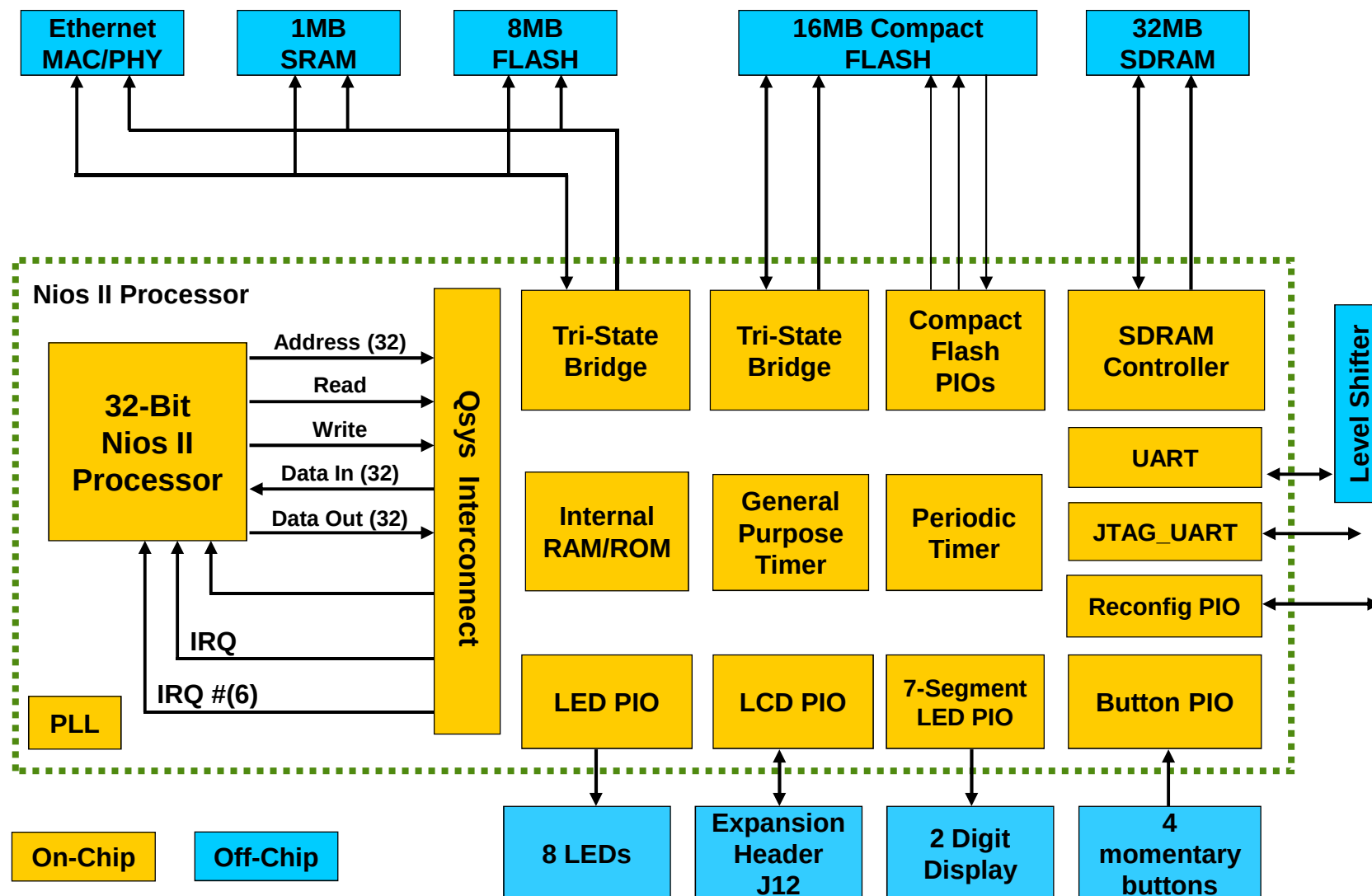
Nios II Development Kits



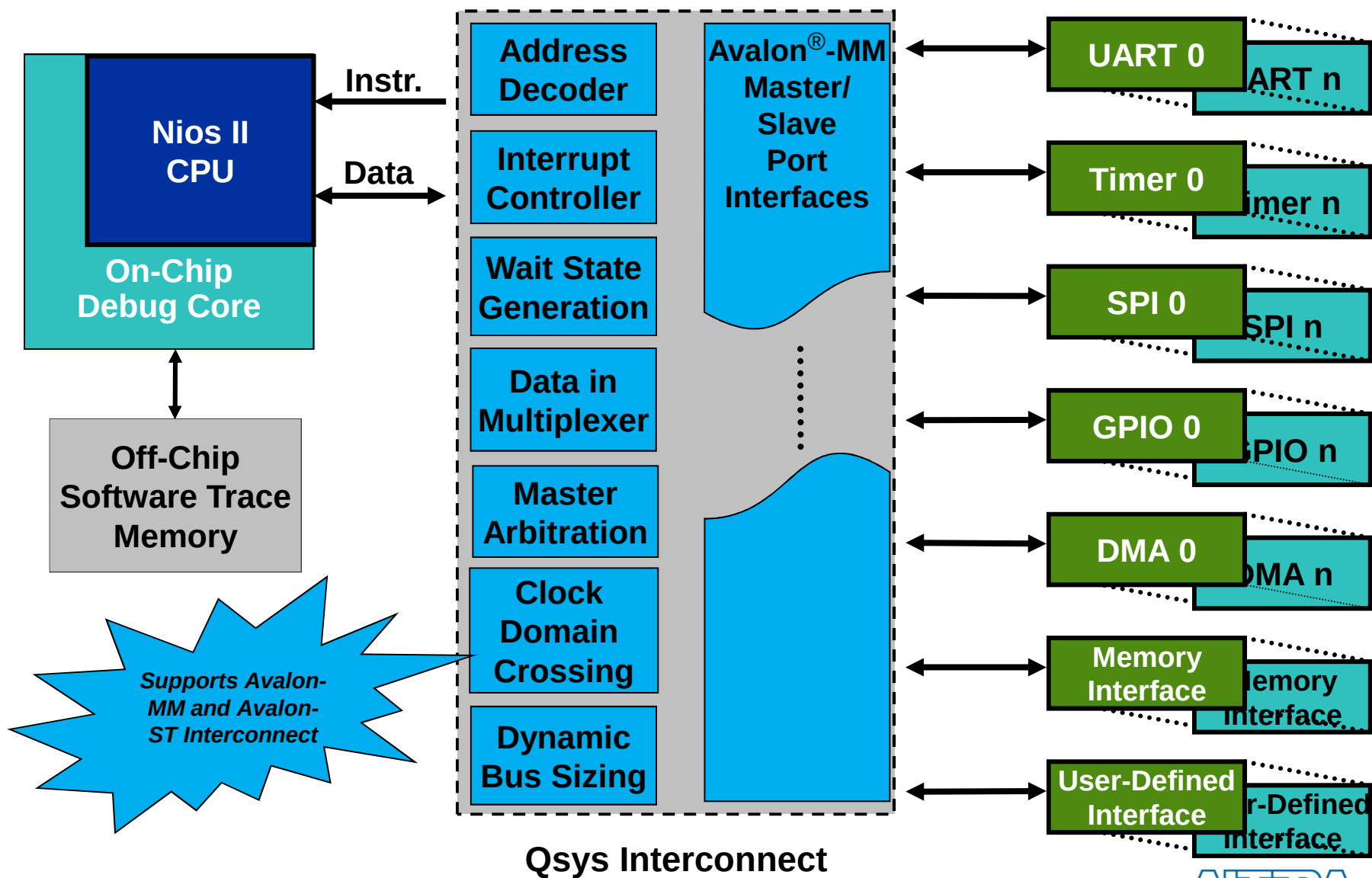
Reference Designs For Dev Kits

- Several reference designs are available
 - On Altera.com
<http://www.altera.com/products/devkits/kit-index.html>
 - On the Altera Wiki
<http://www.alterawiki.com/wiki/Special:Categories>
 - Plus in the Nios II Embedded Design Suite Installation
See: C:\altera\<ver>\nios2eds\examples\verilog
 C:\altera\<ver>\nios2eds\examples\vhdl
- These can be used as-is in final hardware platform or customized for system-specific needs

“Standard” Reference Design Block Diagram



Typical System Architecture

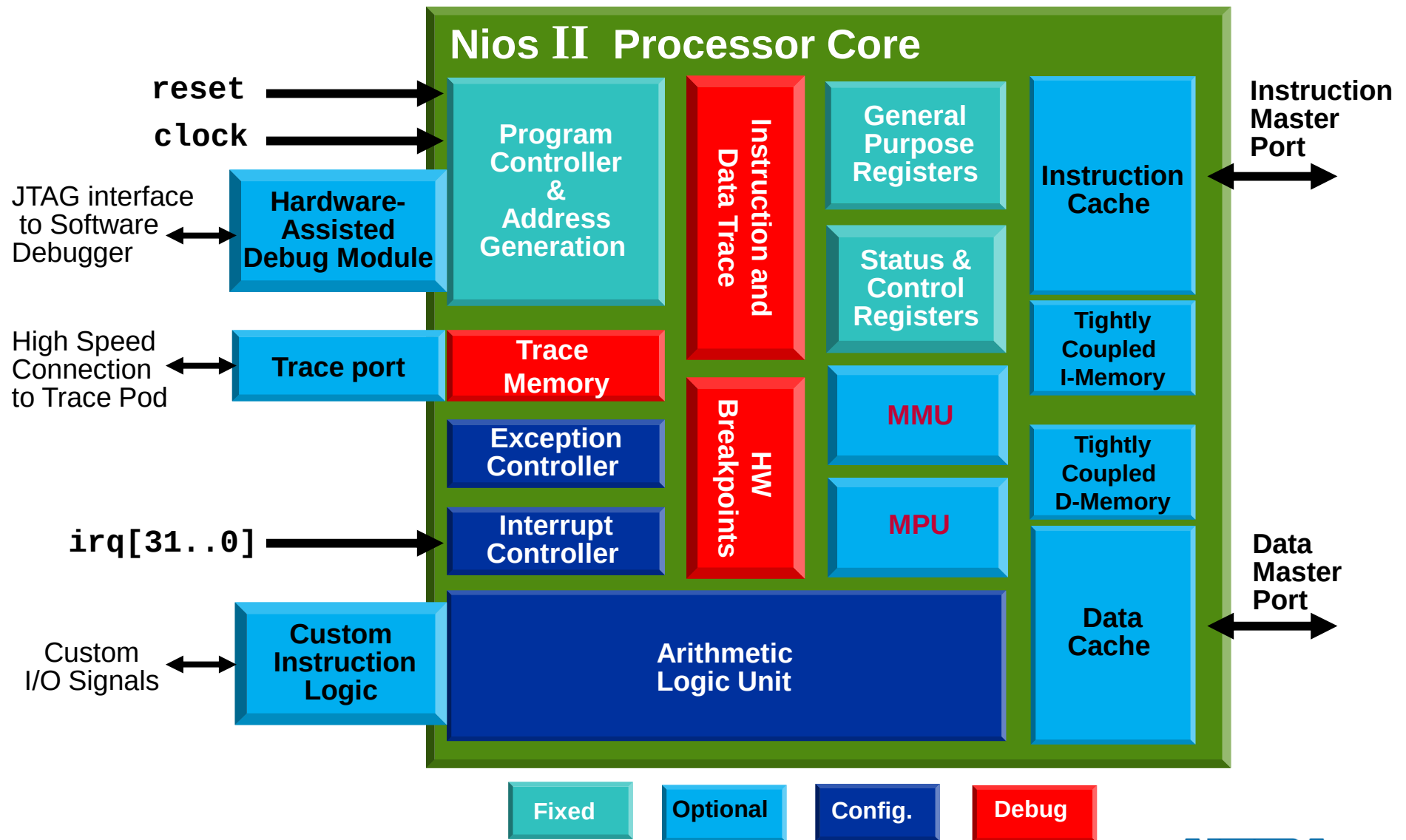


Nios II Processor Architecture

■ Classic Pipelined RISC Machine

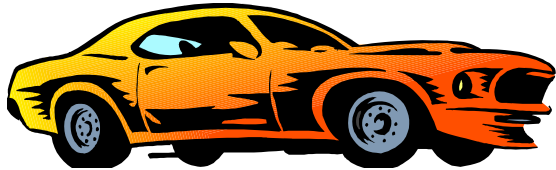
- 32 General Purpose Registers
- 3 Instruction Formats
- 32-Bit Instructions
- 32-Bit Data Path
- Flat Register File
- Separate Instruction and Data Cache (configurable sizes)
- Tightly-Coupled Memory Options
- Branch Prediction
- 32 Prioritized Interrupts
- On-Chip Hardware (Multiply, Shift, Rotate)
- Memory Management Unit (MMU)
- Memory Protection Unit (MPU)
- Custom Instructions
- JTAG-Based Hardware Debug Unit

Nios II Processor Block Diagram



Nios II Processor Versions

- Nios II processor comes in three ISA compatible versions



- FAST: Optimized for speed



- STANDARD: Balanced for speed and size



- ECONOMY: Optimized for size

■ Software

- Code is binary compatible
 - No changes required when CPU is changed

Nios II Processor Comparison Chart

	Nios II /f Fast	Nios II /s Standard	Nios II /e Economy
Instruction Throughput (Max)	1 instruction / clock	1 instruction / clock	1 instruction / 6 clocks
Maximum Active Instructions in Pipeline	6	5	1
H/W Multiplier & Barrel Shifter	1 Cycle	3 Cycles	Emulated in Software
Branch Prediction	Dynamic	Static	None
Instruction Cache	Configurable	Configurable	None
Data Cache	Configurable	None	None
Logic Requirements (Typical LEs)	1800 w/o MMU 3200 w/ MMU	1200	600
Custom Instructions	Up to 256		

Hardware Multiplier Acceleration

- Nios II processor, Economy version - No multiply hardware
 - Uses GNUPro Math Library to Implement Multiplier
- Nios II processor, Standard - Full hardware multiplier
 - $32 \times 32 \rightarrow 32$ in 3 Clock Cycles if DSP block present, else uses software only multiplier
- Nios II processor, Fast - Full hardware multiplier
 - $32 \times 32 \rightarrow 32$ in 1 Clock Cycles if DSP block present, else uses software only multiplier

Acceleration Hardware	Clock Cycles ($32 \times 32 \rightarrow 32$)
None	250
Standard MUL in Stratix [®] FPGA	3
Fast MUL in Stratix FPGA	1

Hardware Multiplier Support

- Stratix device family DSP Blocks
- Cyclone device family Multiplier Blocks
- Optional LE implementation
 - Can also be used in lieu of DSP or multiplier blocks
 - Mul, Shift, Rotate (~ 11 Clocks Per Mul)
 - Nios II Processor Handbook – core implementation details

Licensing

- Nios II processor delivered as encrypted Megacore
 - Licensed via feature line in existing Quartus II software license file
 - Consistent with general Altera Megacore delivery mechanism
 - Enables Detection Of Nios II processor IP in customer designs (Talkback)
- No Nios II processor feature line (OpenCore Plus Mode)
 - System runs if tethered to host PC
 - System times out if disconnected from PC after ~ 1 hr
- Nios II processor feature line (active subscriber)
 - Customers can obtain licenses from www.altera.com
 - Nios II CPU RTL remains encrypted
 - No extra cost when migrating to HardCopy devices
- Nios II processor source license
 - Available upon request on case-by-case basis
 - Required when migrating to non-Altera ASIC

Installation

- Web download (or install DVD in Kit)
 - Note: Limited Quartus II Software Web Edition available for free

The screenshot shows the Altera website's 'Design Software' page. The page has a blue header with the Altera logo and navigation links: Products, End Markets, Technology, Training, Support, About Altera, and Buy Online. A search bar is located in the top right corner. The main content area is titled 'Design Software' and includes a breadcrumb trail: Home > Products > Design Software. Below this is a large image showing a person working on a laptop with a 'QUARTUS II' software box overlaid. The page is divided into sections: 'Logic Design' (highlighted with a green oval), 'DSP Design', 'Getting Started', 'Switching to Quartus II', and 'Partners'. The 'Logic Design' section lists two options: 'Quartus II Subscription Edition Software' and 'Quartus II Web Edition Software'. The 'Web Edition' option is highlighted with a green oval and includes the text: 'A FREE, no license required version of Quartus® II software for your CPLD or medium-density FPGA'. To the right of the main content are sections for 'Next Steps' (Download Software, License Software, Request Software DVD, Get Training), 'Buy Now' (Buy Software, Purchase Dev Kits, Buy Cables), and 'Support' (Get Software Support, View Knowledge Database).

ALTERA

Download Center Literature

Products End Markets Technology Training Support About Altera Buy Online

Logic Design

- Quartus II Subscription Edition
 - Quartus II Web Edition
 - ModelSim-Altera
 - What's New
- DSP Design**
 - DSP Builder
- Getting Started**
 - FPGAs & CPLDs
 - HardCopy ASIC
- Switching to Quartus II**
 - ASIC Users
 - MAX+PLUS II Users
- Partners**
 - EDA Partners

Design Software

Home > Products > Design Software

Logic Design

- Quartus II Subscription Edition Software**
The industry's #1 design software in performance and productivity
- Quartus II Web Edition Software**
A FREE, no license required version of Quartus® II software for your CPLD or medium-density FPGA

Next Steps

- Download Software
- License Software
- Request Software DVD
- Get Training

Buy Now

- Buy Software
- Purchase Dev Kits
- Buy Cables

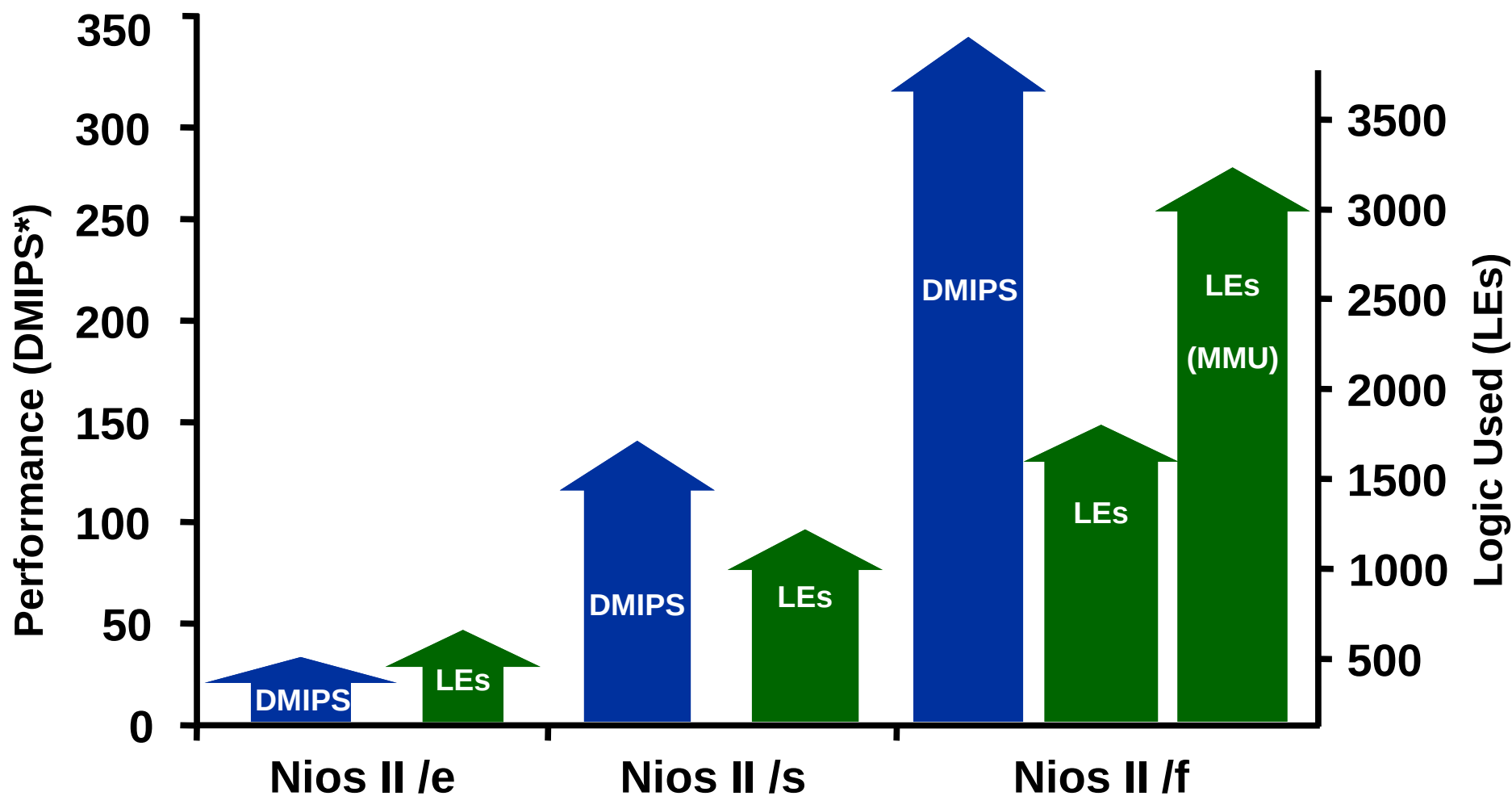
Support

- Get Software Support
- View Knowledge Database

Requirements for Nios II Processor Designs

- Latest Quartus II software version
 - Required for Nios II processor of same version number
- No spaces in Quartus II project pathname
- No spaces in installation path
 - Follow defaults → Install in **altera** directory
- Nios II processor license
 - or
- Programming cable tethered to PC to run OpenCore Plus version of the Nios II processor

Performance Range of Nios II Processor



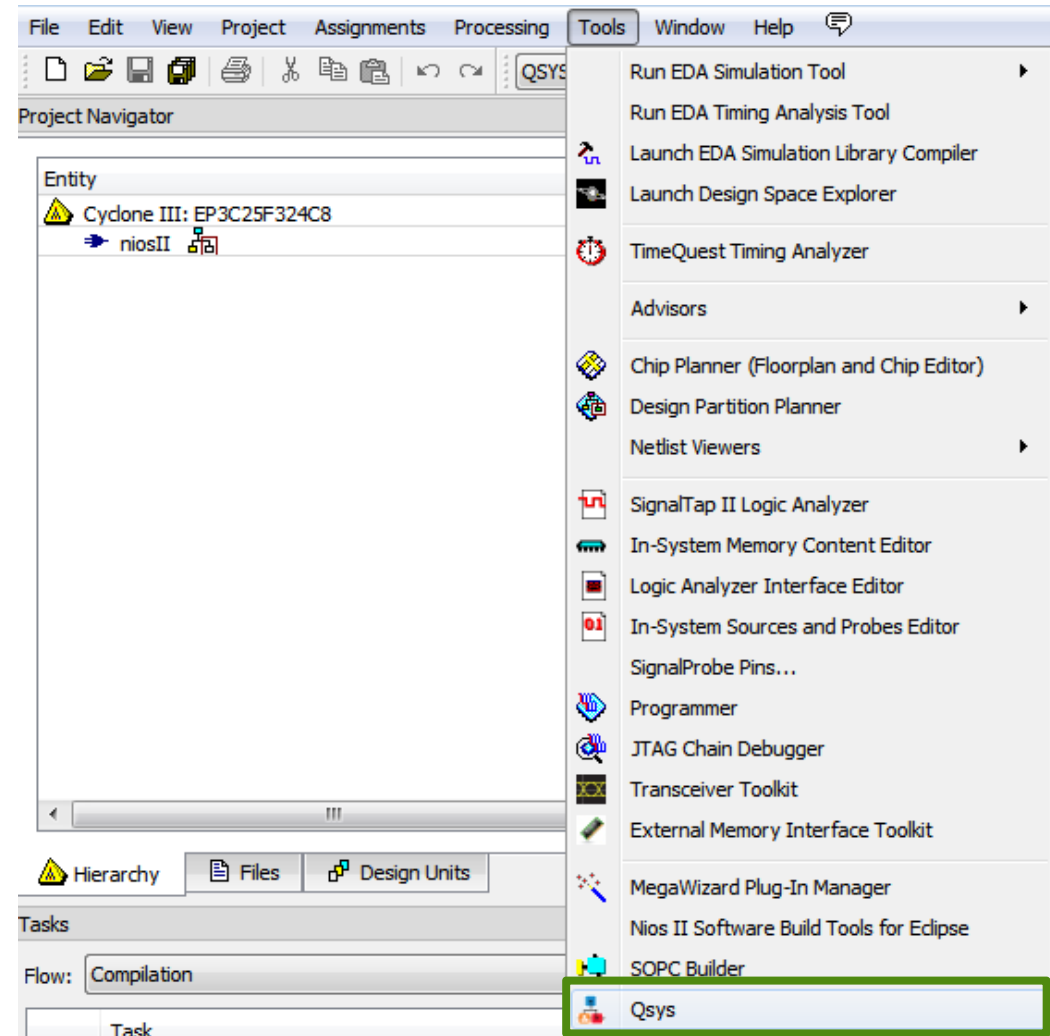
* Dhrystone 2.1 Benchmark

Nios II Processor Performance (DMIPS)

Device Family	Nios II /f	Nios II /s	Nios II /e
Std Cell ASIC (90 nm est.)	>500		
Stratix III / IV FPGAs (prelim)	340	140/150	48
Stratix II FPGAs	250	110	45
Hardcopy IV devices	288	163	39
Cyclone IV GX	233	102	33
Cyclone III FPGAs	195	90	30
Cyclone III LS FPGAs	186	80	27
Cyclone II FPGAs	145	55	18

Open Qsys from Quartus II Software

- See Quartus II software **Tools** Menu
- Select **Qsys**
- Open or create new Qsys system



Qsys System-Integration Tool

The screenshot displays the Qsys System-Integration Tool interface. The main window is titled "Qsys - sm_transfer_system.qsys* (C:\altera_trn\Qsys\Ex1\sm_transfer_system.qsys)". The interface is divided into several sections:

- Component Library:** Located on the left, it contains a tree view of components categorized by Project, Library, and various functional blocks like Clock Source, Reset Bridge, Avalon Verification Suite, etc.
- System Contents:** The central panel shows a hierarchical view of the system components. A green circle highlights the tabs at the top: "System Contents", "Address Map", "Clock Settings", "Project Settings", "System Inspector", "HDL Example", and "Generation". A green arrow points to the "System Contents" tab. The main area shows a list of components with their connections, names, descriptions, and properties.
- Messages:** Located at the bottom, it displays a list of messages, including warnings and errors. The current message is "3 Warnings".

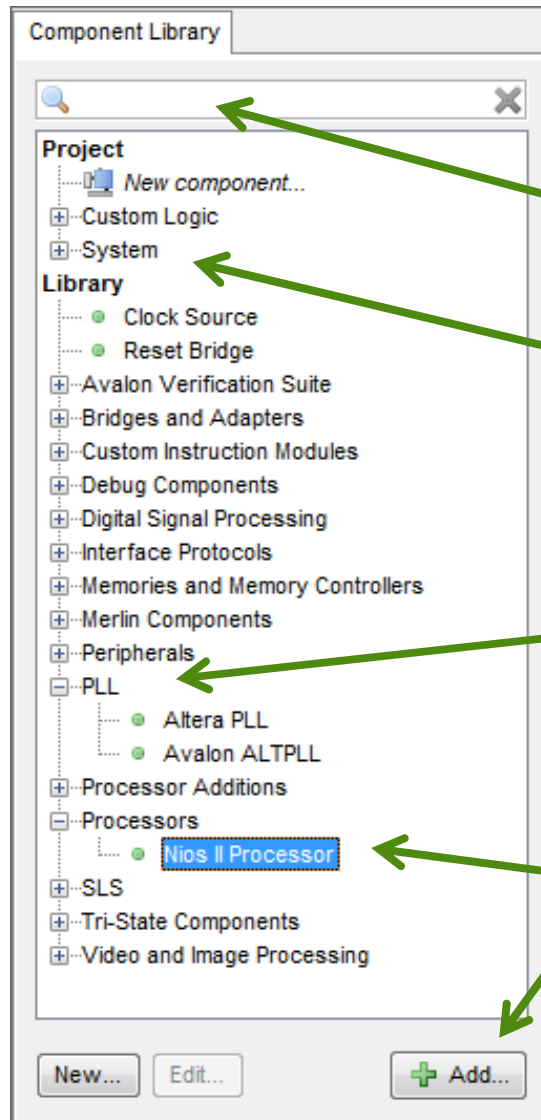
System Contents Table:

Use	Connections	Name	Description	Export	Clock	Base	End	IRC
☒		clk	Clock Source					
☒		clk_in	Clock Input	clk_clk_in				
☒		clk_in_reset	Reset Input	clk_clk_in_...				
☒		clk	Clock Output	Click to	clk			
☒		clk_reset	Reset Output	Click to				
☒		pll	Avalon ALTPLL					
☒		inclk_interface	Clock Input	Click to	clk			
☒		inclk_interface_reset	Reset Input	Click to	[inclk_interface]			
☒		pll_slave	Avalon Memory Mapped Slave	Click to	[inclk_interface]	0x00000010	0x0000001f	
☒		c0	Clock Output	Click to	sys_clk			
☒		c1	Clock Output	Click to	ssram_clk			
☒		locked_conduit	Conduit	Click to				
☒		phasedone_conduit	Conduit	Click to				
☒		start_pushbutton	PIO (Parallel IO)		sys_clk	0x00000060	0x0000006f	
☒		av_sm_master	Avalon State Machine Master					
☒		clock	Clock Input	Click to	sys_clk			
☒		reset	Reset Input	Click to	[clock]			
☒		avalon_master	Avalon Memory Mapped Master	Click to	[clock]			
☒		led_out	LED Flasher		sys_clk			
☒		led_timing_adapter	Avalon-ST Timing Adapter		sys_clk			
☒		led_fifo	On-Chip FIFO Memory		sys_clk	0x00000000	0x00000007	
☒		test_mem	On-Chip Memory (RAM or ROM)		sys_clk	0x00002000	0x00003fff	
☒		dma_ssram_to_l...	DMA Controller		sys_clk	0x00000020	0x0000003f	
☒		sram_controller	Generic Tri-State Controller		sys_clk	0x00000000	0x000fffff	
☒		ext_mem_bus	Tri-State Conduit Bridge					

Messages:

Description	Path
3 Warnings	
This memory is not initialized during device programming.	System.test_mem
pll.locked_conduit must be exported, or connected to a matching conduit.	System.pll
pll.phasedone_conduit must be exported, or connected to a matching conduit.	System.pll
0 Errors, 3 Warnings	

Component Library



- Lists available IP and systems

Type search string to filter the list

Reuse previous systems – hierarchy (discussed later)

Expand categories to browse components

Double-click component or click Add button to add selected component to system

Qsys UI: System Contents Tab

- Displays components and subsystems added to system
- Use to add/remove component and connect to system

The screenshot shows the Qsys UI with the 'System Contents' tab selected. The tab bar at the top includes 'System Contents', 'Address Map', 'Clock Settings', 'Project Settings', 'System Inspector', 'HDL Example', and 'Generation'. The main area displays a table of components and their connections.

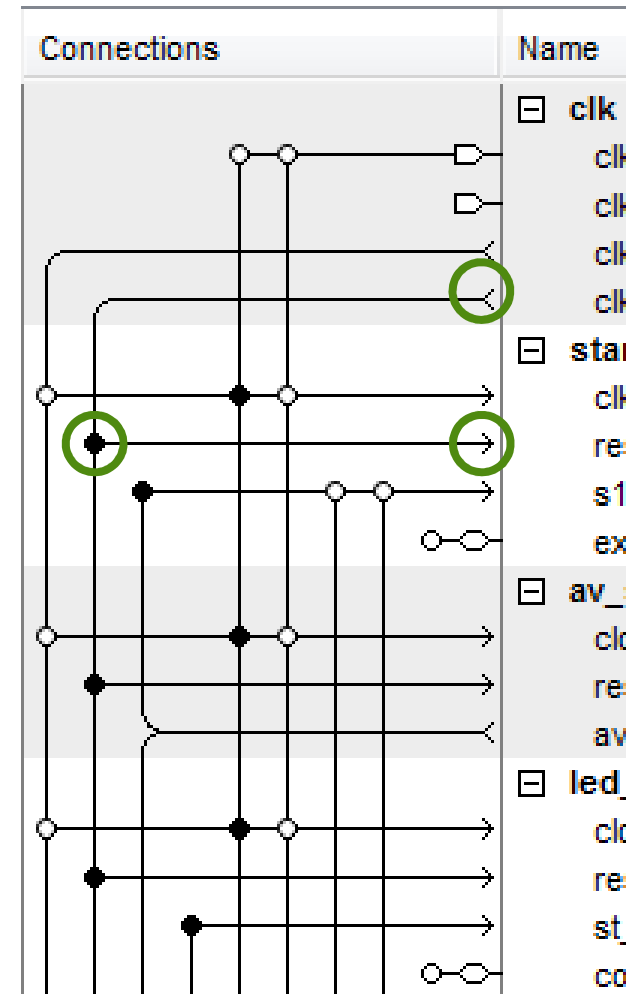
Use	Connections	Name	Description	Export	Clock	Base
☒		☒ clk	Clock Source			
		clk_in	Clock Input	clk_clk_in		
		clk_in_reset	Reset Input	clk_clk_in_...		
		clk	Clock Output	Click to	clk	
		clk_reset	Reset Output	Click to		
☒		☒ start_pushbutton	PIO (Parallel I/O)			
		clk	Clock Input	Click to	sys_clk	
		reset	Reset Input	Click to	[clk]	
		s1	Avalon Memory Mapped Slave	Click to	[clk]	0x00000060
		external_connection	Conduit Endpoint	start_push...		
☒		☒ av_sm_master	Avalon State Machine Master			
		clock	Clock Input	Click to	sys_clk	
		reset	Reset Input	Click to	[clock]	
		avalon_master	Avalon Memory Mapped Master	Click to	[clock]	
☒		☒ led_out	LED Flasher			
		clock	Clock Input	Click to	sys_clk	
		reset	Reset Input	Click to	[clock]	
		led_out.reset	Avalon Streaming Sink			
		Conduit	Conduit			

Annotations in the image:

- A green circle highlights the 'System Contents' tab.
- A green arrow points from the text 'System Components' to the 'Use' column.
- A green arrow points from the text 'Enable/disable components' to the 'Connections' column.
- A green arrow points from the text 'Enable/disable components' to the 'led_out.reset' component.

System Contents: Connections Panel

- You use the Connections panel to specify interface connectivity
 - Clocks, resets
 - Master to slaves
 - Sources to sinks
 - Interrupt senders and receivers
 - Custom instruction senders and receivers
- Each dot represents a connection between two interfaces
 - Qsys generates the system interconnect based on this information
 - Design changes that normally take days become mouse clicks
- Connection direction shown with arrows at start and end points
- Hide connections for added readability
 - Collapsing components
 - Using filters



System Contents: Addressing

- Each memory-mapped master interface has own address map
 - When slave ports are shared, the address map converges
 - Maximum 32-bit address space (4GB) for each master interface
- Master address map is a collection of the following
 - Connected slave interface base addresses
 - Connected slave interface address spans (determines end address)
 - Lowest and highest slave addresses make up the address space of the master
- Manually assign slave addresses
 - Double-click or let Qsys auto-assign

Master interface



→ ⊕ av_sm_master	Avalon State Machine Master	sys_clk		
→ ⊕ led_fifo	On-Chip FIFO Memory	sys_clk	0x00000000	0x00000007
→ ⊕ dma_ssramm_to_led	DMA Controller	sys_clk	0x00000020	0x0000003f
→ ⊕ dma_source_to_ssram	DMA Controller	sys_clk	0x00000040	0x0000005f
→ ⊕ start_pushbutton	PIO (Parallel I/O)	sys_clk	0x00000060	0x0000006f

Master address space
based on connected slaves



Qsys UI: Address Map Tab

- Table of memory-mapped addresses
- Double-click cell to manually edit slave addressing
- Supports per-master addressing for shared slaves
 - Single slave represented by different address ranges for different masters

Master interfaces represented by columns

	dma_ssramm_to_led.read_master	dma_ssramm_to_led.write_master	av_sm_master.avalon_master
source.s1			
dma_source_to_ssram.control_port_slave			0x00000040 - 0x0000005f
dma_ssramm_to_led.control_port_slave			0x00000020 - 0x0000003f
led_fifo.in		0x00000040 - 0x00000047	0x00000000 - 0x00000007
sram_controller.uas	0x00000000 - 0x000fffff		
pll.pll_slave			0x00000010 - 0x0000001f
test_mem.s1		0x00002000 - 0x00003fff	
start_pushbutton.s1			0x00000060 - 0x0000006f

Slave interfaces represented by rows

Qsys UI: Clock Settings Tab

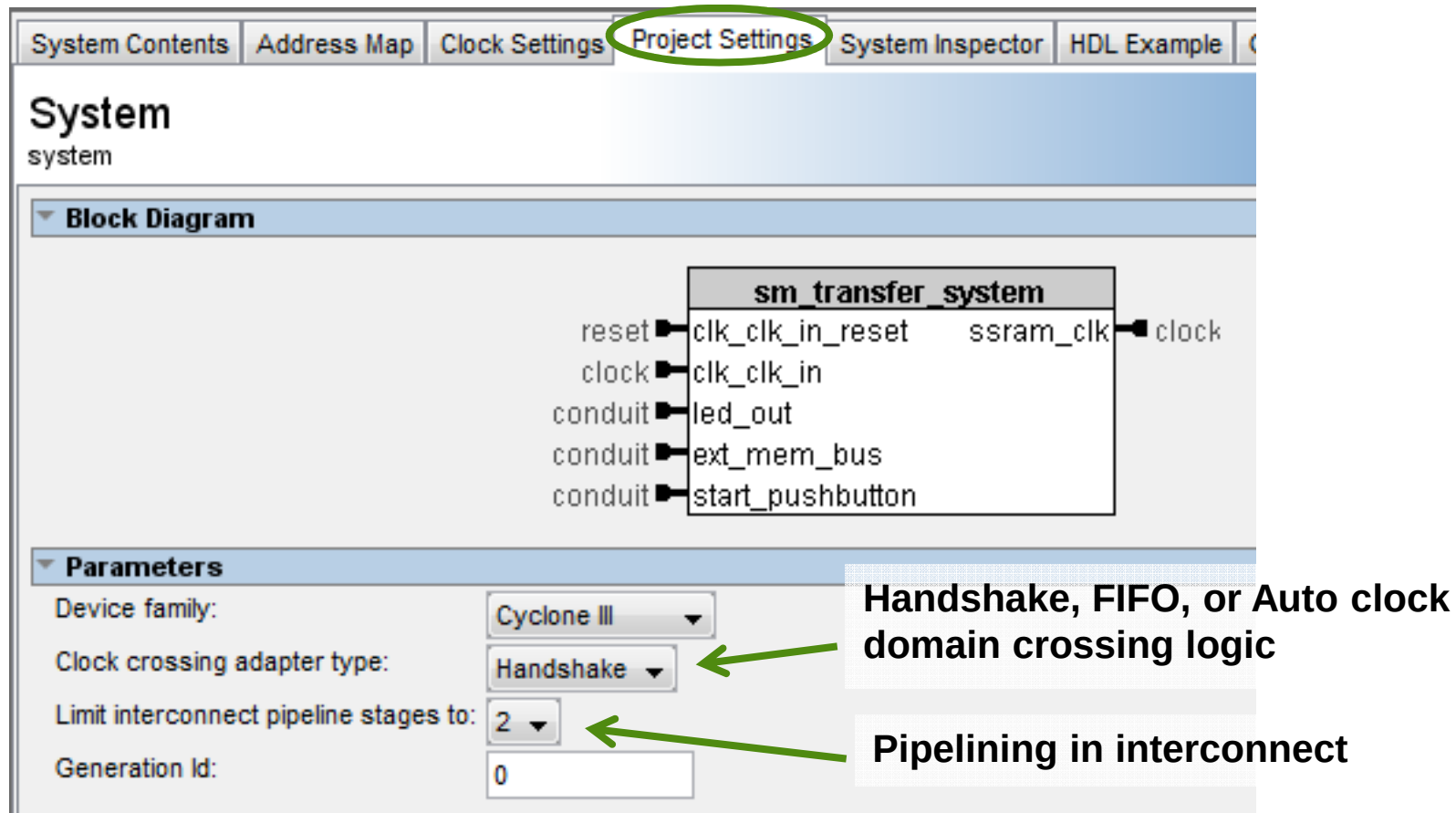
- Use for further clock management
- Add new clocks to system
- Rename system clocks for readability
- Specify clock frequencies
 - You must also create proper TimeQuest SDC constraints

Clock Settings		
Name	Source	MHz
clk	External	50.0
sys_clk	pll.c0	100.0
ssram_clk	pll.c1	100.0

Double-click to change name
(all) or frequency (non-PLL)

Qsys UI: Project Settings Tab

■ Control interconnect implementation



The screenshot displays the Qsys UI interface with the **Project Settings** tab selected. The **Block Diagram** section shows a block named **sm_transfer_system** with the following connections:

- reset to clk_clk_in_reset
- clock to clk_clk_in
- conduit to led_out
- conduit to ext_mem_bus
- conduit to start_pushbutton
- ssram_clk to clock

The **Parameters** section shows the following settings:

- Device family: Cyclone III
- Clock crossing adapter type: Handshake
- Limit interconnect pipeline stages to: 2
- Generation Id: 0

Annotations with green arrows point to the **Handshake** dropdown and the **2** value in the **Limit interconnect pipeline stages to** field, with the following text:

- Handshake, FIFO, or Auto clock domain crossing logic
- Pipelining in interconnect

Qsys UI: System Inspector Tab

- Review system and component details
 - System hierarchy
 - Top-level system connection
 - Component interfaces
 - Connections between components
 - Component details
- Edit component settings

System Inspector – System View

System Contents | Address Map | Clock Settings | Project Settings | **System Inspector** | HDL Example | Generation

System
system

Block Diagram

reset clk_clk_in_reset ssram_clk clock
clock clk_clk_in
conduit led_out
conduit ext_mem_bus
conduit start_pushbutton

Parameters

Device family: Cyclone III
Clock crossing adapter type: Handshake
Limit interconnect pipeline stages to: 2
Generation Id: 0

Top-level symbol diagram

System level settings

System Inspector – Component View

The screenshot displays the System Inspector interface with the 'System Inspector' tab selected. On the left, the 'Submodules' list shows a tree of components under 'sm_transfer_system'. The 'dma_source_to_ssram' component is highlighted with a green box. A green arrow points from the text 'All system components' to this box. On the right, the 'DMA Controller' component is selected, showing its 'Block Diagram' and 'Parameters'.

All system components

Selected component parameter editor settings*

Block Diagram

clock clk irq interrupt
reset reset read_master avalon
avalon control_port_slave write_master avalon

Parameters

actualDmaTransactionRegisterWidth: 20

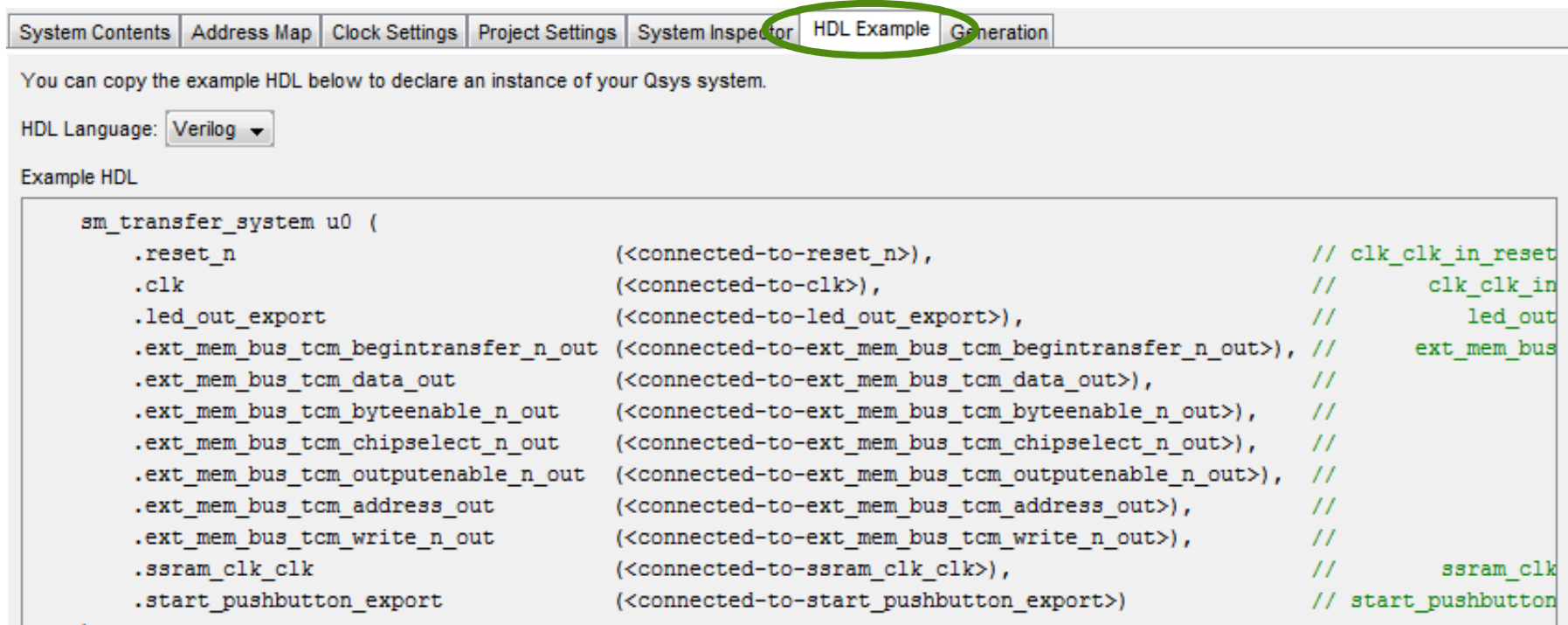
☒ allowByteTransactions
☒ allowDoubleWordTransactions
☒ allowHalfWordTransactions
☒ allowQuadWordTransactions
☒ allowWordTransactions
☐ burstEnable

fifoDepth: 32
maxBurstSize: 1
minimumDmaTransactionRegisterWidth: 13
minimumNumberOfByteTransfers: 8191

* Some wizard settings can be edited

Qsys UI: HDL Example Tab

- Creates Verilog or VHDL instantiation template for Qsys system
- Use to instantiate Qsys system as sub-module in design



Qsys UI: Generation Tab

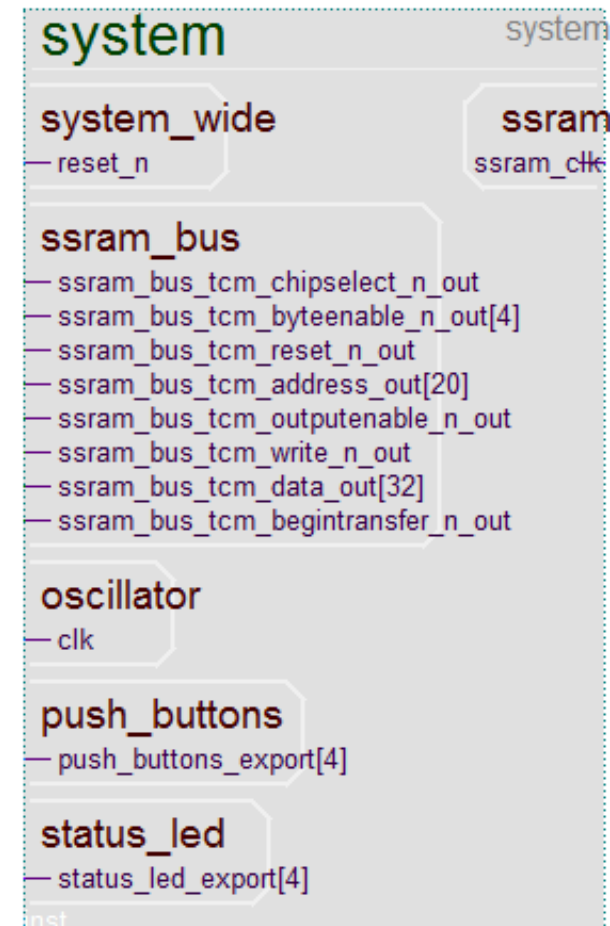
- Choose what files to generate (Simulation, synthesis, or symbol)
- Choose **Output Directory Path** (default is subdirectory)
- Click **Generate** button (bottom of tab)

The screenshot shows the Qsys UI Generation tab. The 'Generation' tab is selected and circled in green. Below it, three sections are visible: 'Simulation', 'Synthesis', and 'Output Directory', each with its title circled in green. In the 'Simulation' section, three dropdown menus are set to 'None'. In the 'Synthesis' section, the checkbox 'Create HDL design files for synthesis' is checked, while 'Create block symbol file (.bsf)' is unchecked. In the 'Output Directory' section, the 'Path' field contains 'C:/altera_trn/Qsys/Ex1/sm_transfer_system' and is followed by a browse button (...). Below the 'Path' field are three empty text boxes for 'Simulation:', 'Testbench:', and 'Synthesis:'. The 'Synthesis:' field contains the path 'C:/altera_trn/Qsys/Ex1/sm_transfer_system/synthesis/'. At the bottom of the tab is a 'Generate' button.

System Contents	Address Map	Clock Settings	Project Settings	System Inspector	HDL Example	Generation
Simulation						
Create simulation model:		None				
Create testbench Qsys system:		None				
Create testbench simulation model:		None				
Synthesis						
☒ Create HDL design files for synthesis						
☐ Create block symbol file (.bsf)						
Output Directory						
Path:		C:/altera_trn/Qsys/Ex1/sm_transfer_system				...
Simulation:						
Testbench:						
Synthesis:		C:/altera_trn/Qsys/Ex1/sm_transfer_system/synthesis/				
Generate						

Qsys Output Files

- **Verilog (.v)** synthesis files
- **Verilog** simulation files
- **Block Symbol File (.bsf)** - top level schematic
- **.QIP** file - IP file for system
 - Add to Quartus II project
- **.QSYS** file - system archive file
- **.SOPCinfo** file - describes hardware system for Nios II software development tools
- **.ptf** file – for legacy Nios II IDE



.SOPCinfo File

■ Needed by Nios II software tools

- Text file that describes and archives Qsys system contents

■ Contains

- Project Name and Qsys tool version
- HDL Language, component names
- File locations on disk
- Module names and versions
- Interface information, including signal names, types, properties
- Parameter names and values
- Information about each connection
 - Component and interface connections
 - Memory-map address seen by each master, IRQ Numbers (IRQs), etc.

Qsys UI: Messages

- Documents system error, warning and information messages
- Includes summary of error and warning count

Messages	
Description	Path
✖ 1 Error	
✖ dma_ssramm_to_led.control_port_slave (0x20..0x3f) overlaps pll.pll_slave (0x20..0x2f)	System.av_sm_master.avalon_master
⚠ 1 Warning	
⚠ This memory is not initialized during device programming.	System.test_mem
ℹ 4 Info Messages	
ℹ User is required to provide memory initialization files for memory .	System.source
ℹ Memory will be initialized from C:\Designs\my_test_designs\qsys\qsys_no_nios_ver_sram\source2.hex	System.source
ℹ For Avalon-ST and Avalon-MM combinations, data width is 32 bits.	System.led_fifo
ℹ PIO inputs are not hardwired in test bench. Undefined values will be read from PIO inputs during simulation.	System.start_pushbutton
1 Error, 1 Warning	

Add and Configure Nios II CPU

Block Diagram

Block diagram showing the Nios II Processor (nios2_qsys_0) connected to various components:

- clock: clk
- reset: reset_n
- interrupt: d_irq
- avalon: jtag_debug_module
- data_master: avalon
- instruction_master: avalon
- jtag_debug_module_reset: reset
- custom_instruction_master: nios_custom_instruction

Select a Nios II Core Implementation

Nios II Core: ☐ Nios II/e ☐ Nios II/s ☒ Nios II/f

	Nios II/e	Nios II/s	Nios II/f
Nios II Selector Guide	RISC 32-bit	RISC 32-bit Instruction Cache Branch Prediction Hardware Multiply Hardware Divide	RISC 32-bit Instruction Cache Branch Prediction Hardware Multiply Hardware Divide Barrel Shifter Data Cache Dynamic Branch Prediction
Memory Usage (e.g. Stratix IV)	Two M9Ks (or equiv.)	Two M9Ks + cache	Three M9Ks + cache

Hardware Arithmetic Operation

Hardware multiplication type:

☐ Hardware divide

Messages

6 Errors, 0 Warnings

- Error: nios2_qsys_0: "Reset vector memory" (resetSlave) out of range
- Error: nios2_qsys_0: "Exception vector memory" (exceptionSlave) out of range
- Error: nios2_qsys_0: Reset slave is not specified. Please select the reset slave
- Error: nios2_qsys_0: Exception slave is not specified. Please select the exception slave

Cancel Finish

Multiple Configuration Settings Tabs

Add and Configure Nios II CPU

Component Library

- Project
 - New component...
 - Custom Logic
 - System
- Library
 - Clock Source
 - Reset Bridge
 - Avalon Verification Suite
 - Bridges and Adapters
 - Custom Instruction Modules
 - Debug Components
 - Digital Signal Processing
 - Interface Protocols
 - Memories and Memory Controllers
 - Merlin Components
 - Peripherals
 - PLL
 - Altera PLL
 - Avalon ALTPLL
 - Processor Additions
 - Processors
 - Nios II Processor**

Nios II Processor - nios2_qsys_0

Hardware Arithmetic Operation

Hardware multiplication type: Embedded Multipliers

☐ Hardware divide

Reset Vector

Reset vector memory: None

Reset vector offset: 0x00000000

Reset vector: 0x00000000

Exception Vector

Exception vector memory: None

Exception vector offset: 0x00000020

Exception vector: 0x00000000

MMU and MPU

☐ Include MMU

Only include the MMU using an operating system that explicitly supports an MMU.

Fast TLB Miss Exception vector memory: None

Fast TLB Miss Exception vector offset: 0x00000000

Fast TLB Miss Exception vector: 0x00000000

☐ Include MPU

Messages

6 Errors

- "Reset vector memory" (resetSlave) out of range
- "Exception vector memory" (exceptionSlave) out of range
- Reset slave is not specified. Please select the reset slave

6 Errors, 0 Warnings

Errors:

- Error: nios2_qsys_0: "Reset vector memory" (resetSlave) out of range
- Error: nios2_qsys_0: "Exception vector memory" (exceptionSlave) out of range
- Error: nios2_qsys_0: Reset slave is not specified. Please select the reset slave
- Error: nios2_qsys_0: Exception slave is not specified. Please select the exception slave

Annotations:

- Choose Multiplier Implementation
- Define Reset Vector Location
- Define Exception Vector Location
- Add MMU or MPU (only Nios II "Fast" Processor Core)

Reset and Exception Addresses

- Set after memory components added to system
- Reset address
 - Non-volatile memory recommended
 - Volatile memory requires a boot loader
- Exceptions processed at “exception location”
 - Exception handler code provided by HAL system library
 - Software Exceptions
 - Software Traps (currently, not implemented)
 - Unimplemented instructions
 - Maintains compatibility between Nios II processor cores
 - Hardware Interrupts
 - Internal interrupt controller supports 32 Level-sensitive interrupts
 - External interrupt controller supports unlimited number of interrupts

Select Cache and TCM Settings

Nios II Processor
altera_nios2_qsys

Block Diagram

clock → clk
reset → reset_n
interrupt → d_irq
avalon → jtag_debug_module

nios2_qsys_0

data_master → avalon
instruction_master → avalon
jtag_debug_module_reset → reset
custom_instruction_master → nios_custom_instruction

Core Nios II | **Caches and Memory Interfaces** | Advanced Features | MMU and MPU Settings | JTAG Debug Module | Custom Instruction

Instruction Master

Instruction cache: 4 Kbytes
Burst transfers (burst size = 32 bytes): Disable
Number of tightly coupled instruction master port(s): None

Data Master

☐ Omit data master port

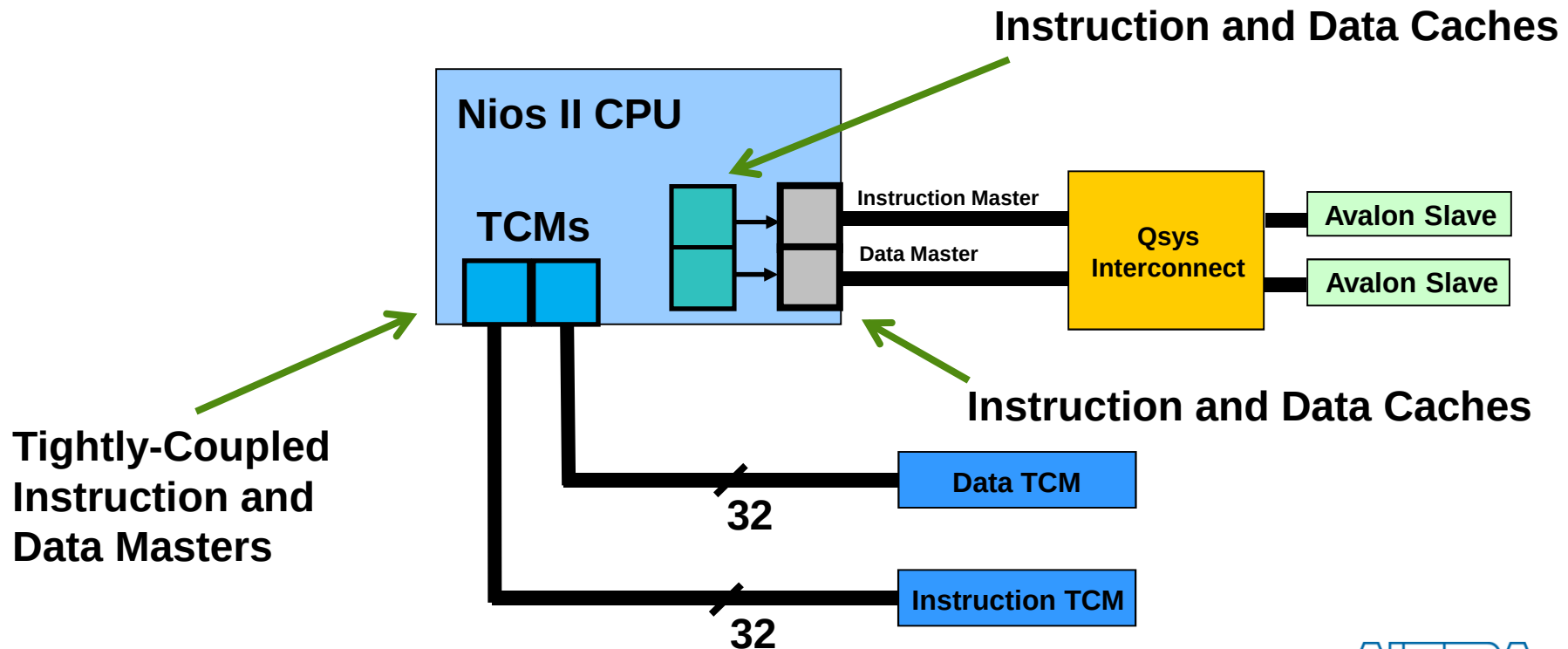
Data cache: 2 Kbytes
Data cache line size: 32 Bytes
Bursts transfers (burst size = data cache line size): Disable
Number of tightly coupled data master port(s): None

Configure Instruction Master
- Cache size, burst and tightly-coupled memory support

Configure Data Master
- Cache properties, burst, and tightly-coupled memory support

Tightly Coupled Memory

- Tightly-coupled memory accessed in parallel with cache
 - Act like cache with a 100% hit rate
 - Great for ISRs and other time critical functions or data
 - Address decoders in CPU determine if address resides in TCM or normal system address range - assigning TCM's to high address space can increase Fmax



Advanced Features Tab

Nios II Processor - nios2_qsys_0

Nios II Processor
altera_nios2_qsys

Block Diagram

clock → clk
reset → reset_n
interrupt → d_irq
avalon → jtag_debug_module

nios2_qsys_0

data_master → avalon
instruction_master → avalon
jtag_debug_module_reset → reset
custom_instruction_master → nios_custom_instruction

Core Nios II | Caches and Memory Interfaces | **Advanced Features** | MMU and MPU Settings | JTAG Debug Module | Custom Instruction

General

Interrupt controller: Internal

Number of shadow register sets (0-63): 0

External interrupt controller and shadow register set only supported in Nios II/f core. Internal interrupt controller is the only option for Nios II/e and Nios II/s core.

☐ Include cpu_resetrequest and cpu_resettaken signals

These signals appear on the top-level Qsys system. You must manually connect these signals to logic external to the Qsys system.

☐ Assign cpuid control register value manually

cpuid control register value: 0x00000000

Select internal or external (i.e. vectored) interrupt controller, include CPU reset signals, assign CPU id

Selecting Vectored Interrupt Controller

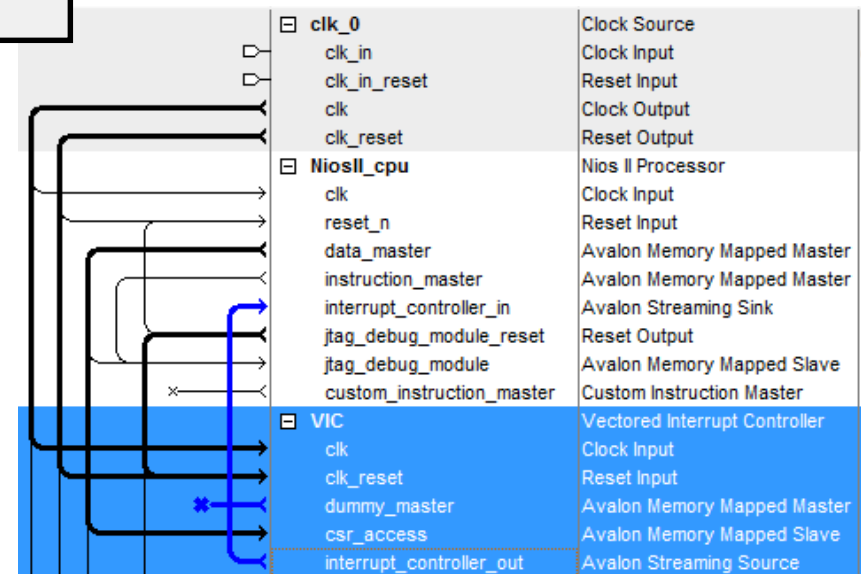
General

Interrupt controller: External ▼

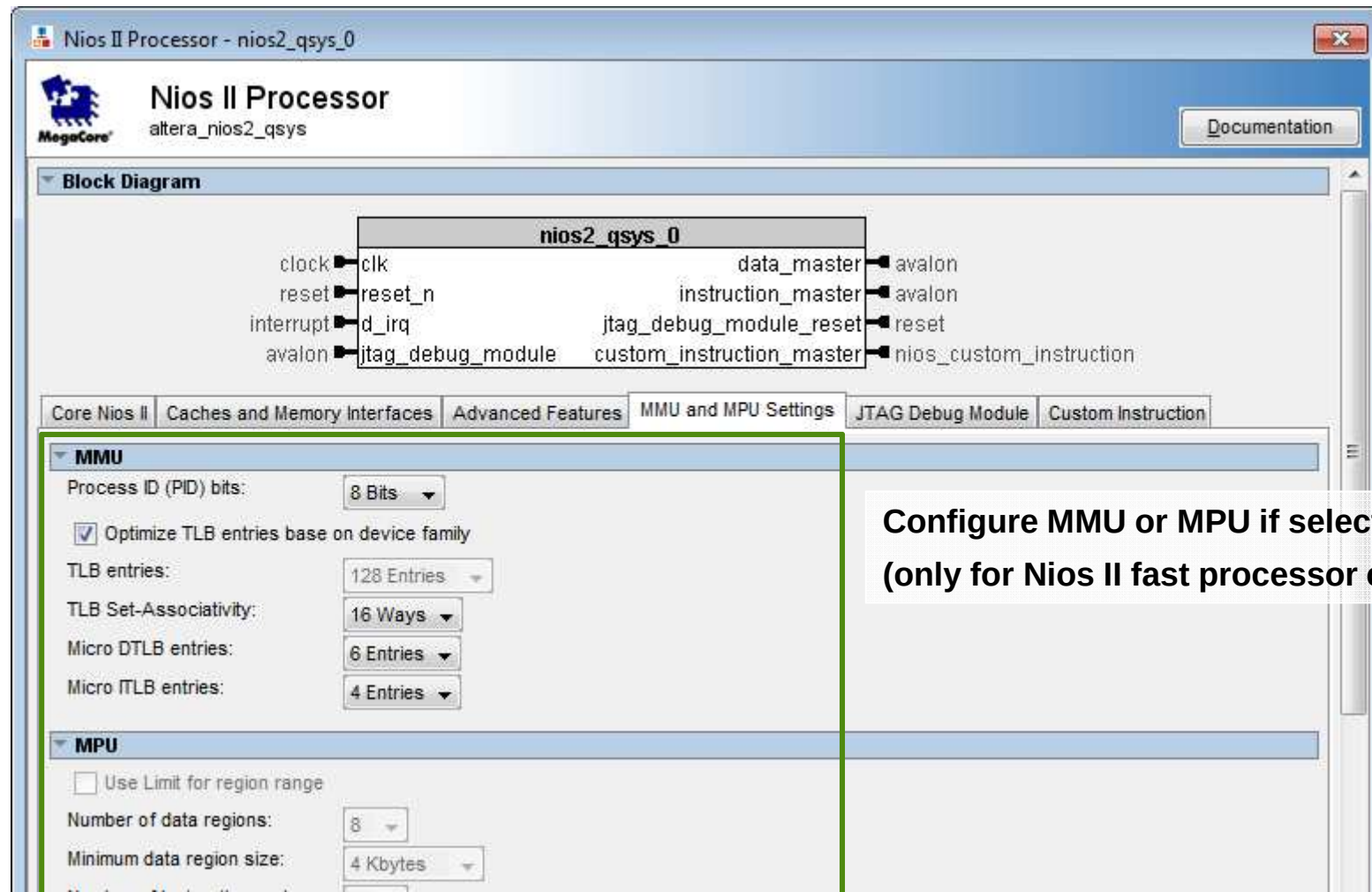
Number of shadow register sets (0-63):

External interrupt controller and shadow register set only supported in Nios II/f core.
Internal interrupt controller is the only option for Nios II/e and Nios II/s core.

- Requires Nios II /f processor
- Select External Interrupt Controller
- Set # of shadow register sets
 - One for each high performance interrupt
- Manually instantiate and connect VIC peripheral to Nios II CPU
 - Lower IRQ # → higher priority interrupt (same as Internal Interrupt Controller)
 - Have option of cascading multiple VICs
 - Assign individual VIC priority in software development tools



MMU and MPU Settings



Choose JTAG Debug Core

Nios II Processor - nios2_qsys_0

Nios II Processor
altera_nios2_qsys

Block Diagram

clock → clk data_master → avalon
 reset → reset_n instruction_master → avalon
 interrupt → d_irq jtag_debug_module_reset → reset
 avalon → jtag_debug_module custom_instruction_master → nios_custom_instruction

Core Nios II Caches and Memory Interfaces Advanced Features MMU and MPU Settings **JTAG Debug Module** Custom Instruction

Select a Debugging Level

Debug level:

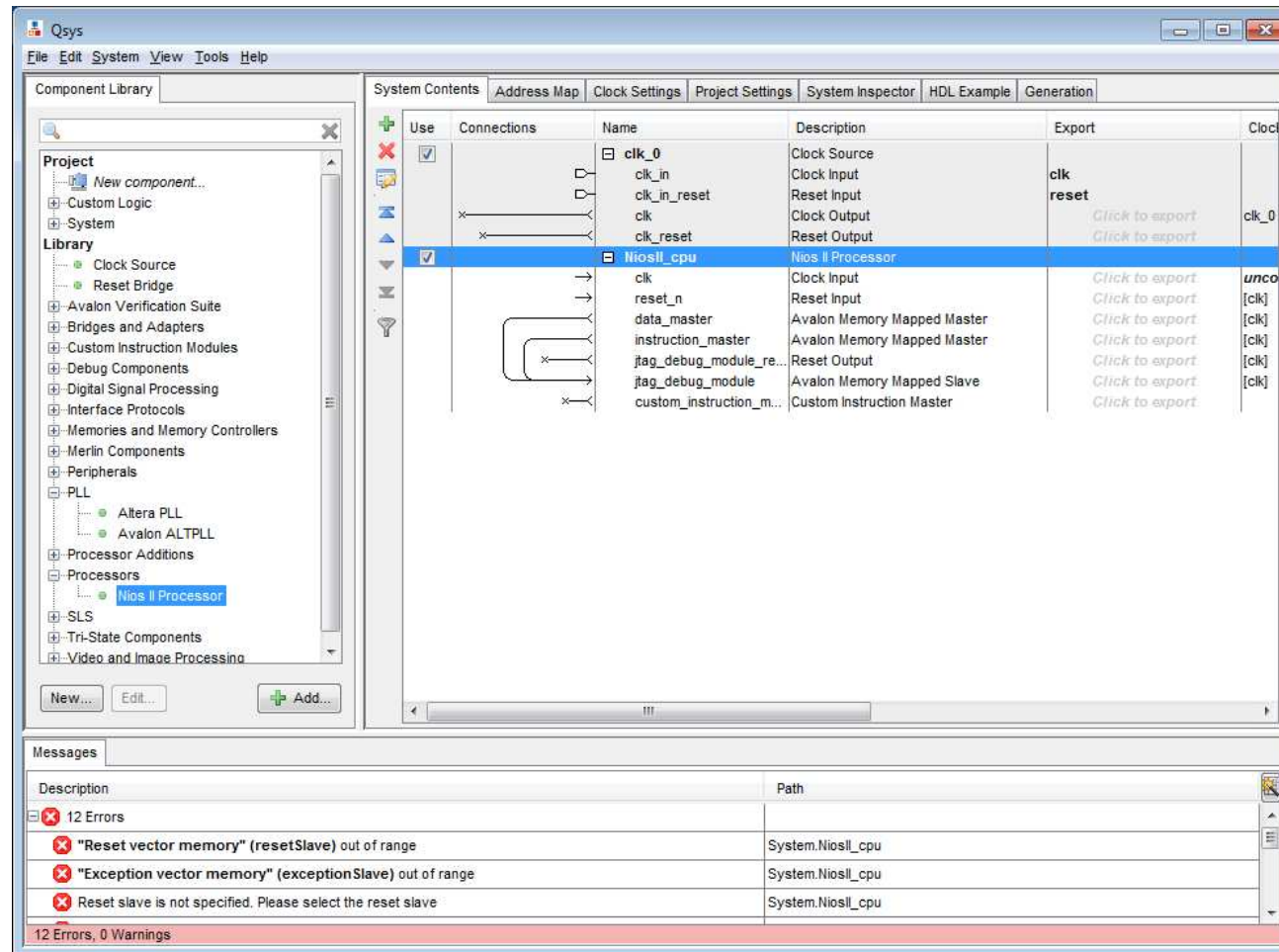
- ☐ No Debugger
- ☒ Level 1
- ☐ Level 2
- ☐ Level 3
- ☐ Level 4

No Debugger	Level 1	Level 2	Level 3	Level 4
	JTAG Target Connection Download Software Software Breakpoints	JTAG Target Connection Download Software Software Breakpoints 2 Hardware Breakpoints 2 Data Triggers	JTAG Target Connection Download Software Software Breakpoints 2 Hardware Breakpoints 2 Data Triggers	JTAG Target Connection Download Software Software Breakpoints 4 Hardware Breakpoints 4 Data Triggers

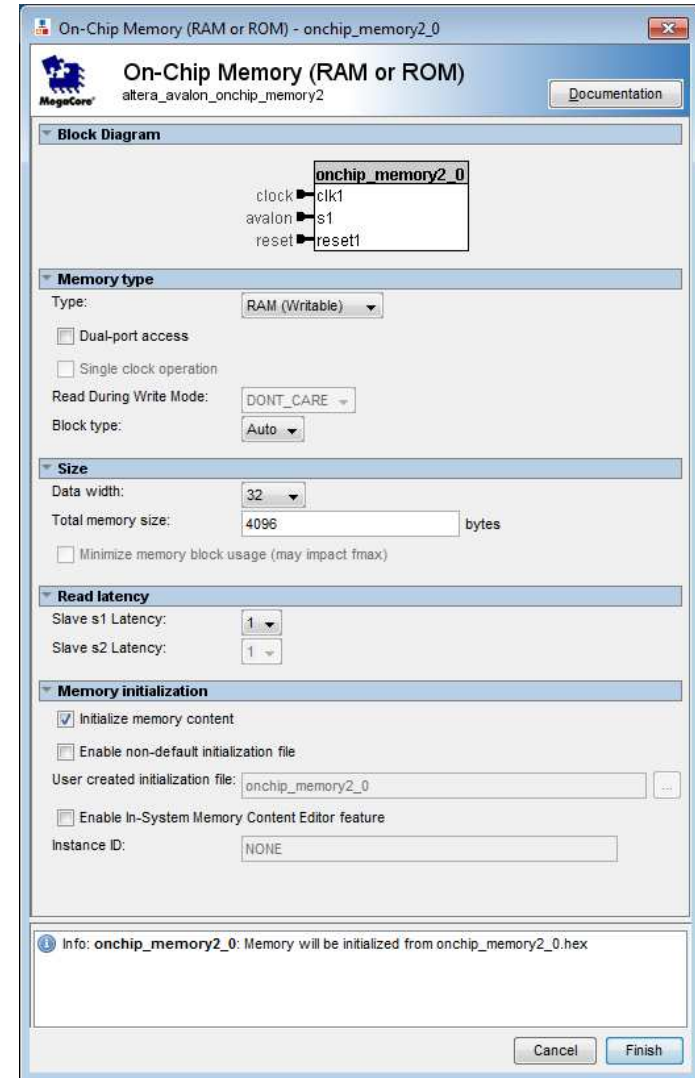
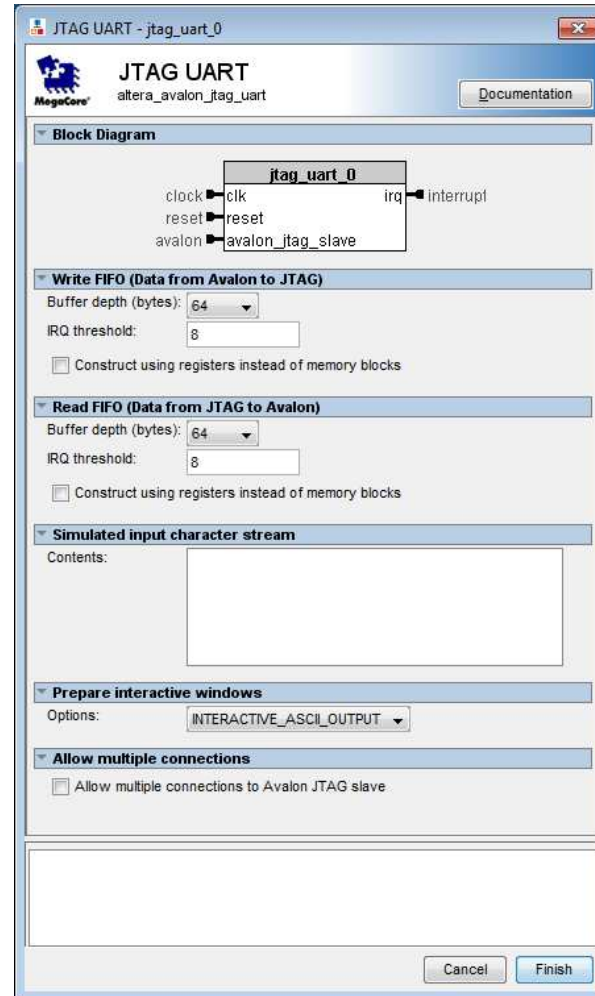
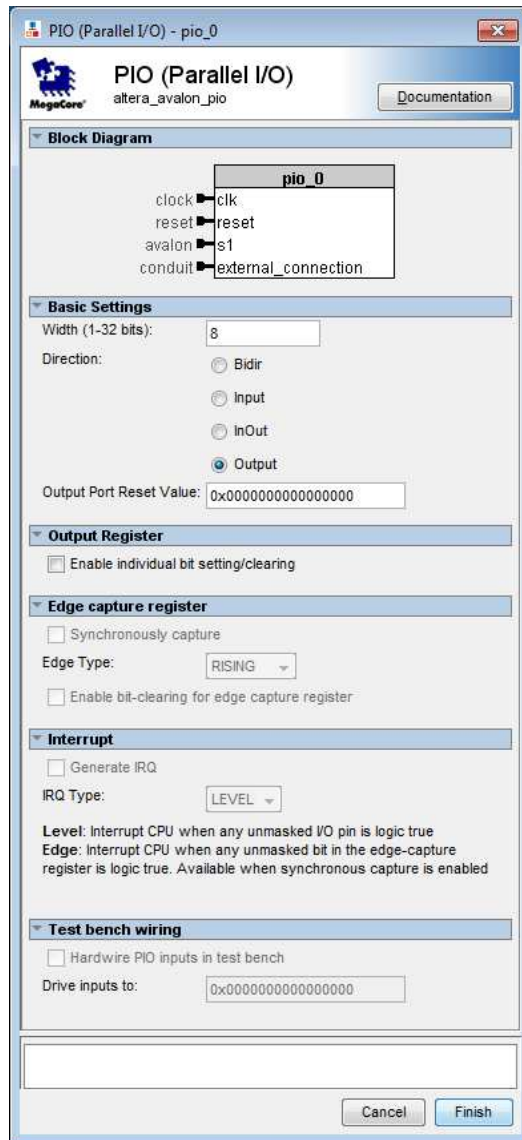
Select level of debug capability desired (capabilities for each shown)

CPU Added to System

- Right-click on peripheral to rename



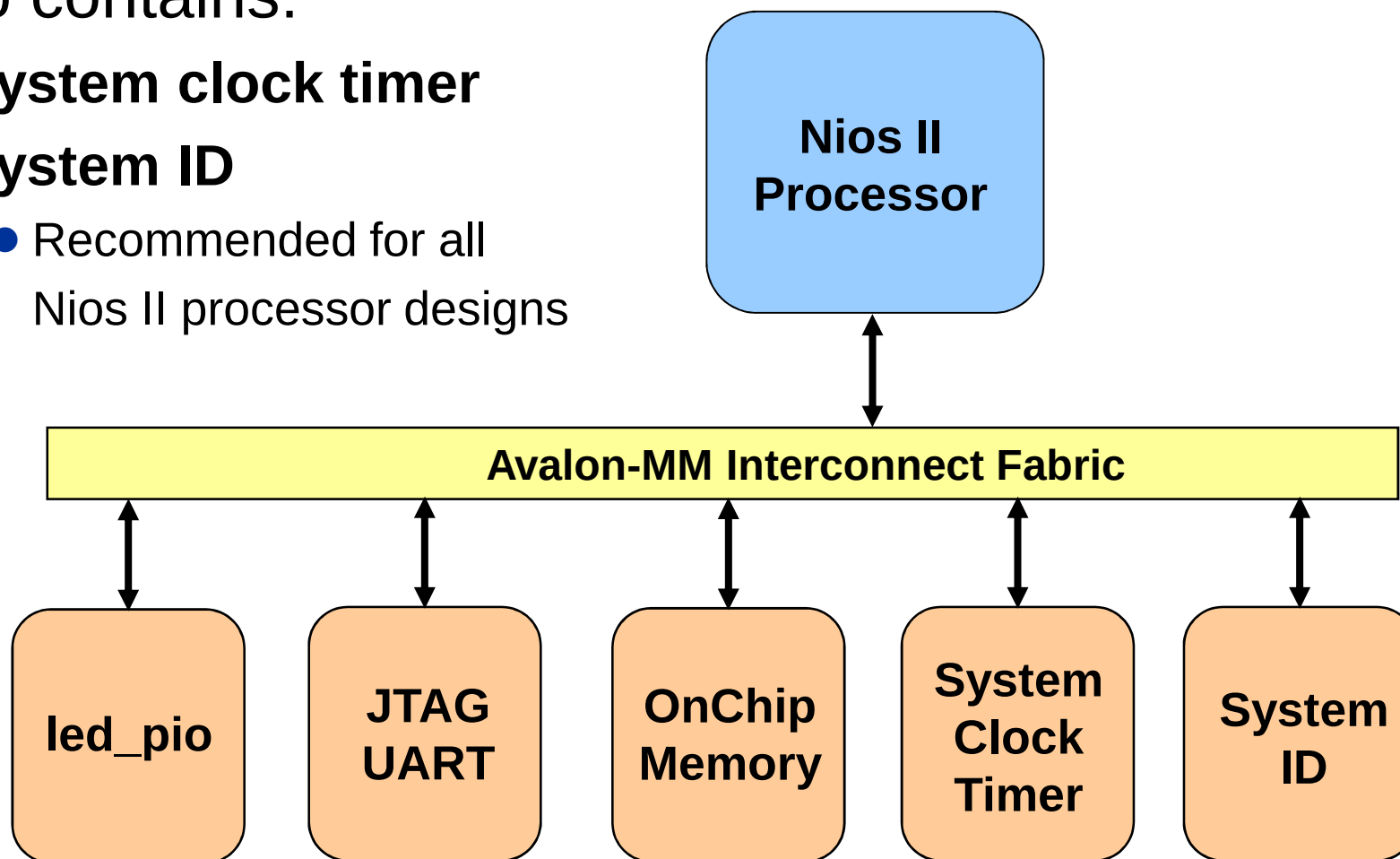
Other Example Useful Component's GUIs



Example System Block Diagram

■ Also contains:

- **System clock timer**
- **System ID**
 - Recommended for all Nios II processor designs



Connecting Up the System

The screenshot shows the Qsys software interface with the 'System Contents' tab selected. The 'Component Library' on the left lists various components like Clock Source, Reset Bridge, and Nios II Processor. The main area displays a system diagram with connections between components. A table on the right lists the components and their properties.

Export whatever signals you like

Connect all clocks

Connect component interfaces through drop-down menus

Connect all IRQs

Can also make connections by clicking on the dots in the Qsys patch panel

Use	Connections	Name	Description	Export	Clock	Base	End	IRQ
☒		clk_0	Clock Source	clk	clk_0			
☒		clk_in	Clock Input	Click to export	clk_0			
☒		clk_in_reset	Reset Input	Click to export	clk_0			
☒		clk_reset	Reset Output	Click to export	clk_0			
☒		NiosII_cpu	Nios II Processor					
☒		reset_n	Reset Input	Click to export	clk_0			
☒		data_master	Avalon Memory Mapped Master	Click to export	clk_0			
☒		instruction_master	Avalon Memory Mapped Master	Click to export	clk_0			
☒		jtag_debug_module_reset	Reset Input	Click to export	clk_0			
☒		jtag_debug_module	Avalon Memory Mapped Slave	Click to export	clk_0			
☒		custom_instruction_master	Reset Input	Click to export	clk_0			
☒		led_pio	LED (Parallel IO)	led_export	clk_0			
☒		reset	Reset Input	Click to export	clk_0			
☒		s1	Avalon Memory Mapped Slave	Click to export	clk_0	0x00000000	0x000000ff	
☒		external_connection	Conduit Endpoint	Click to export	clk_0			
☒		jtag_uart	JTAG UART					
☒		clk	Clock Input	Click to export	clk_0			
☒		reset	Reset Input	Click to export	clk_0			
☒		avalon_jtag_slave	Avalon Memory Mapped Slave	Click to export	clk_0	0x00000000	0x00000007	
☒		onchip_memory	On-Chip Memory (RAM or ROM)	Click to export	clk_0			
☒		clk1	Clock Input	Click to export	clk_0			
☒		s1	Avalon Memory Mapped Slave	Click to export	clk_1	0x00000000	0x000000ff	
☒		reset1	Reset Input	Click to export	clk_1			

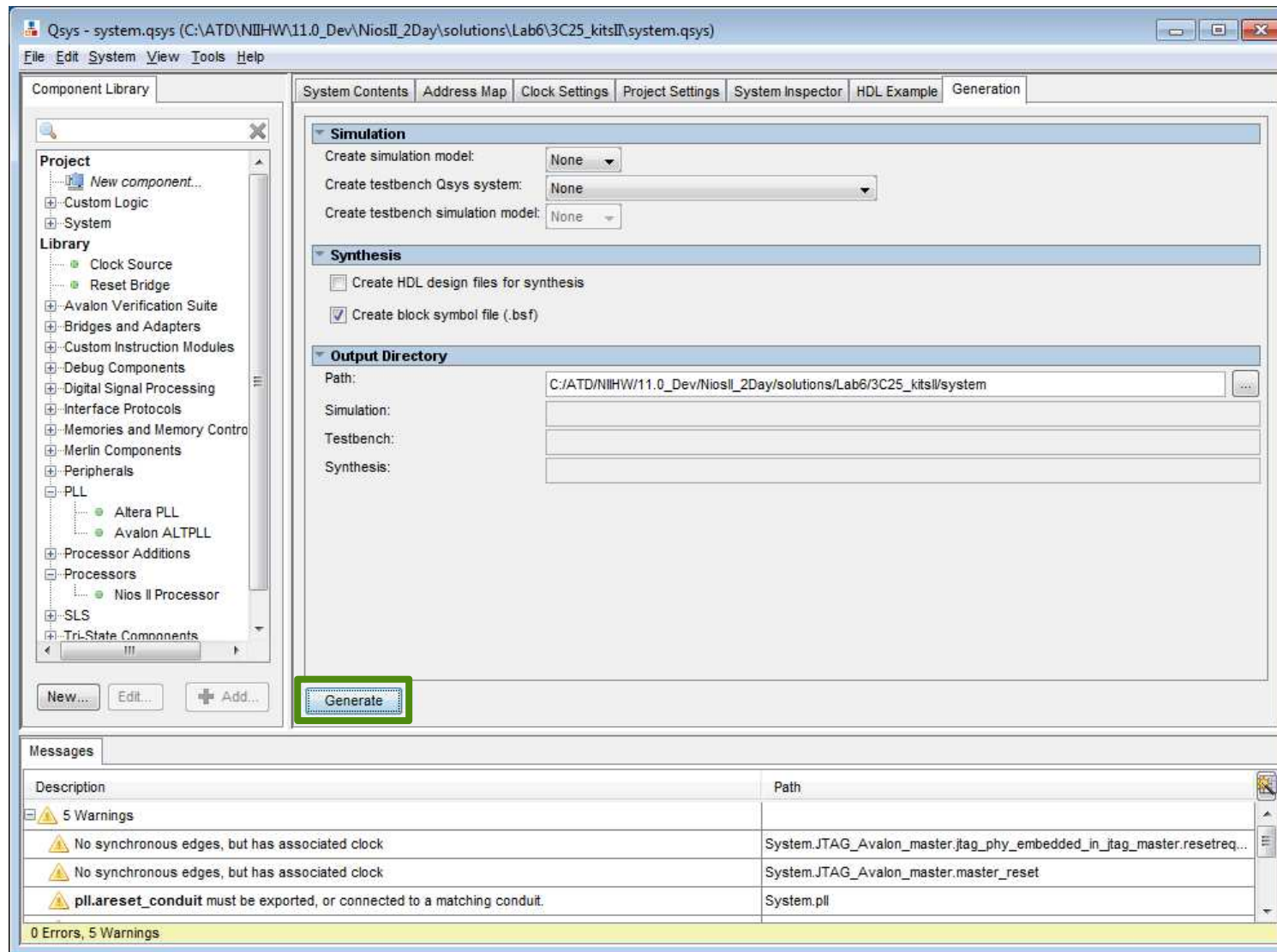
Messages

16 Errors

- "Reset vector memory" (resetSlave) out of range
- "Exception vector memory" (exceptionSlave) out of range
- Reset slave is not specified. Please select the reset slave

16 Errors, 1 Warning

Qsys – System Generation Page



Quartus II and Qsys Software – HDL Directories

■ Quartus II project folder

- Top level HDL code
- Constraint files (.qsf, .sdc, etc.)
- Quartus II project database (db)



  my_project

 db

■ Qsys output folder

- Files for synthesis
- Files for simulation



  niosII_system

  synthesis

  testbench

⋮

■ Software folder

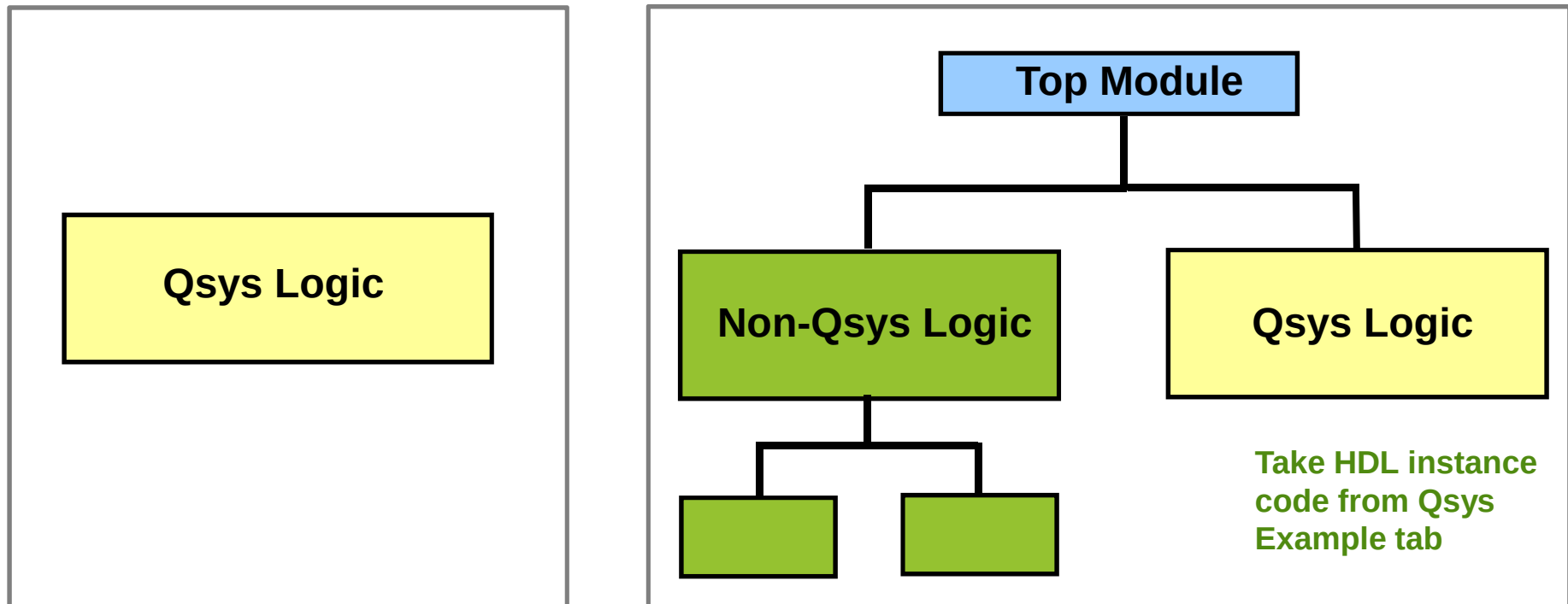
- Application source code
- Library files, etc.



  software

Integrating the Qsys Sub-System

- Use Qsys system as top level of FPGA design **or** instantiate into separate top level file in Quartus II via HDL code or schematic entry tool
- Compile design in Quartus II 



Example Verilog Instantiation

```
module top_level (  
    // inputs:  
    button_pio,  
    reset_n,  
    sys_clk,  
  
    // outputs:  
    clk_to_sdram,  
    clk_to_sdram_n,  
    ddr_a,  
    :  
    ddr_ras_n,  
    ddr_we_n,  
    led_pio,  
    seven_seg_pio,  
    pll_c0_out,  
    pll_c1_out);  
  
// Port Declarations  
:  
  
// Wire Declarations  
:
```

```
Qsys_system Qsys_system_inst  
(  
    .clk_to_sdram            (clk_to_sdram),  
    .clk_to_sdram_n0        (clk_to_sdram_n),  
    .ddr_a_from_the_ddr_sdram_0 (ddr_a),  
    .ddr_ba_from_the_ddr_sdram_0 (ddr_ba),  
    .ddr_cas_n_from_the_ddr_sdram_0 (ddr_cas_n),  
    .ddr_cke_from_the_ddr_sdram_0 (ddr_cke),  
    .ddr_cs_n_from_the_ddr_sdram_0 (ddr_cs_n),  
    .ddr_dm_from_the_ddr_sdram_0 (ddr_dm),  
    .ddr_dq_to_and_from_the_ddr_sdram_0 (ddr_dq),  
    .ddr_dqs_to_and_from_the_ddr_sdram_0 (ddr_dqs),  
    .ddr_ras_n_from_the_ddr_sdram_0 (ddr_ras_n),  
    .ddr_we_n_from_the_ddr_sdram_0 (ddr_we_n),  
    .in_port_to_the_button_pio (button_pio),  
    .out_port_from_the_led_pio (led_pio),  
  
    :  
  
);  
  
endmodule
```

Example VHDL Instantiation

```
entity top_level is
  port (
    -- inputs:
    signal button_pio      : IN STD_LOGIC_VECTOR
    signal reset_n        : IN STD_LOGIC;
    signal sys_clk         : IN STD_LOGIC;
    -- outputs:
    signal clk_to_sdram    : OUT STD_LOGIC_VECTOR
    signal clk_to_sdram_n : OUT STD_LOGIC_VECTOR
    signal ddr_a           : OUT STD_LOGIC_VECTOR
    :
    :
  );
end entity top_level;

architecture structural of top_level is

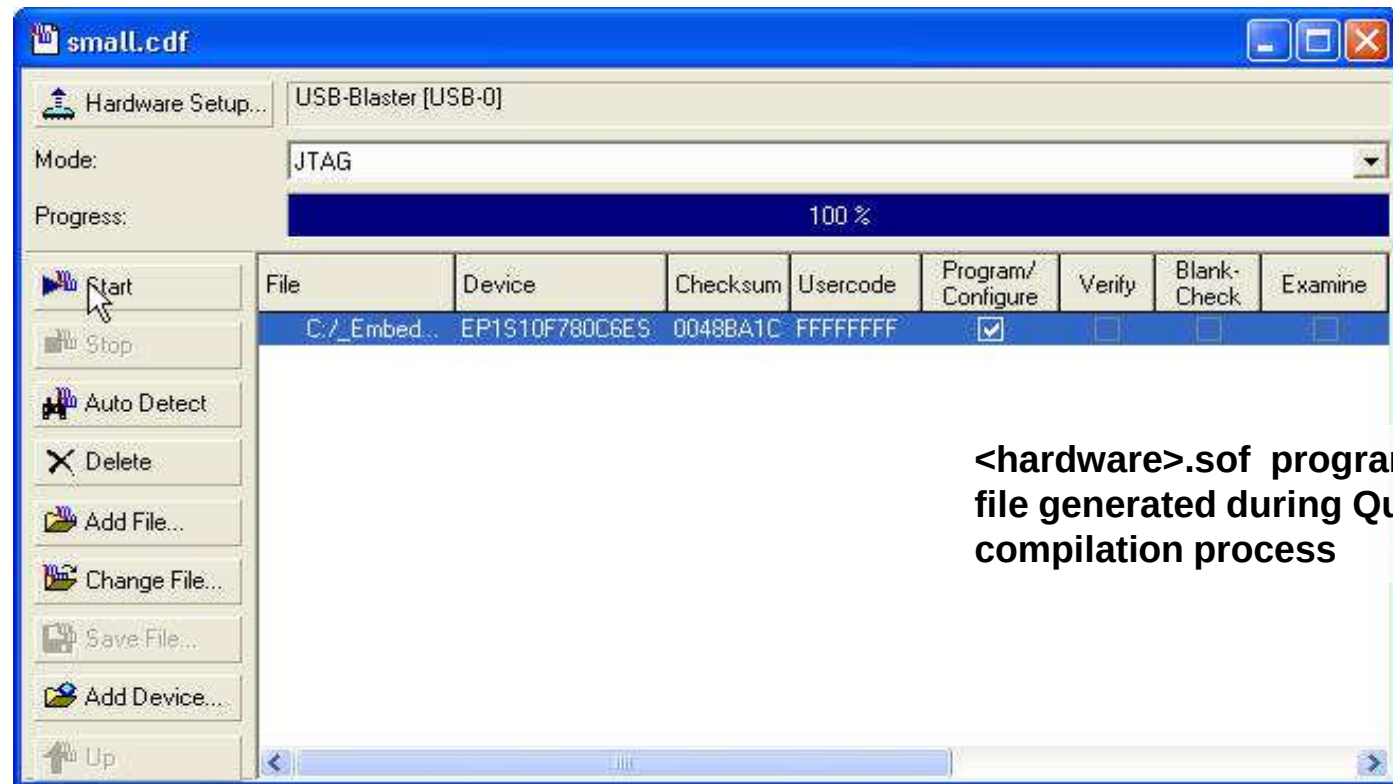
  component Qsys_system is
    PORT (
      signal ddr_dm : OUT STD_LOGIC_VECTOR (1 DOWNTO 0);
      signal ddr_dm : OUT STD_LOGIC_VECTOR (1 DOWNTO 0);
      :
      signal reset_n : OUT STD_LOGIC;
      signal clk      : IN  STD_LOGIC;
      :
    );
  end component Qsys_system;
```

```
begin

  Qsys_system_instance Qsys_system
    port map(
      clk_to_sdram    => clk_to_sdram,
      clk_to_sdram_n => clk_to_sdram_n,
      ddr_a           => internal_ddr_a,
      ddr_ba          => internal_ddr_ba,
      ddr_cas_n       => internal_ddr_cas_n,
      ddr_cke         => single_bit_ddr_cke,
      ddr_cs_n        => single_bit_ddr_cs_n,
      ddr_dm          => internal_ddr_dm,
      ddr_dq          => ddr_dq,
      ddr_dqs         => ddr_dqs,
      ddr_ras_n       => internal_ddr_ras_n,
      ddr_we_n        => internal_ddr_we_n,
      button_pio      => button_pio,
      led_pio         => led_pio,
      seven_seg_pio   => seven_seg_pio,
      :
      :
    );
```

Using the Quartus II Programmer

- Launch from Quartus II software after compiling design
 - To program FPGA with .sof file (ie. FPGA programming bitstream)



<hardware>.sof programming
file generated during Quartus II
compilation process

Some Noteworthy Qsys Peripherals

■ JTAG UART

- Single JTAG connection:
 - Device Configuration
 - Flash Programming
 - Code Download
 - Debug
 - Target STDIO (printing)

■ LCD Display

■ Memory Interfaces

- EPCS Serial Flash Controller
- On-Chip RAM / ROM
- Off-Chip
 - SRAM / SSRAM
 - SDRAM
 - CFI Flash

■ System ID Peripheral

- Ensures Hardware/ Software version synchronization at run-time
- Simple 2 read-only register peripheral containing hardware ID tags
 - Register 1 contains random number
 - Register 2 contains data stamp

All Nios II processor designs
should have a sysid peripheral!

■ PCI Express

■ Triple-Speed Ethernet MegaCore

- MAC, PCS, PMA

■ On-Chip FIFO

■ DMA

■ Scatter-Gather DMA

Noteworthy Qsys Peripherals (cont.)

■ Clock Crossing Bridge

- For high-throughput clock domain crossing from Avalon-MM masters to slaves (faster than default clock crossing logic)

■ Pipelined Bridge

- For pinpoint pipelining of data path segments
 - Helps manage larger designs

■ Generic Tristate Controller

- Defined by presence bi-directional data port
- Configurable with several memory presets to choose from

■ Tristate Bridge

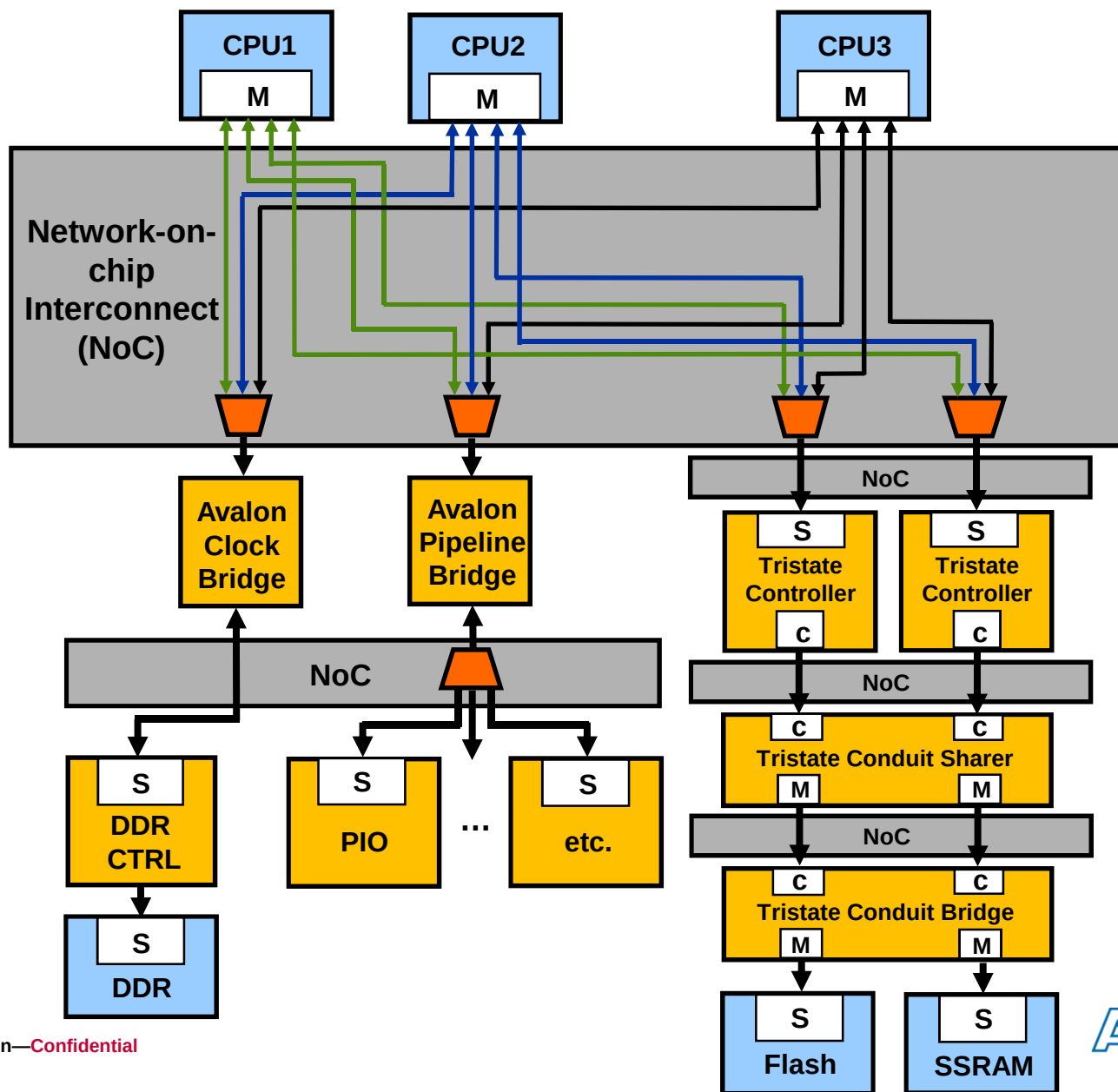
■ Tristate Conduit Pin Sharer

■ JTAG / Avalon Master Bridge

- Allows control of system over JTAG

■ SPI / Avalon Master Bridge

Example System with Bridges, etc.



Example Employing Bridges

Use	Connections	Name	Description	Export	Clock
☒		☐ clk	Clock Source		
☒		☐ pll	Avalon ALTPLL		clk
⋮					
☒		☐ cpu3	Nios II Processor		
	→	clk	Clock Input	Click to export	sys_clk
	→	reset_n	Reset Input	Click to export	[clk]
	→	data_master	Avalon Memory Mapped Ma...	Click to export	[clk]
	→	instruction_master	Avalon Memory Mapped Ma...	Click to export	[clk]
	→	jtag_debug_module_reset	Reset Output	Click to export	[clk]
	→	jtag_debug_module	Avalon Memory Mapped Slave	Click to export	[clk]
	→	custom_instruction_master	Custom Instruction Master	Click to export	
☒		☐ sys_clk_tick	Interval Timer		sys_clk
☒		☐ sysid	System ID Peripheral		sys_clk
☒		☐ DDR_Clock_Bridge	Avalon-MM Clock Crossing ...		multiple
☒		☐ DDR_Controller	DDR SDRAM Controller with ...		ddr_clk
☒		☐ ssram_controller	Generic Tri-State Controller		sys_clk
☒		☐ flash_controller	Generic Tri-State Controller		sys_clk
☒		☐ pin_sharer	Tri-State Conduit Pin Sharer		sys_clk
☒		☐ tristate_bridge	Tri-State Conduit Bridge		
	→	clk	Clock Input	Click to export	sys_clk
	→	reset	Reset Input	Click to export	[clk]
	→	tcs	Tristate Conduit Slave	Click to export	[clk]
	→	out	Conduit	tristate bus	
☒		☐ pipeline_bridge	Avalon-MM Pipeline Bridge		
	→	clk	Clock Input	Click to export	sys_clk
	→	reset	Reset Input	Click to export	[clk]
	→	s0	Avalon Memory Mapped Slave	Click to export	[clk]
	→	m0	Avalon Memory Mapped Ma...	Click to export	[clk]
☒		☐ push_buttons	PIO (Parallel IO)		sys_clk
☒		☐ status_leds	PIO (Parallel IO)		sys_clk
☒		☐ jtag_uart	JTAG UART		sys_clk

Can use clock-crossing bridge for high speed clock crossing

Share FPGA tri-state pins between ssram and flash

Can pipeline branches of system to increase clock frequency

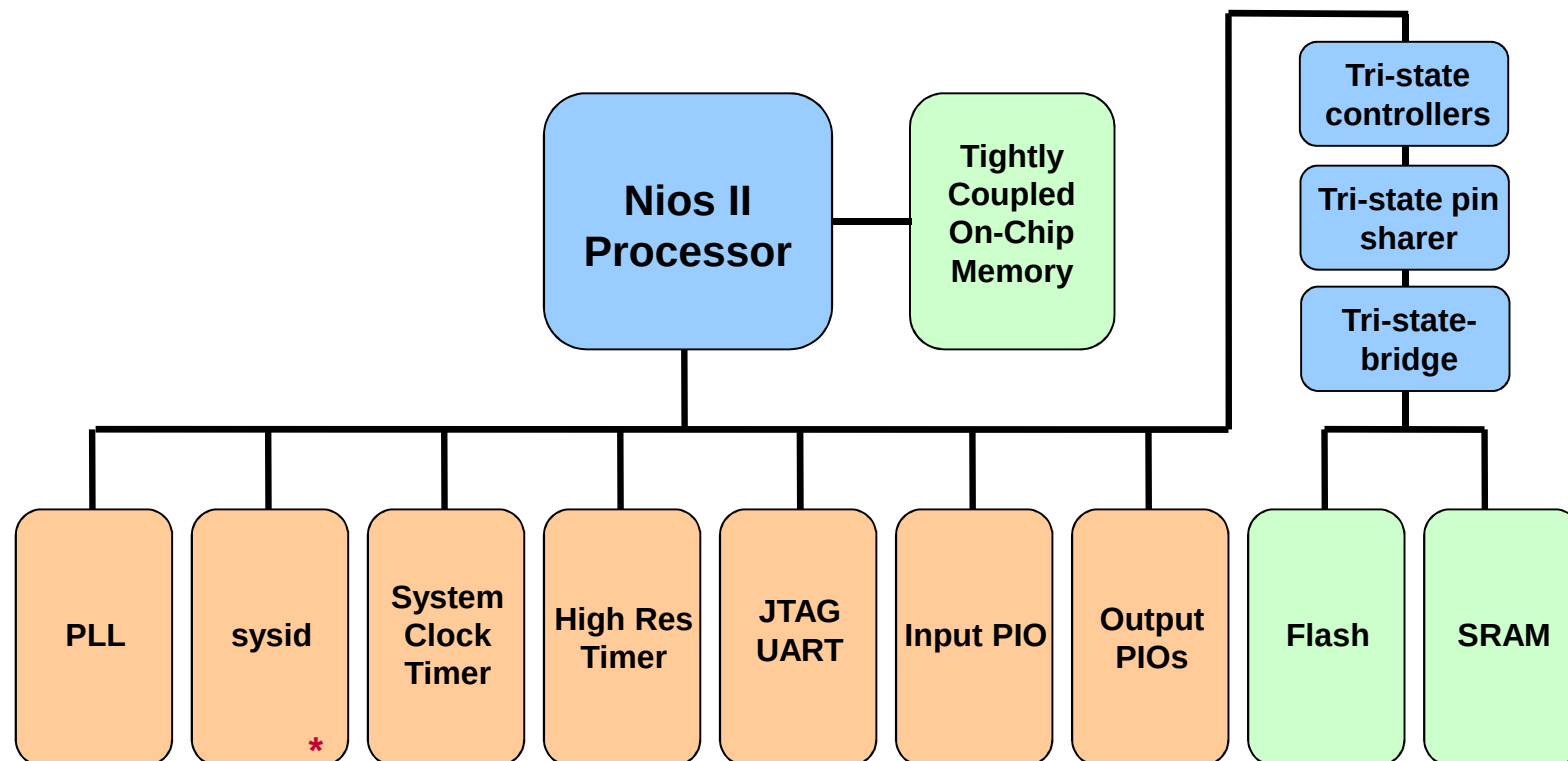
Note: could be a good candidate for its own hierarchical partitions

Also Available for Nios II Processor

- RTOS Support
 - Various operating systems available
 - MicroC/OS-II developer's license included with kit
 - License required to ship products
- Middleware Support
 - Protocol stacks, file systems, graphics libraries, etc.
- Various Built-In Software Components
 - ROZIPFS, TCP/IP Stack, Host-Based File System
 - Interniche TCP/IP stack included with kit (small licensing fee)
- Different Debugger and Compiler tool options
- See www.altera.com > Technology > Embedded Processing
> Altera Embedded Alliance > Partners
- See also www.alteraforum.com
- And www.nioswiki.com

Information for Exercise 1

- You will build a system that resembles the following



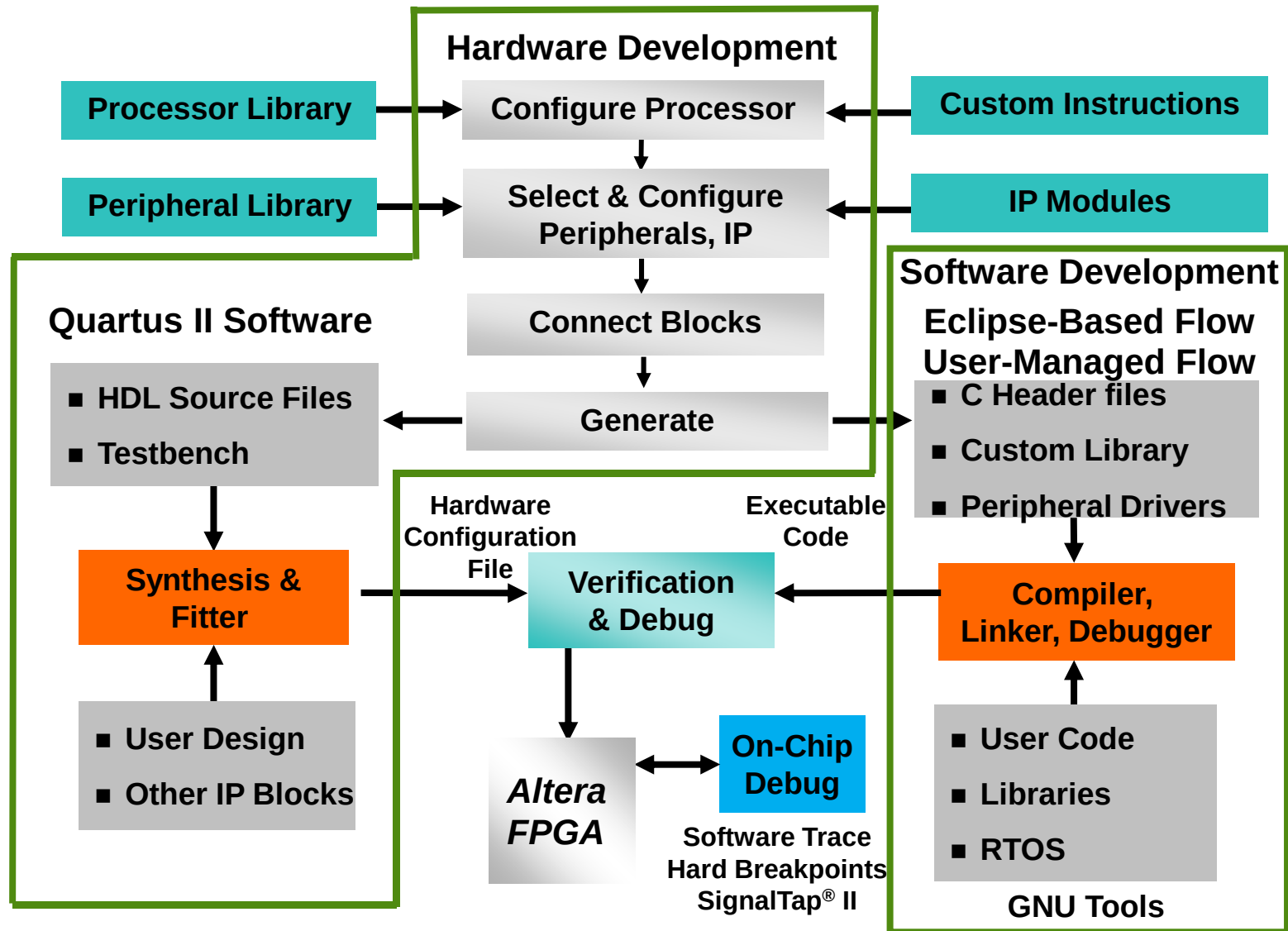
**Reminder: all systems should have a sysid peripheral*

Please go to Exercise 1

Designing with the Nios II Processor and Qsys

Software Development and
Debug Tools

Nios II Processor System – Design Flow

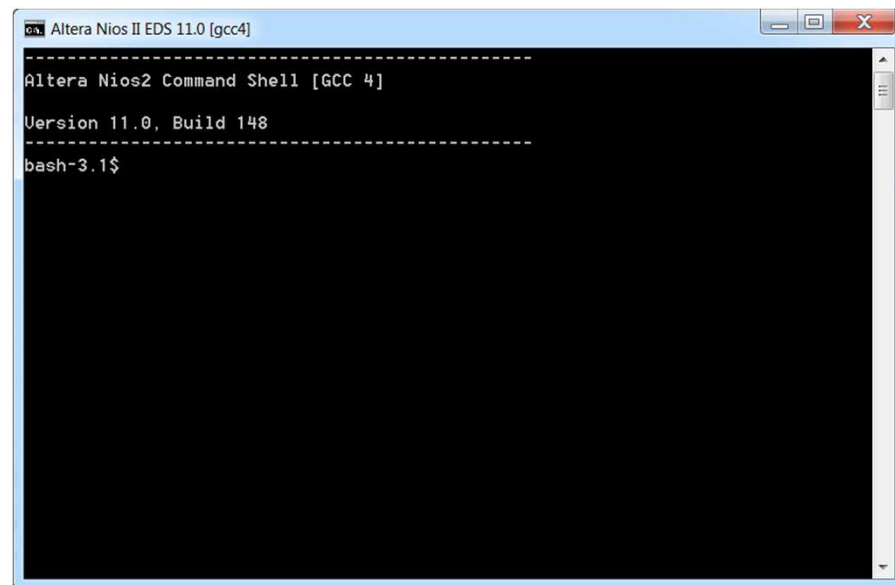


Nios II Embedded Design Suite

- Nios II Software Build Tools (*Command Line*)
 - Set of powerful commands, utilities and scripts
 - Manage build options for applications, board support packages and software libraries
- Nios II Software Build Tools for Eclipse (*GUI*)
 - Eclipse Integrated Development Environment
 - Source navigator and editor, debugger and profiler
 - Compiler, linker and assembler for C and C++
 - Nios II plug-ins for Eclipse
 - Nios II Project Manager
 - Nios II Software Templates
 - Nios II BSP Editor
 - Nios II Command Shell
 - Quartus II Programmer
 - Nios II Flash Programmer

Nios II Command Shell

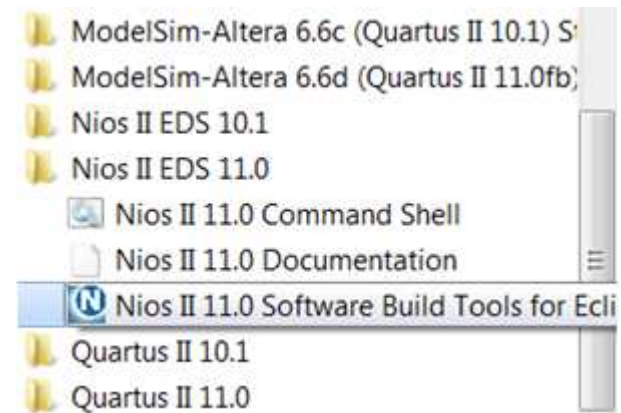
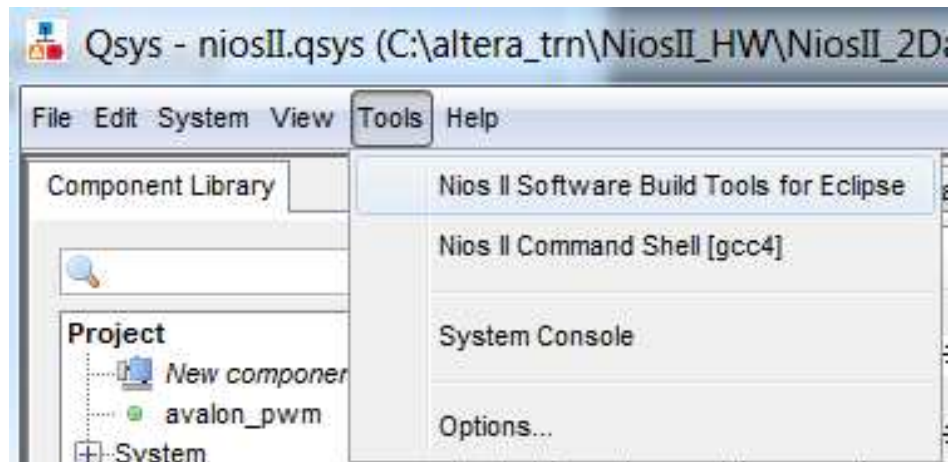
- Used to support shell-driven flow
 - Plus other general commands
- Can launch terminal to interface to JTAG UART
- Compile and Run code
- Create scripts to control build process
- Provides UNIX-like interface
- Open from **Start Menu, Qsys, or Nios II SBT for Eclipse GUI**



```
Altera Nios II EDS 11.0 [gcc4]
-----
Altera Nios2 Command Shell [GCC 4]
Version 11.0, Build 148
-----
bash-3.1$
```

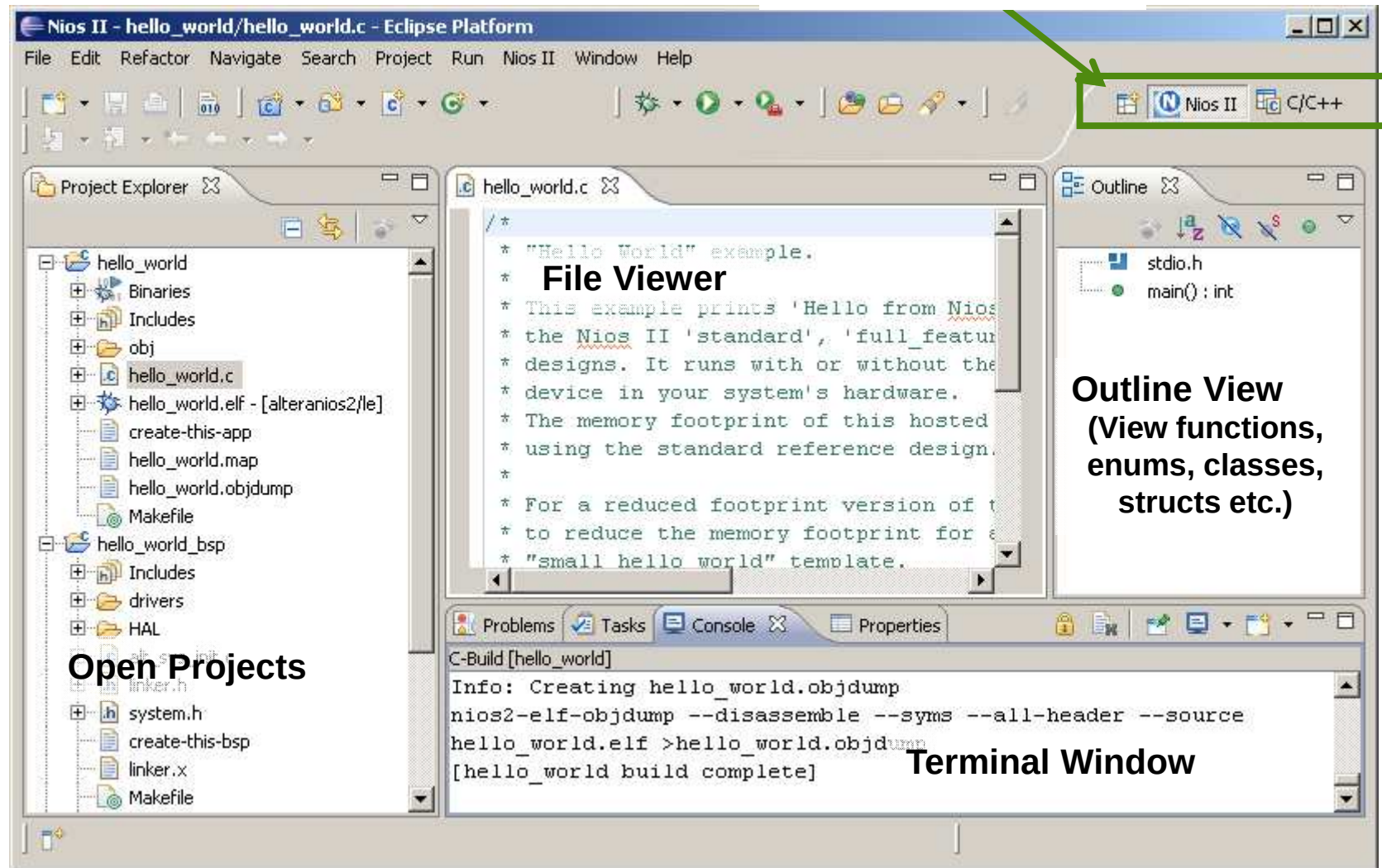
Opening the Tools

- Launch from Windows Start menu, Qsys, or desktop



Nios II Software Build Tools for Eclipse

Select Perspective



- **Note:** C++ files must have extension `.cpp` In-line assembly code offset by `asm()`;

Creating a Nios II Application and BSP

Nios II Application and BSP from Template

Nios II Software Examples
Create a new application and board support package based on a software example template

Target hardware information

SOPC Information File name: ...

CPU name:

Application project

Project name:

☒ Use default location

Project location:

Project template

Templates

- Blank Project
- Board Diagnostics
- Count Binary
- Hello Freestanding
- Hello MicroC/OS-II
- Hello World**
- Hello World Small
- Memory Test
- Simple Socket Server
- Web Server

template description

Hello World prints 'Hello from Nios II' to STDOUT.

This example runs with or without the MicroC/OS-II RTOS and requires an STDOUT device in your system's hardware.

For details, click Finish to create the project and refer to the readme.txt file in the project directory.

The BSP for this template is based on the Altera HAL operating system.

For information about how this software example relates to Nios II hardware design examples, refer to the Design Examples page of the Nios II documentation available with your installation at:

Choose from several templates

File > New > Nios II Application and BSP from Template

Name the project

Browse for .sopcinfo file

< Back Next > Finish Cancel

Application and BSP from Template (2)

Nios II Application and BSP from Template

Nios II Software Examples

Select a board support package for your application

☒ Create a new BSP project based on the application project template

Project name:

☒ Use default location

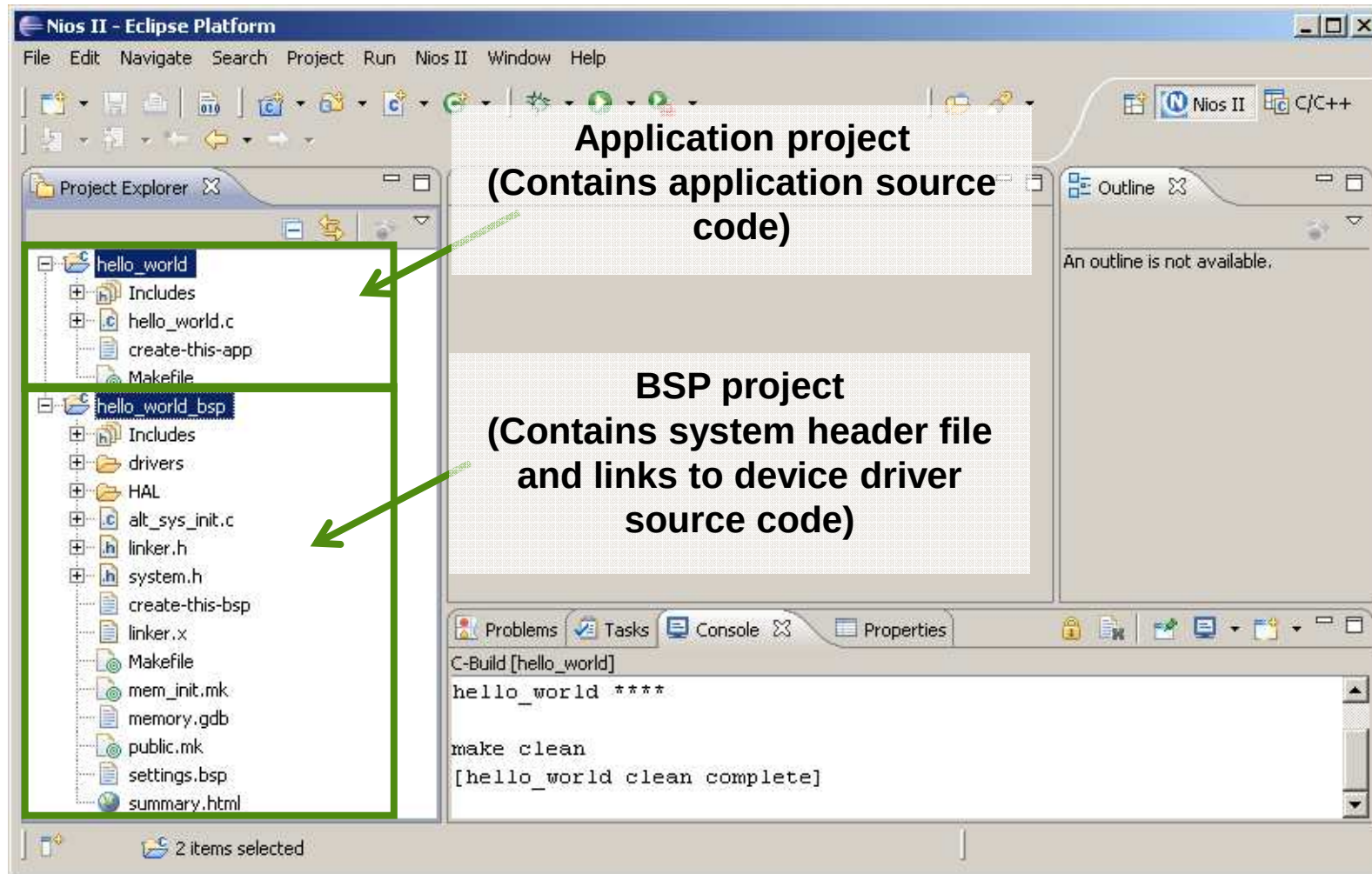
Project location:

☐ Select an existing BSP project from your workspace

< Back Next > **Finish** Cancel

Create new BSP or
choose existing BSP

Application and BSP Project Folders

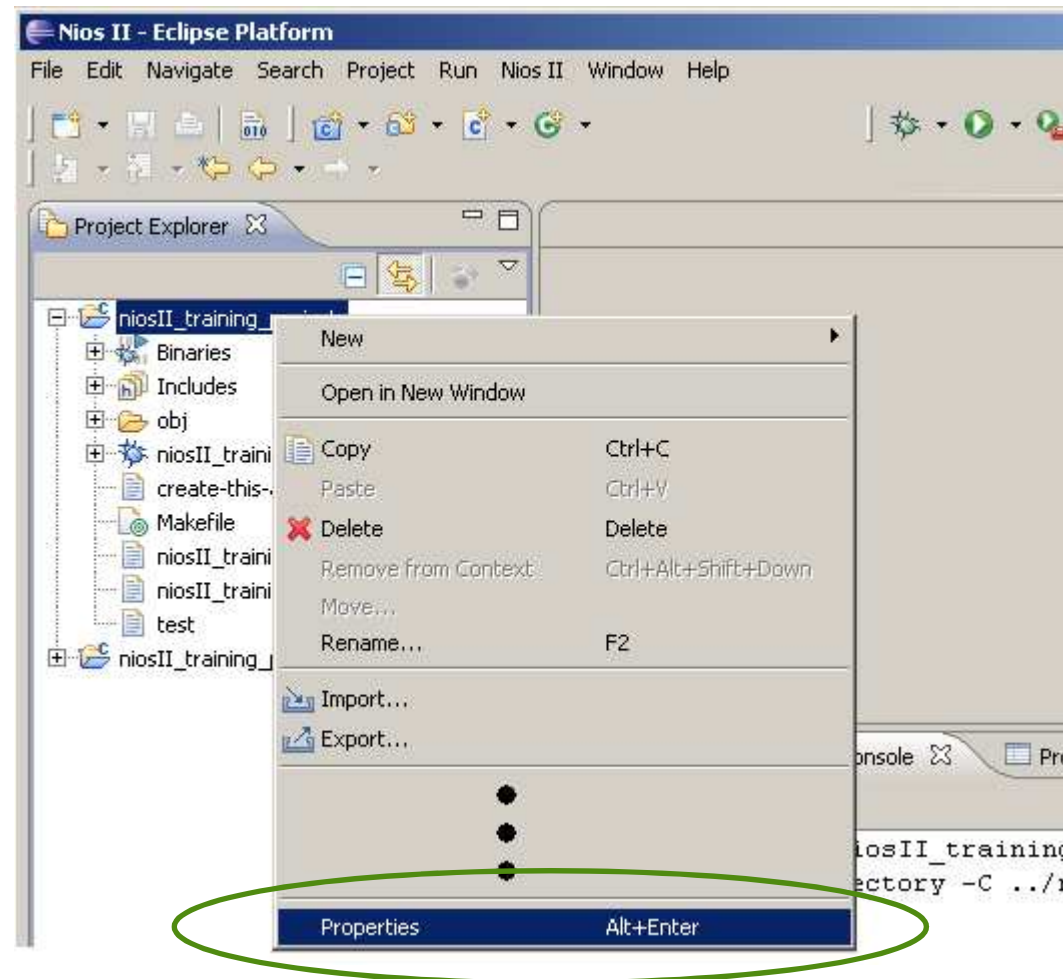


Application and BSP Projects

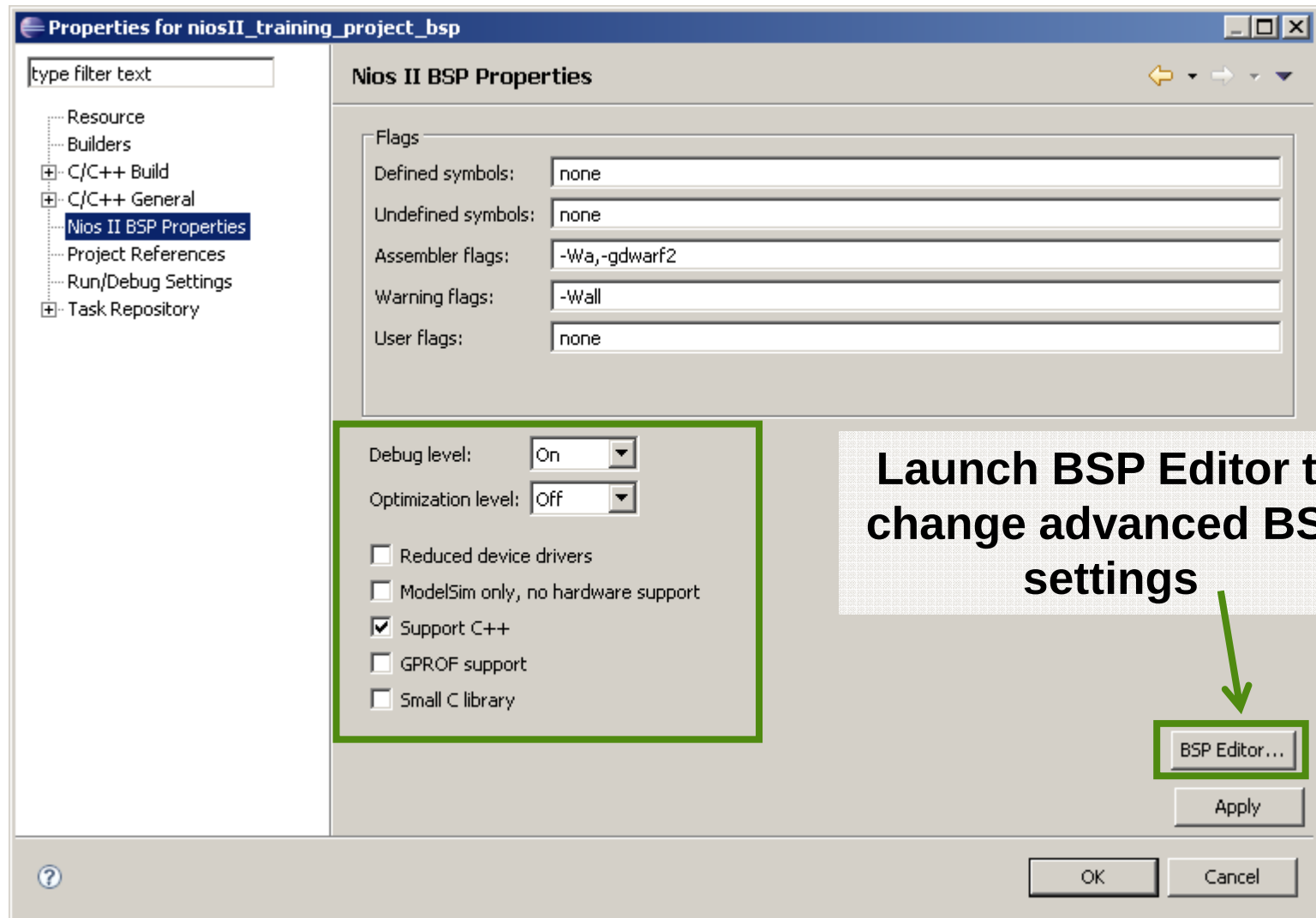
- **Application Projects build executables**
- **BSP Projects contain interface to the hardware**
 - Nios II processor device drivers (Hardware Abstraction Layer)
 - Optional RTOS (MicroC/OS-II)
 - Optional software components
 - Nichestack TCP/IP stack
 - Read Only Zip File System

Setting Project Properties

- Right-click on Application or BSP Project



BSP Project Properties



BSP Editor – Settings Tab (1)

Common settings

Code size options

Specify stderr/in/out

Timer control

Stack options

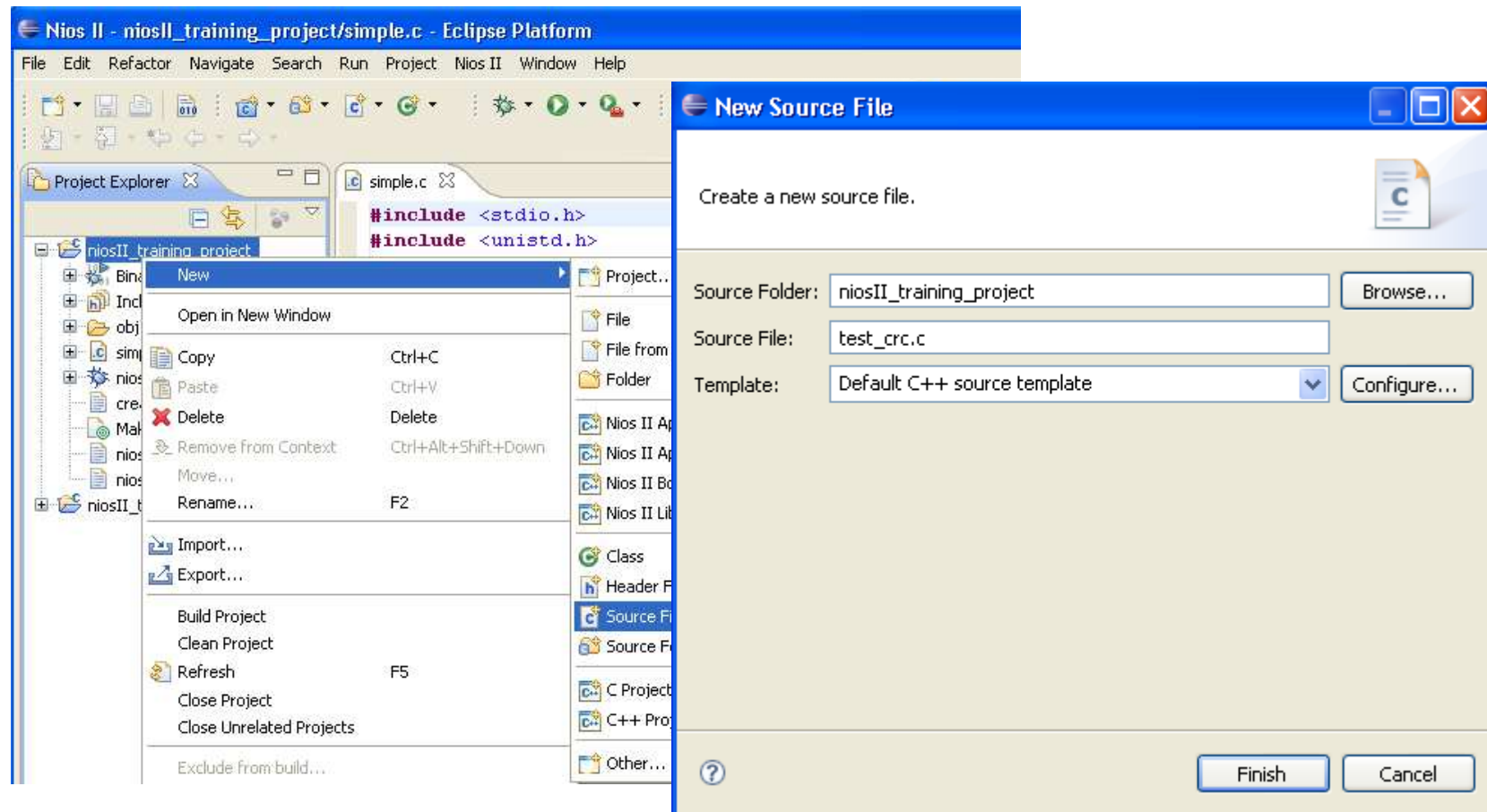
Information | Problems | Processing

- Loading drivers from ensemble report.
- Finished loading drivers from ensemble report.
- Finished loading SOPC Builder system info file "...system.sopcinfo [relative to settings file]"

Generate

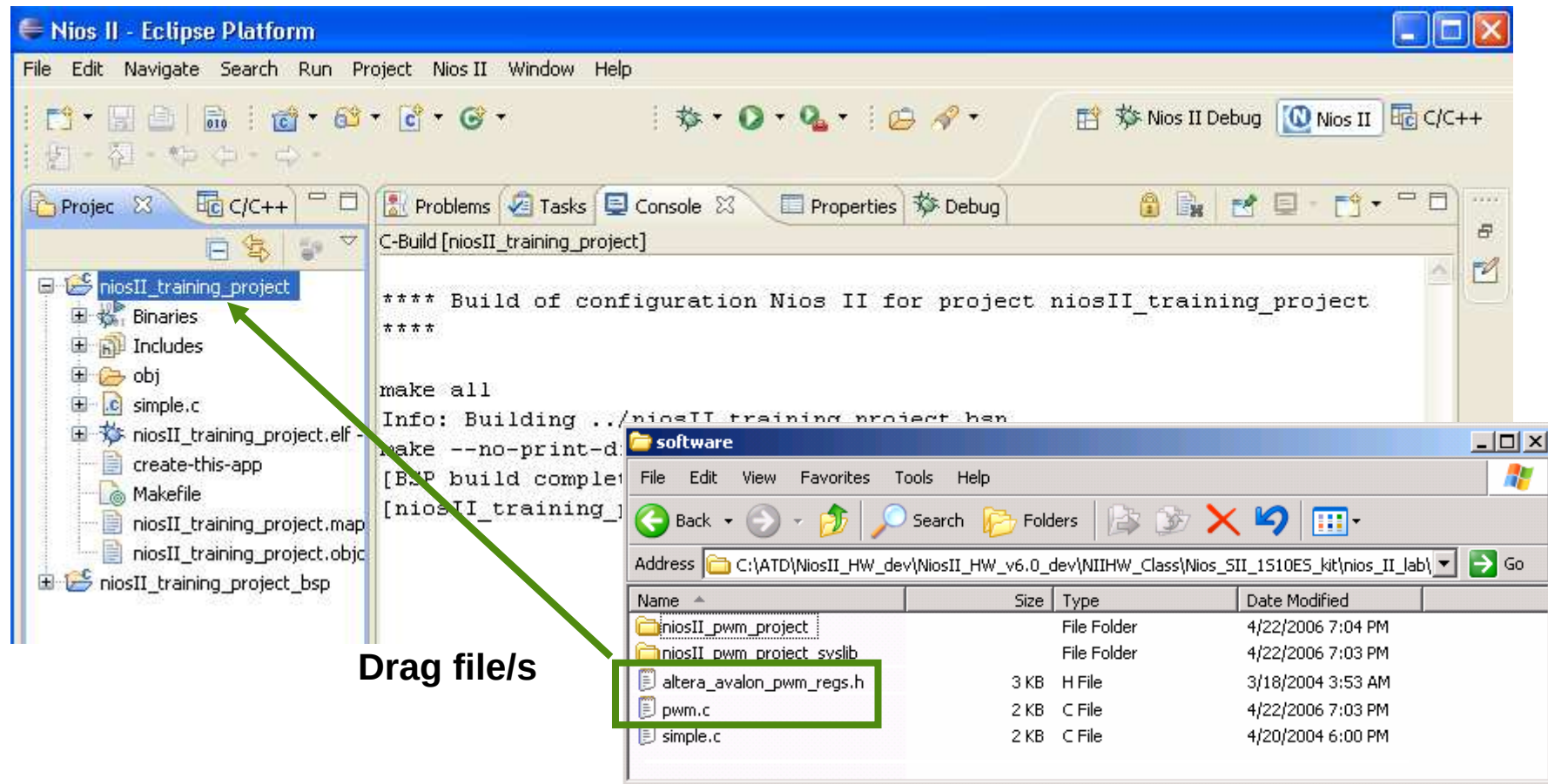
Creating a New Source File

- From **File** menu or by right-clicking on project
 - Specify source folder, file name and template



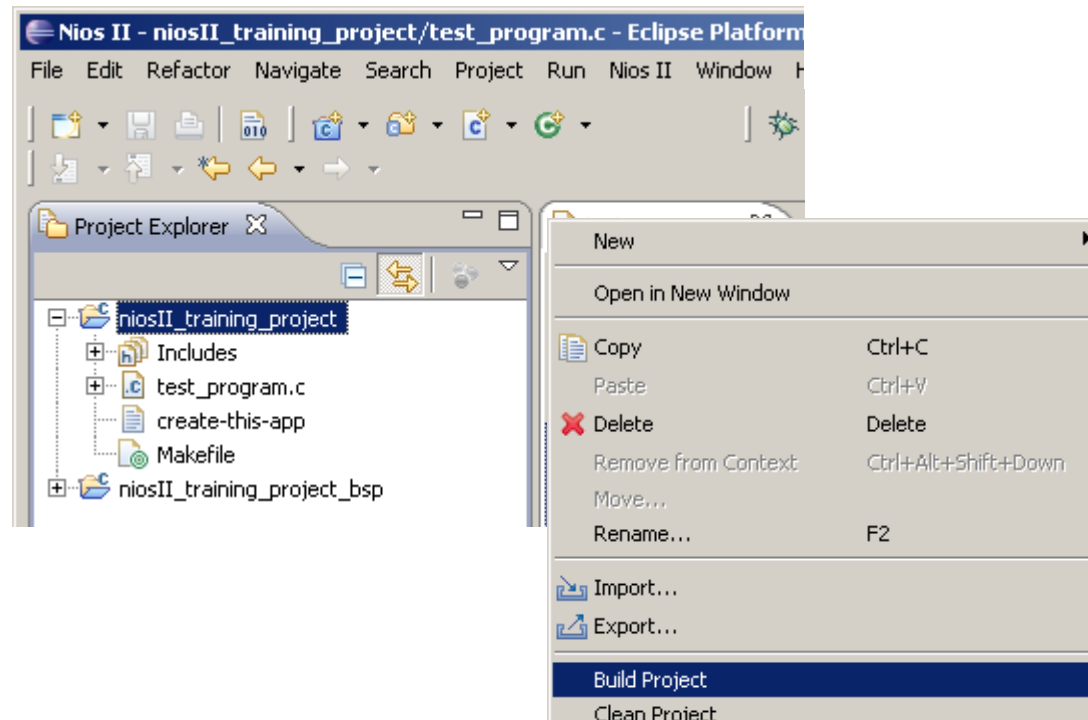
Dragging Source Files Into a Project

- Drag file/s directly onto Application Project
 - Right-Click and Refresh to update project if necessary



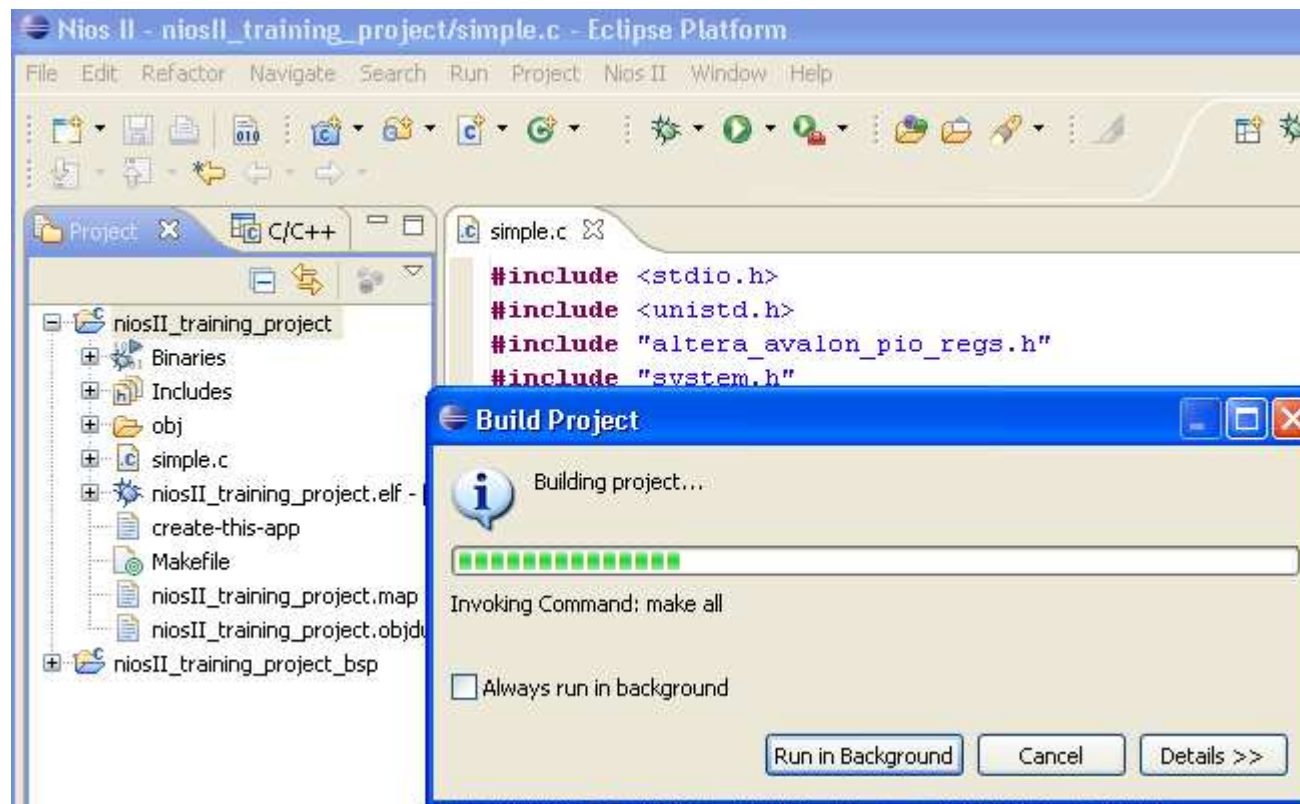
Software Compilation

- To compile a software application, highlight project, right-click, and select **Build Project**, or go to **Projects** menu
 - Compiles BSP project first on initial build
 - Evaluates makefile for compiling application code



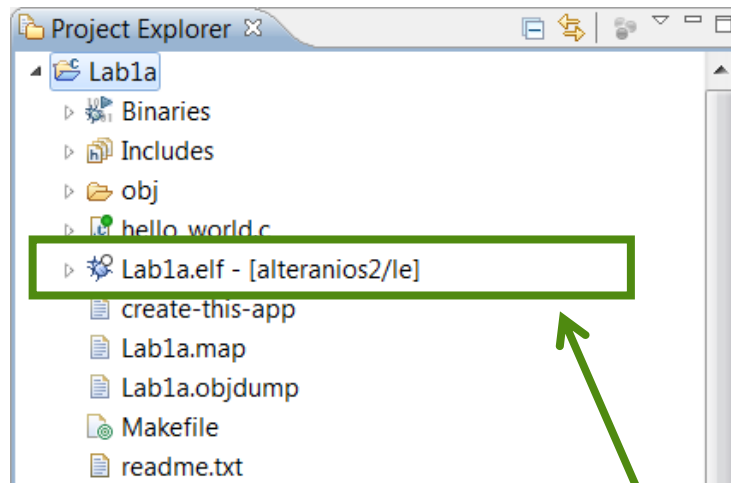
Can Build In Background

- Option for Improved Productivity
 - Carry on other activities in foreground

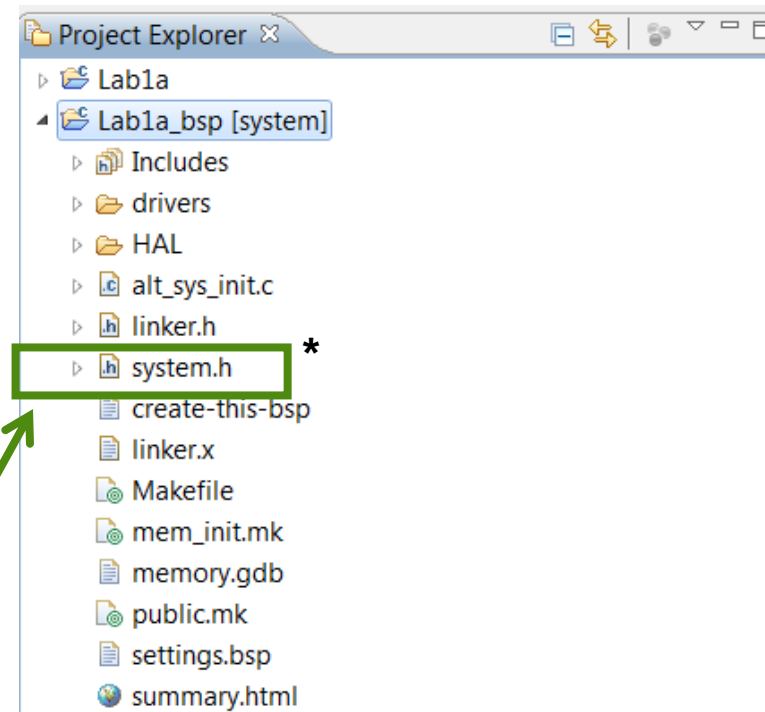


Directory Structure After Build

Application Project



BSP Project



Key Files

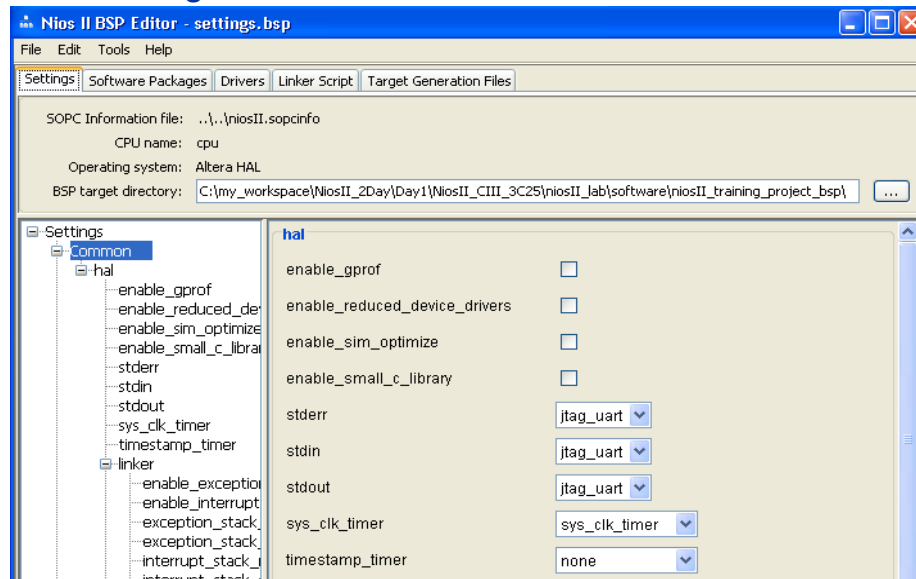
** Created when project created*

System Header File Generation

Qsys System Contents

Use	Connections	Module Name	Description	Clock	Base	End
☒		cpu	Nios II Processor			
		instruction_master	Avalon Memory Mapped Master	sys_clk		
		tightly_coupled_instr...	Avalon Memory Mapped Master			
		data_master	Avalon Memory Mapped Master			
		jtag_debug_module	Avalon Memory Mapped Slave			
					IRQ 0	IRQ 31
					0x02200800	0x02200fff
☒		tightly_coupled_instr...	On-Chip Memory (RAM or ROM)	multiple	multiple	multiple
☒		ext_ram_bus	Avalon-MM Tristate Bridge			
		avalon_slave	Avalon Memory Mapped Slave	sys_clk		
		tristate_master	Avalon Memory Mapped Tristate Master			
☒		ext_ssram	Cypress CY7C1380C SSRAM	sys_clk	0x02100000	0x021fffff
☒		ext_flash	Flash Memory Interface (CFI)	sys_clk	0x01000000	0x01ffffff
☒		jtag_uart	JTAG UART	sys_clk	0x02201080	0x02201087
☒		led_pio	PIO (Parallel I/O)	sys_clk	0x02201060	0x0220106f
☒		button_pio	PIO (Parallel I/O)	sys_clk	0x02201070	0x0220107f
☒		sys_clk_timer	Interval Timer	sys_clk	0x02201000	0x0220101f
☒		high_res_timer	Interval Timer	sys_clk	0x02201020	0x0220103f
☒		sysid	System ID Peripheral	sys_clk	0x02201088	0x0220108f
☒		pll_0	PLL	clk	0x02201040	0x0220105f

BSP Settings



system.h

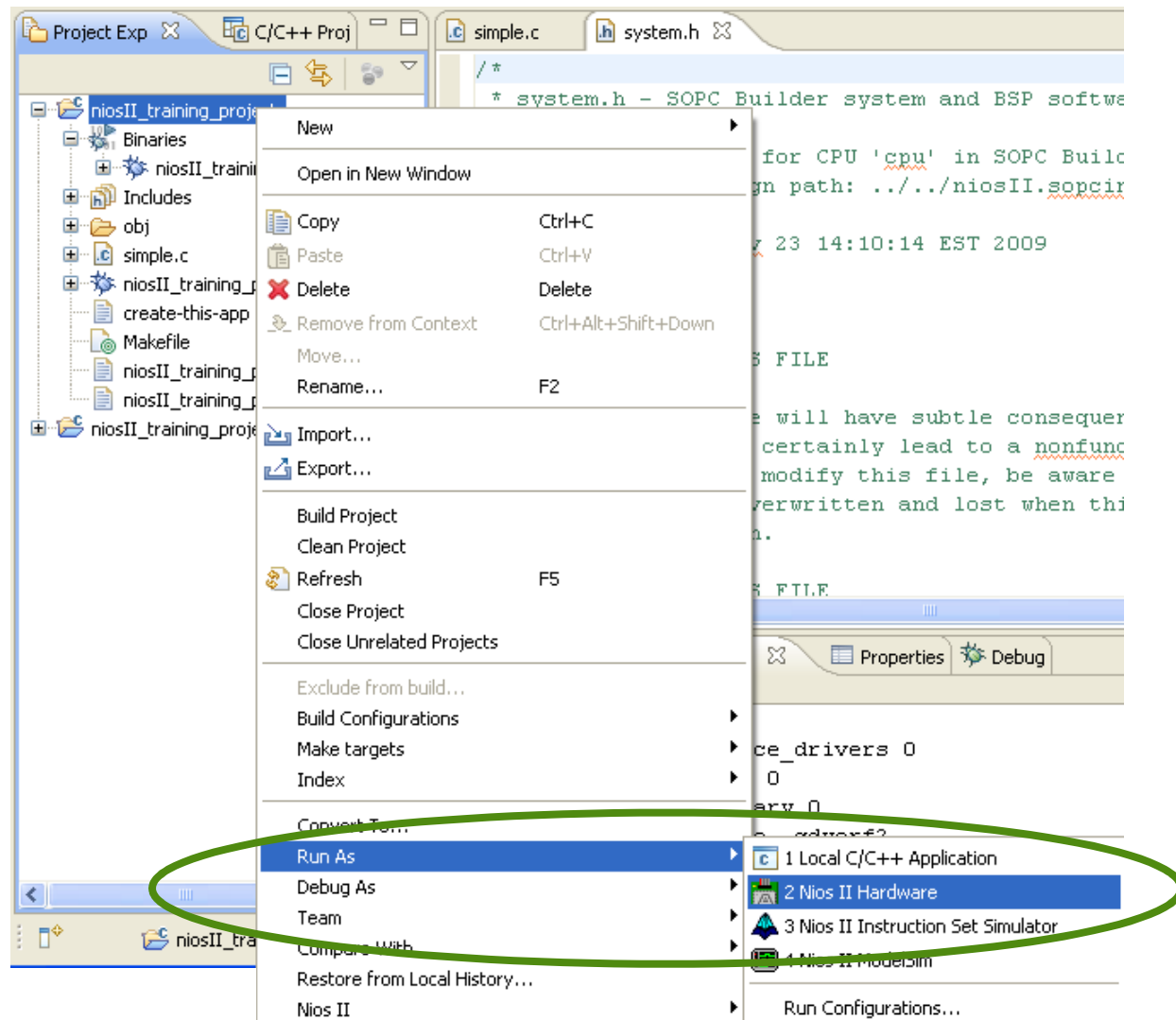
Contains all symbolic C-language definitions for the peripherals in your hardware system, plus more...

Designing with the Nios II Processor and Qsys

Software Run and Debug

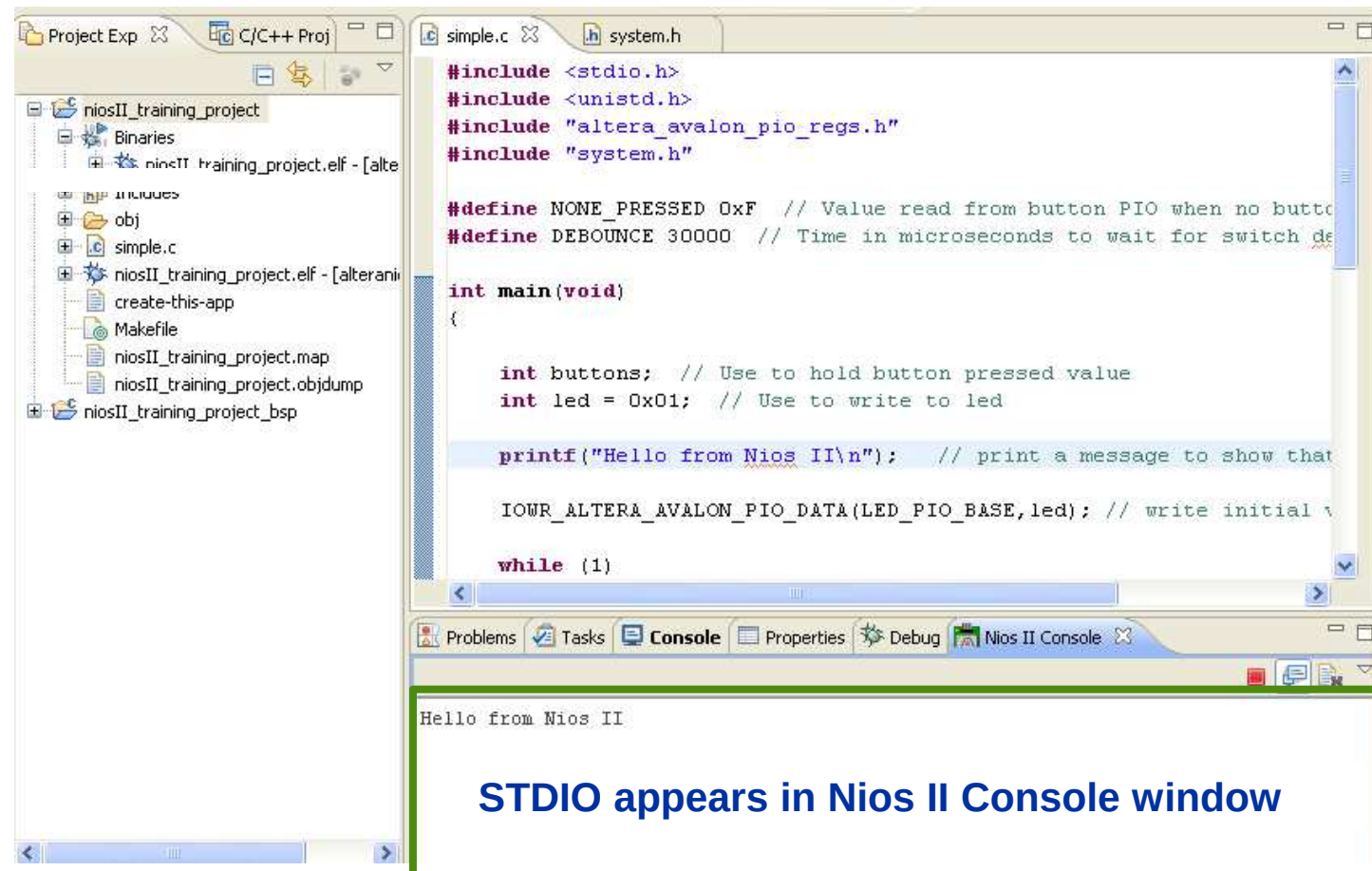
Running Code On A Target

- Download code to target board
- Right-click on Project or go to Run menu

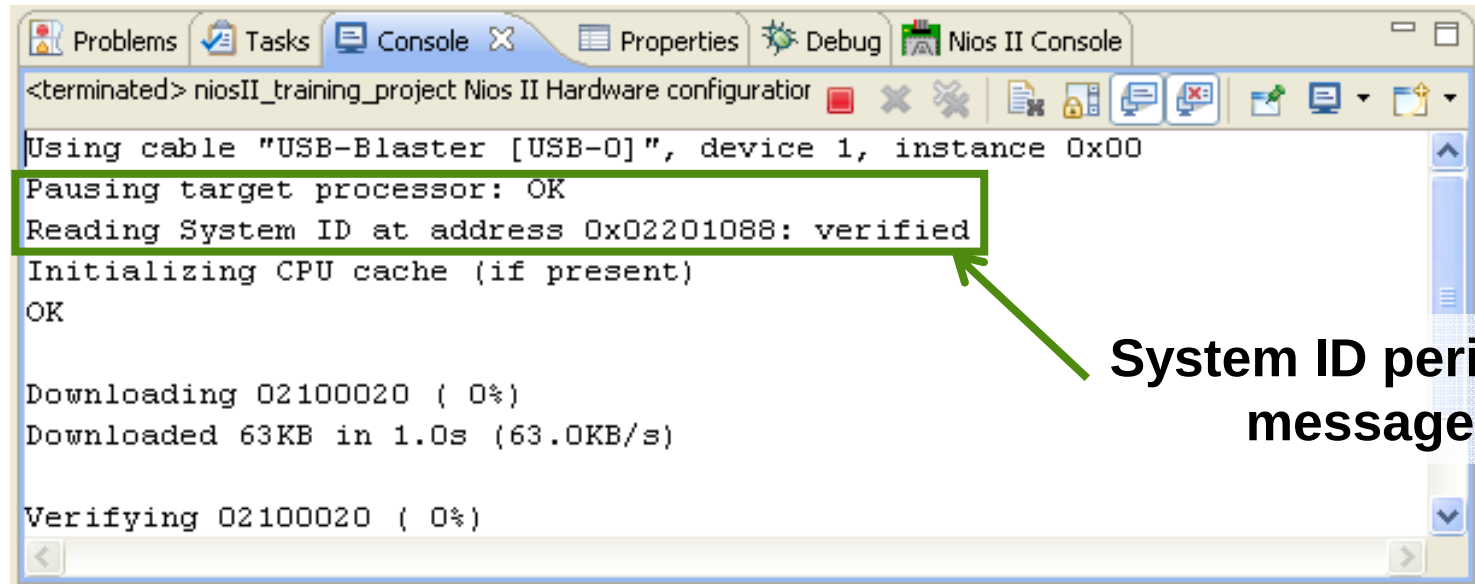


Running Code On A Target

- Download messages appear in Eclipse Console window



System ID Peripheral Check



```
<terminated> niosII_training_project Nios II Hardware configurator
Using cable "USB-Blaster [USB-0]", device 1, instance 0x00
Pausing target processor: OK
Reading System ID at address 0x02201088: verified
Initializing CPU cache (if present)
OK

Downloading 02100020 ( 0%)
Downloaded 63KB in 1.0s (63.0KB/s)

Verifying 02100020 ( 0%)
```

**System ID peripheral
messages**

System ID checks

- ☐ Ignore mismatched system ID
- ☐ Ignore mismatched system timestamp

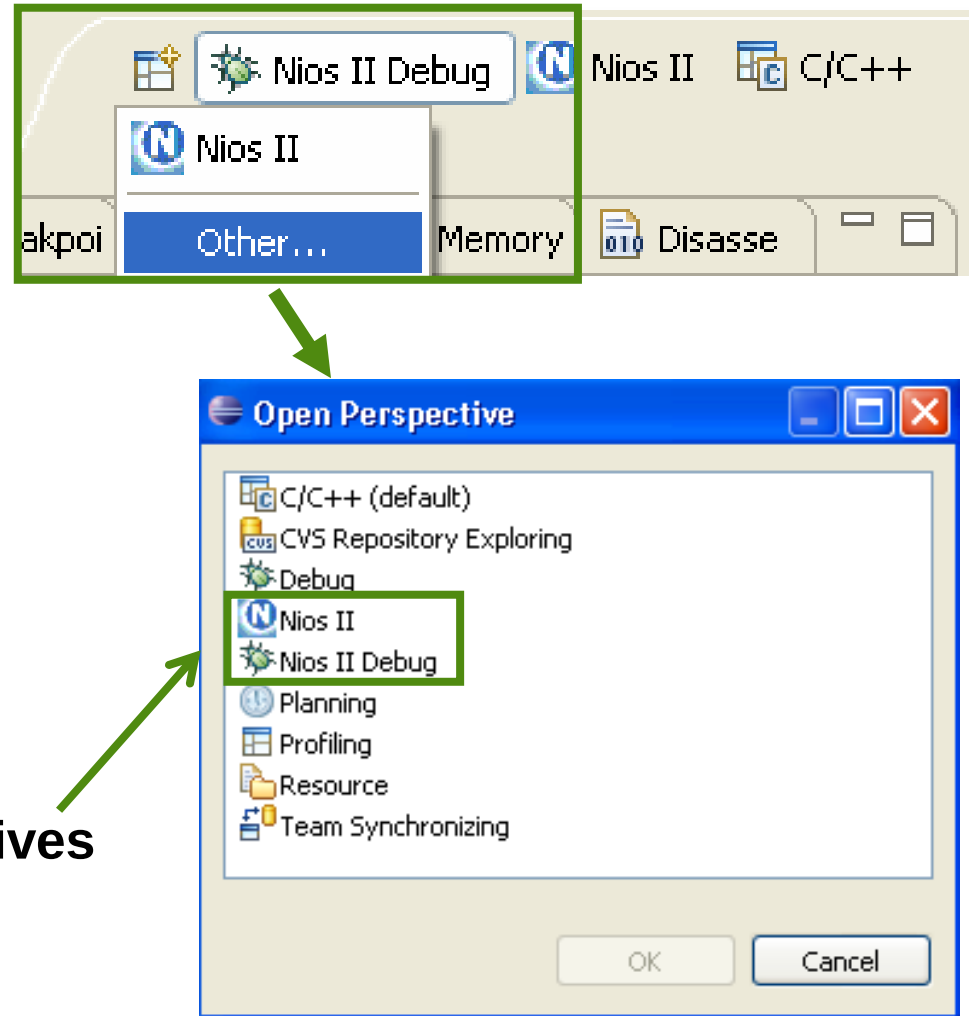
Disable checks

(From Run → Run Configurations → Target Connections)

Nios II Perspectives

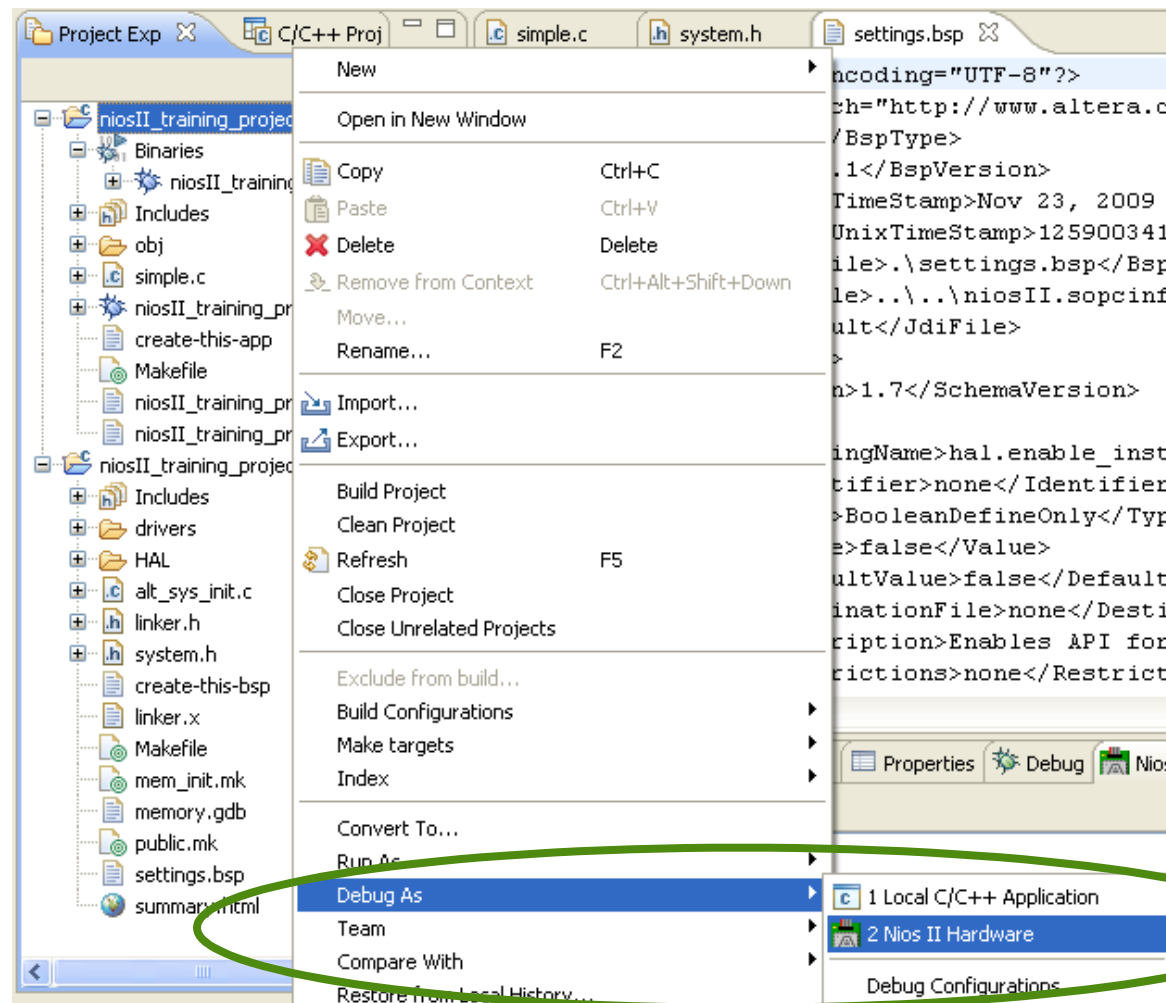
- Provides a set of tool capabilities
 - Debugging
 - Running
 - Profiling
 - Etc.
- Switch between perspectives at will

Nios II Perspectives



Nios II Debugger

**Must have JTAG
Debug Core
enabled in Nios
II processor**



Nios II Debug Perspective

Perspective Selector

Run project

Debug project

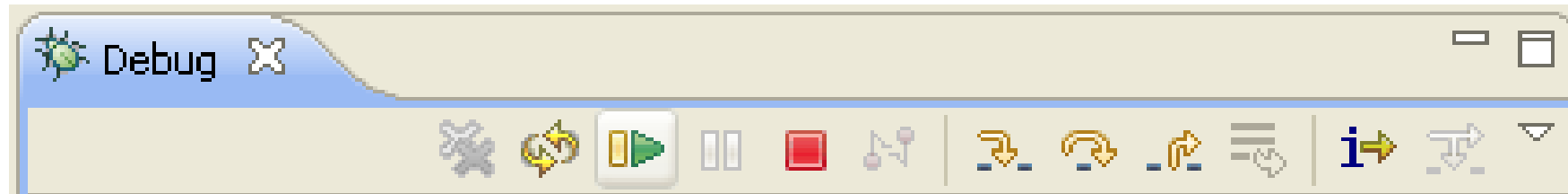
**View/modify
(Variables, registers, signals...)**

**Double-click to add
breakpoints**

Name	Value
buttons	34659996
led	34658780

```
while (1)
{
    buttons = IORD_ALTERA_AVALON_PIO_DATA(BUTTON_PIO_BASE); //
```

Nios II Debugger Controls



Restart
Resume
Suspend
Terminate
Disconnect

Step Into
Step Over
Step Return

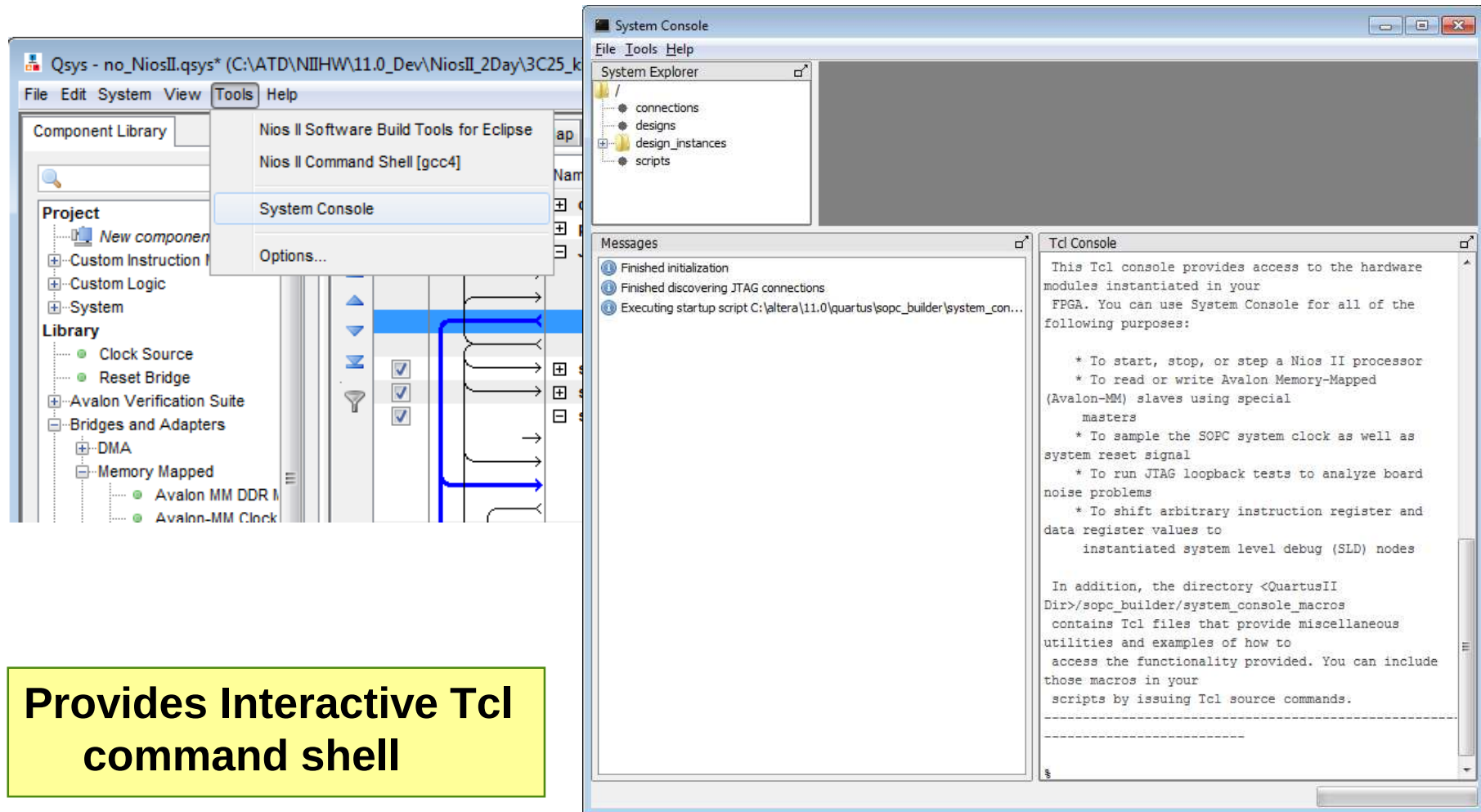
Designing with the Nios II Processor and Qsys

Board Bring-Up Tools

What is the System Console?

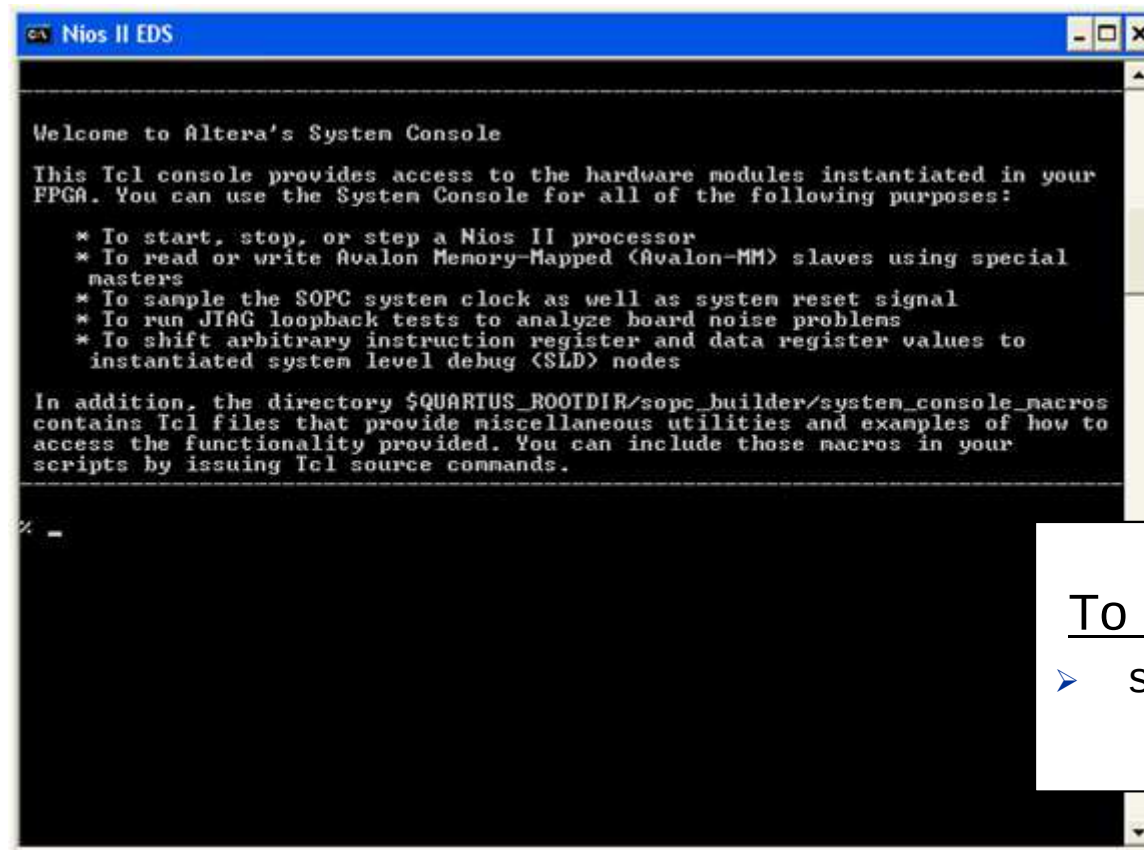
- Qsys console for low-level system test and debug over JTAG
 - TCL-based
 - Familiar development tool language
 - Interactive
 - Opens as a separate window
 - Opens in the Nios II Command Shell
 - Scriptable
 - TCL files can be “sourced”
 - Supports command line arguments
 - Supports standard input/output

Qsys System Console Launch



Command Line Interface

- Available from Nios II Command Shell
 - Provides alternate interface to system-console utilities



The screenshot shows a window titled "Nios II EDS" with a black background and white text. The text reads: "Welcome to Altera's System Console", "This Tcl console provides access to the hardware modules instantiated in your FPGA. You can use the System Console for all of the following purposes:", followed by a bulleted list of five items: "* To start, stop, or step a Nios II processor", "* To read or write Avalon Memory-Mapped (Avalon-MM) slaves using special masters", "* To sample the SOPC system clock as well as system reset signal", "* To run JTAG loopback tests to analyze board noise problems", and "* To shift arbitrary instruction register and data register values to instantiated system level debug (SLD) nodes". Below the list, it says: "In addition, the directory \$QUARTUS_ROOTDIR/sopc_builder/system_console_macros contains Tcl files that provide miscellaneous utilities and examples of how to access the functionality provided. You can include those macros in your scripts by issuing Tcl source commands." The prompt "% " is visible at the bottom left of the console area.

To Launch, type:
➤ `system-console --cli`

Usage Examples

■ Low-level debug

- Board bring-up and interface testing
- System clock, reset and JTAG chain validity testing
- Qsys component functionality testing

■ System-level debug

- Provide test vectors, return response
- No processor required

Service Types

- sld
 - Low-level access to instances on internal jtag hub in all Altera FPGAs
- jtag_debug
 - Requires JTAG to Avalon MM Bridge component
 - JTAG chain debug
 - SOPC system clock and reset debug
- bytestream
 - Testing "character" devices, i.e. jtag_uart
- master
 - Provides control of Avalon master port on JTAG to Avalon MM Bridge component or Nios II processor
 - Allows read / write to any Avalon slave (memory, peripheral, etc.)
- processor
 - Provides access to processor registers & execution control
- device
 - Offers SOF download and JDI sld node name mapping.
 - The JDI names correspond to components that provide services in your Qsys system

Typing "get_service_types" within an interactive console session will return the service types

Service Type - Master

■ Read/Write to memory

```
set mm [ lindex [ get_service_paths master ] 0 ]
processor_stop $mm
open_service master $mm
set values [list 0xaa 0x55 0xaa 0x55 0xaa 0x55 0xaa 0x55 0xaa 0x55 0xaa 0x55 ]
master_write_memory $mm 0x1000 $values
master_read_8 $mm 0x1000 12
close_service master $mm
```

■ Read/Write access to any Avalon-MM Slave

```
set mm [ lindex [ get_service_paths master ] 0 ]
processor_stop $mm
open_service master $mm
master_write_32 $mm 0x8000 0x87654321
# Writes 0x87654321 to component at 0x8000 (eg. A bidi output PIO)
puts [ format "Current PIO value: 0x%08X" [ master_read_32 $mm 0x1000 1 ] ]
# Validates the data just written to the PIO in previous step
close_service master $mm
```

Basic Flow

1. Ensure that your board is properly connected and configured
2. Launch System Console
3. Locate the service and connect to the Qsys IP that provides it
`set my_service_path [ lindex [ get_service_paths master ] 0 ]`
Note: `$my_service_path` now contains a “service path” of type “master”
4. If using CPU as master, stop it first
`processor_stop $my_service_path`
5. Open the service
`open_service master $my_service_path`
6. Perform operation(s) on this (master) service
`master_write_8 $my_service_path <address> <value>`
`set $read_value [master_read_8 $my_service_path <address>`
`<num_values>]`
7. Close the device
The following command closes the service path opened in Step 5:
`close_service master $my_master`

Example Commands for JTAG-Avalon-MM Master

“turnon_LEDs.tcl”

```
# Define and initialize variables
set led_val 4
set led_pio 0x40000000

# Define a variable to service path: "master"
set jtag_master [lindex [get_service_paths master] 0]

# Open master service path
processor_stop $jtag_master

# Open master service path
open_service master $jtag_master

# Utilize master to write to (poke) led peripheral
master_write_8 $jtag_master $led_pio $led_val

# Close the master service path
close_service master $jtag_master
```

Returns a list of paths of type master; Indexes into the list

➤ **system-console -script=turnon_LEDS.tcl**

Further Information

- System Console User Guide & examples
 - Found online @ <http://www.altera.com/literature/lit-sop.jsp>
 - “System Integration with Qsys” class
- Built-in help
 - Type `help` to see a list of all supported commands and a brief usage statement.
- Basic TCL Reference
 - Just enter `TCL Tutorial` in your favorite web search engine and start browsing through the discovered tutorials

Information for Exercise 2

- Build a Software Project for the Nios II Processor System from Exercise 1 and Test System with System Console

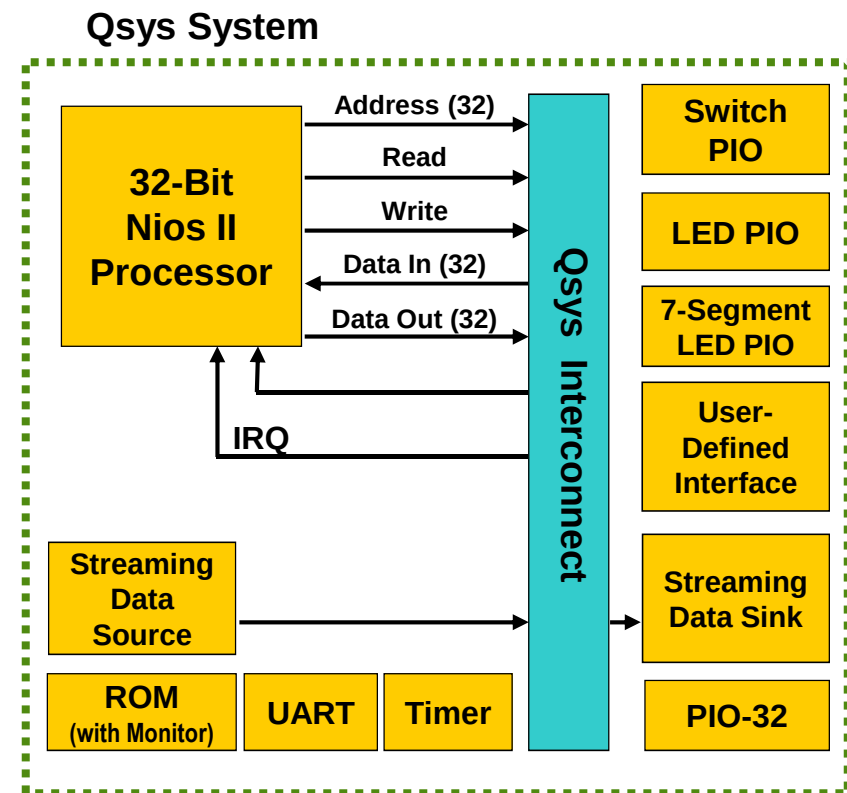
Please go to Exercise 2

Designing with the Nios II Processor and Qsys

Qsys Interconnect

Qsys Interconnect

- Interconnect specification used in Qsys systems
- Network-on-Chip (NoC) architecture
- Principal design goals
 - Low resource utilization for bus logic
 - Synchronous operation
 - High performance
- Transfer Types
 - Slave Transfers
 - Master Transfers
 - Latency-Aware Transfers
 - Burst Transfers
 - Streaming Transfers



NoC Interconnections

- Automatically generated by Qsys
- Custom generated for peripherals in system
 - Contingencies are on per-peripheral basis
 - System not burdened by unnecessary bus complexity
- Qsys takes care of
 - Arbitration
 - Address Decoding
 - Data Path Multiplexing
 - Bus Sizing
 - Wait-State Generation
 - Interrupts

Qsys Interconnect Implementation

■ Memory addressing

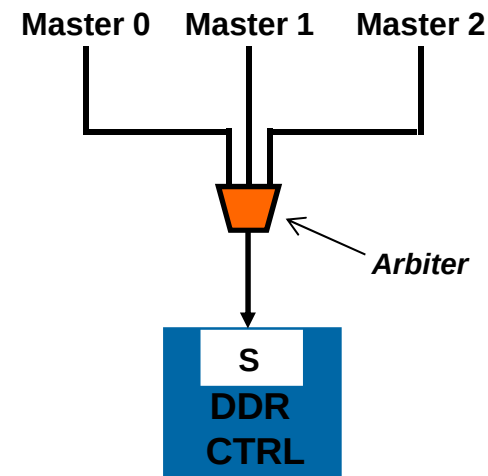
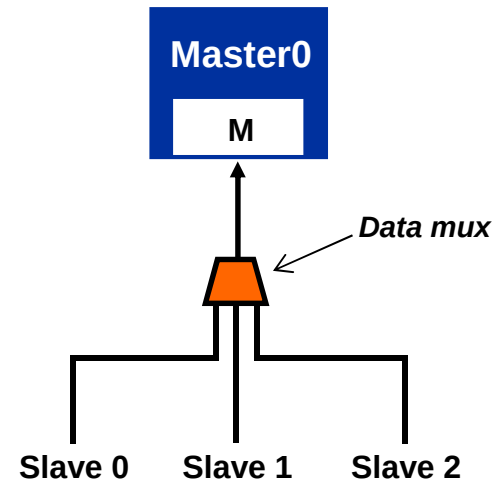
- Qsys automatically generates decoding logic so masters can access slave registers

■ Data multiplexing

- Qsys automatically generates muxes on master interfaces that communicate with multiple slaves

■ Arbitration

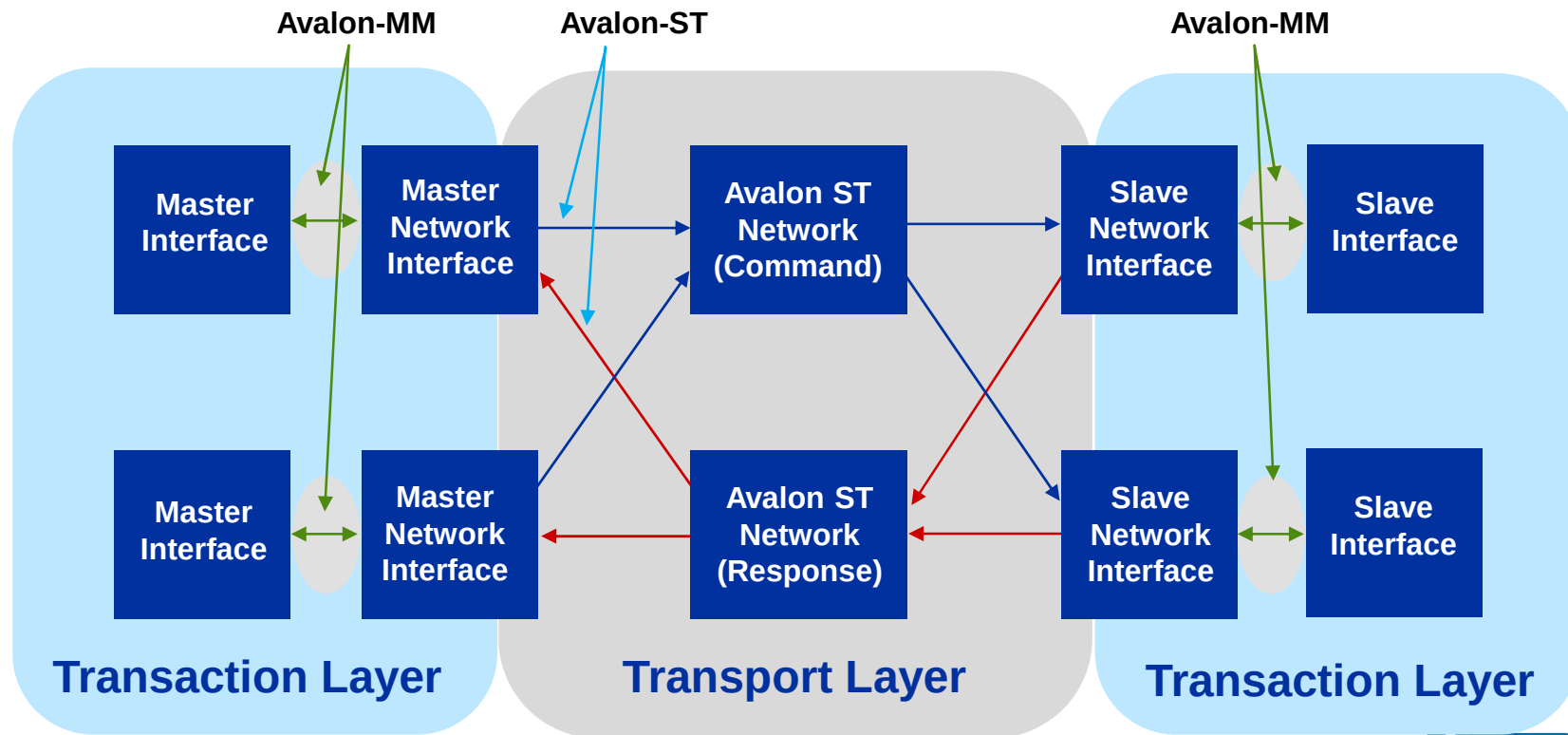
- Qsys automatically generates arbiters on slaves controlled by multiple masters
- Controls which master has current slave access



NoC Architecture

■ Packet transactions and transport

- Memory-mapped
 - Each transfer/request encapsulated in packet and sent to slave
 - Each response encapsulated in packet and sent back to master
- Streaming transfers already packetized



Qsys Packet for Memory Mapped Data

Addr	Trans Type	Data	Byte Enables	Source ID	Dest ID	Byte Count	Burst Wrap	Prot
------	------------	------	--------------	-----------	---------	------------	------------	------

Packet Field	Description
Addr	Byte address of lowest byte in packet
Trans Type	Transaction type (e.g. read, write, lock)
Data	Write - data to be written; Read - data that has been read
Byte Enables	Which bytes of data in packet are valid
Source ID	Command - ID of the master; Response - ID of the slave
Dest ID	Command - ID of the slave; Response - ID of the master
Byte Count	Number of remaining bytes in the transfer
Burst Wrap	Defines the wrapping behavior during bursting
Prot	Access level protection 0 - normal access; 1 – privileged access

Note: See Qsys Interconnect chapter of the Quartus II Handbook for more details on the packet fields

Benefits of NoC Approach

- See White Paper: [Applying the Benefits of Network on a Chip Architecture to FPGA System Design](#)
- Allows independent implementation and optimization of transaction and transport layers
 - Use different network topologies (e.g. ring, star, mesh) to implement transport layer without changing transaction layer
 - Future example: Use high performance components on a wide high frequency crossbar network, but peripherals on area effective mesh network with a packet bridge between networks
- Supports standard interface interoperability
 - Mix and match interface types without changing transport layer
- Scalability
 - Divide network into sub-networks using bridges, pipeline stages, clock crossing logic

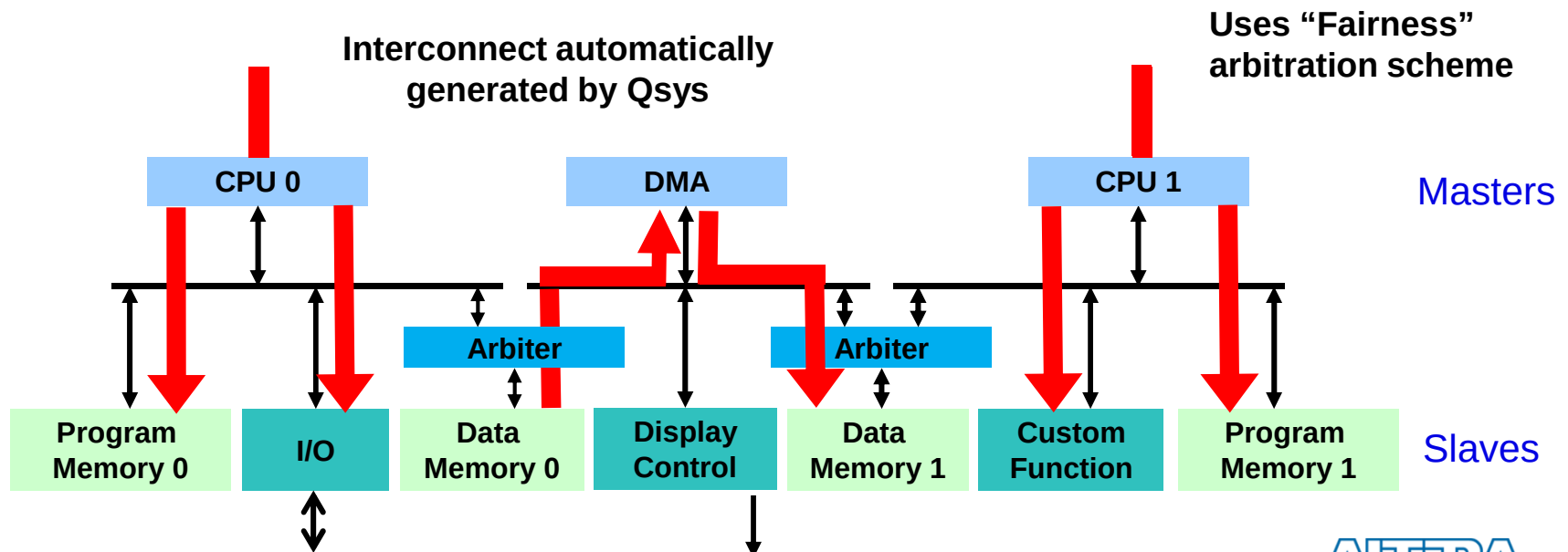
Qsys Component Interfaces

- Avalon Memory-Mapped (Avalon-MM)
 - Address-based, configurable read/write interface that facilitates peripheral development for a Qsys system
 - **Master** → **slave** connections through the fabric
- Avalon Streaming (Avalon-ST)
 - Data streaming interface enabling development of high bandwidth low latency components in an Qsys system
 - Point-to-point connections (**source** → **sink**)

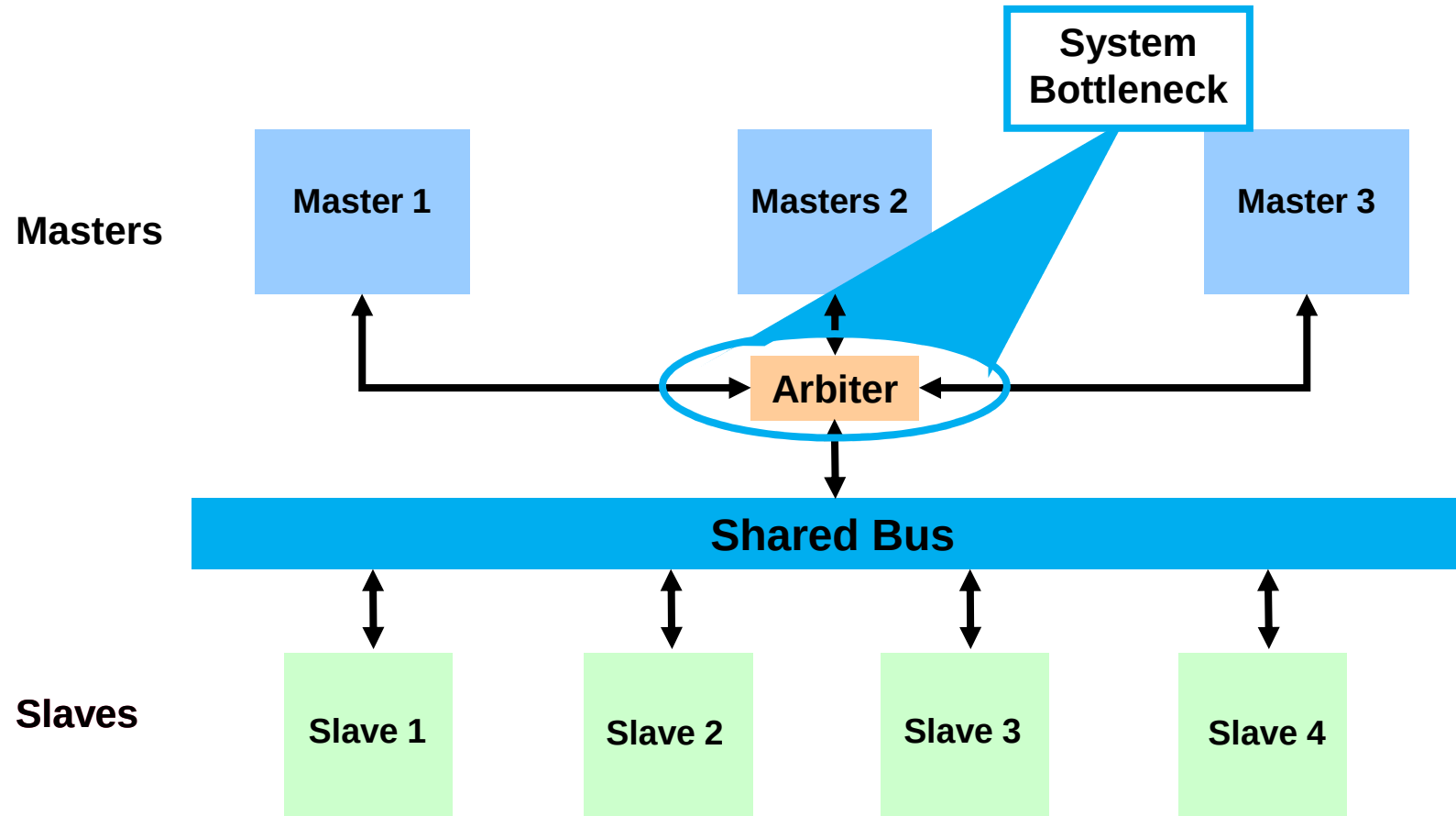
Peripherals need only implement specific signal types needed to support desired transfers

Enables Simultaneous Multi-Mastering

- Qsys Interconnect composed of FPGA routing resources
 - Implements slave-side arbitration
 - Multiple simultaneous bus transactions possible
 - Provided they don't access same slave during bus cycle
 - I/O devices can be grouped based on bandwidth requirements
- Trade-Off
 - Hardware Resource Usage Increases



Versus a “Traditional” Bus Architecture with Master Side Arbitration

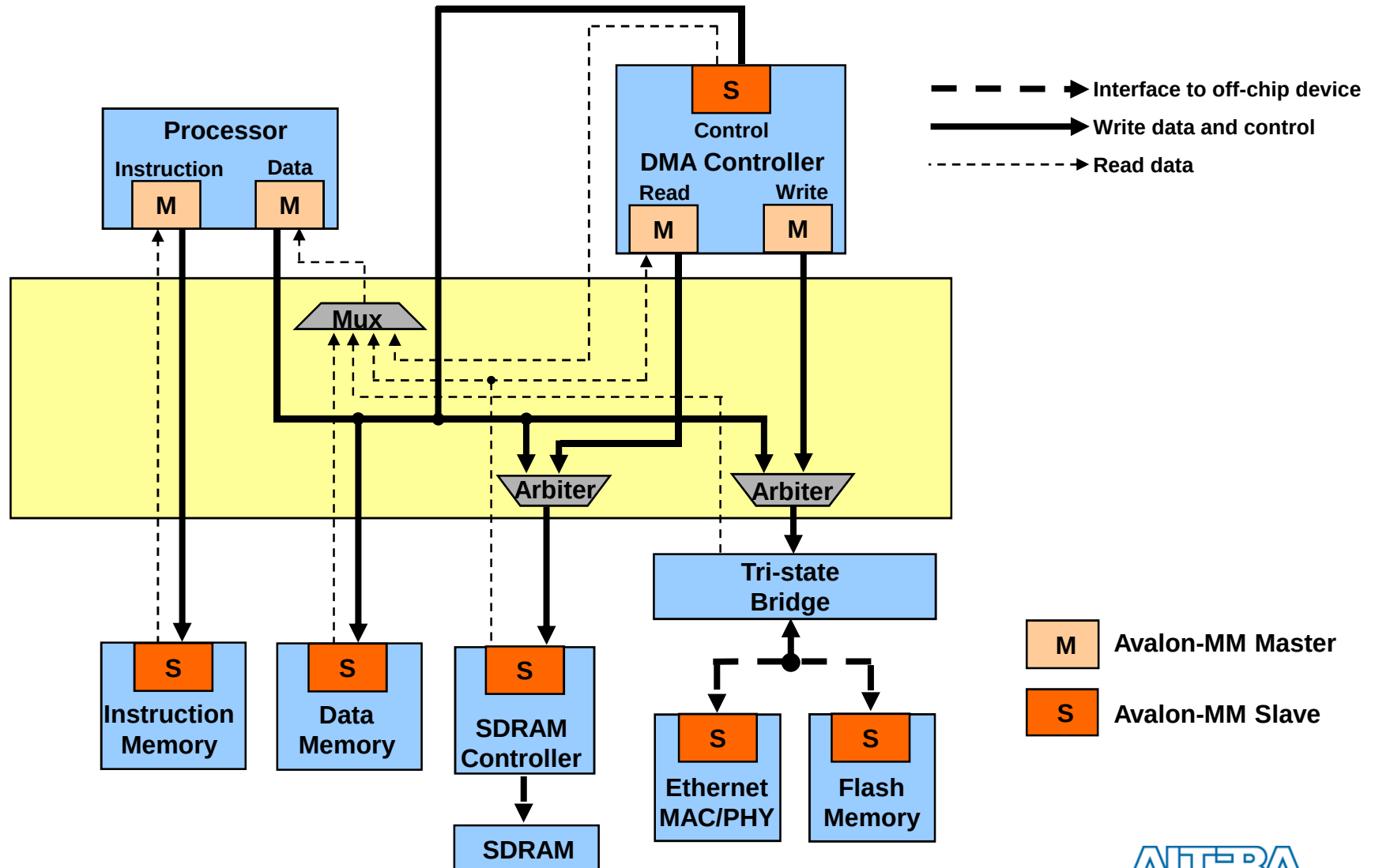


...where masters share and must wait for the bus

Qsys Component Terminology

- **Interface**
 - Group of Avalon-MM or Avalon-ST signals that describes connectivity between components in a Qsys system
- **Peripheral (i.e. component)**
 - Logical device that has one or more Avalon interfaces
- **Master**
 - An interface that initiates Avalon-MM transfers
- **Slave**
 - An interface that responds to Avalon-MM transfers
- **Source / Sink**
 - Interfaces that send / receive streaming data through Qsys system
- **Transfer**
 - Read or write of a unit of data (with fixed or variable latency)

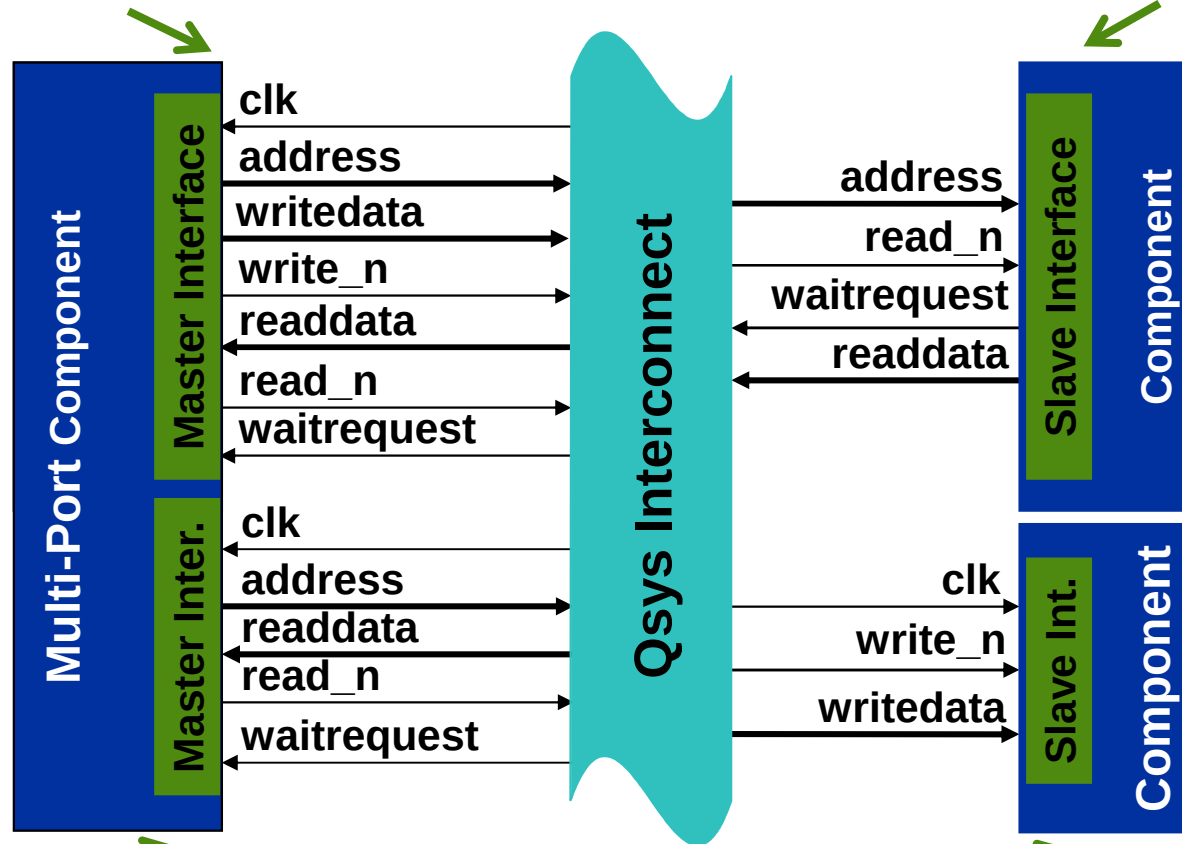
Block Diagram – Components with Avalon-MM Interfaces



Example Avalon-MM Components

Master Interface supporting
read/write transfers

Slave Interface supporting write
transfers with variable latency



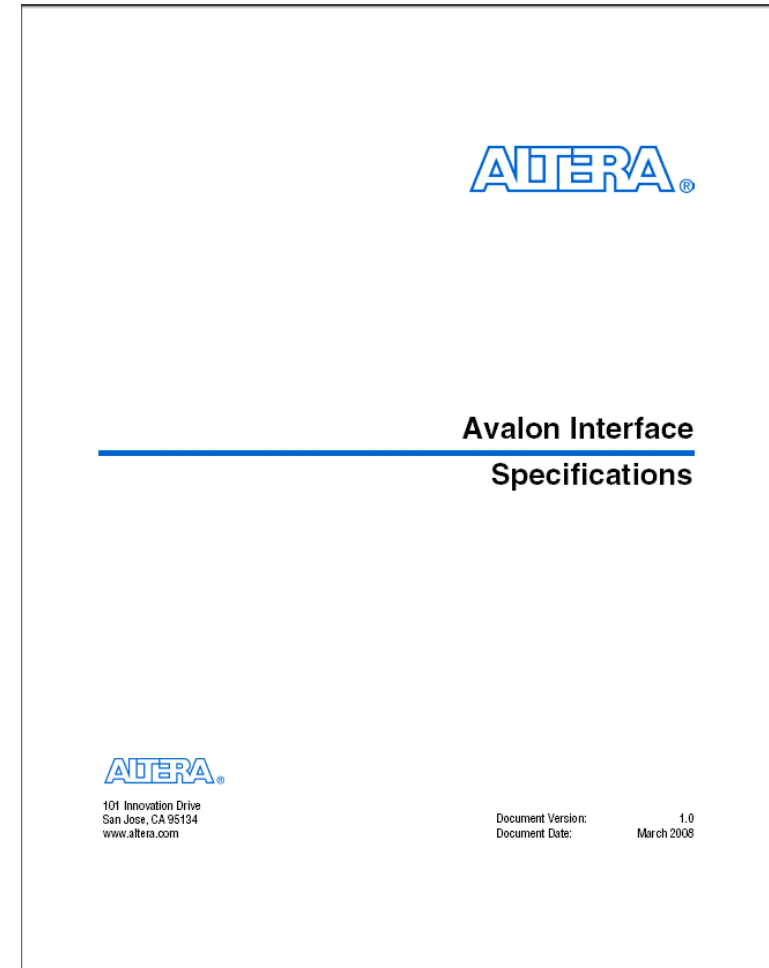
Master Interface supporting
read transfers only

Slave Interface supporting
write transfers only (fixed
or no latency)

Avalon Interface Specification

- Defines the entire Avalon Interface standard
- Provides reference information on additional transfer types
 - Use cases
 - Waveform diagrams

www.altera.com/literature/manual/mnl_avalon_spec.pdf



Designing with the Nios II Processor and Qsys

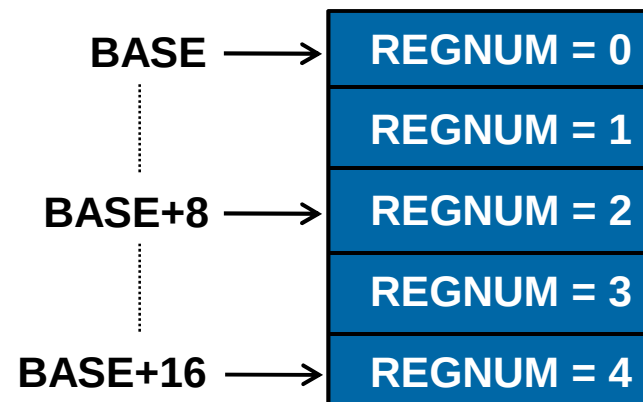
Accessing Peripherals from Nios II

Thoughts on Data Cache

- Data variables are cached by default if you have a processor with data cache
 - Data cache is direct-mapped, write-back
- Memory space is mirrored - 2GB addressable space, not 4GB
 - Lower half is cacheable
 - Upper half is un-cacheable and un-reachable
 - Bit 31 is a control bit used to disable cache
 - *Cannot locate peripherals in upper half of memory*

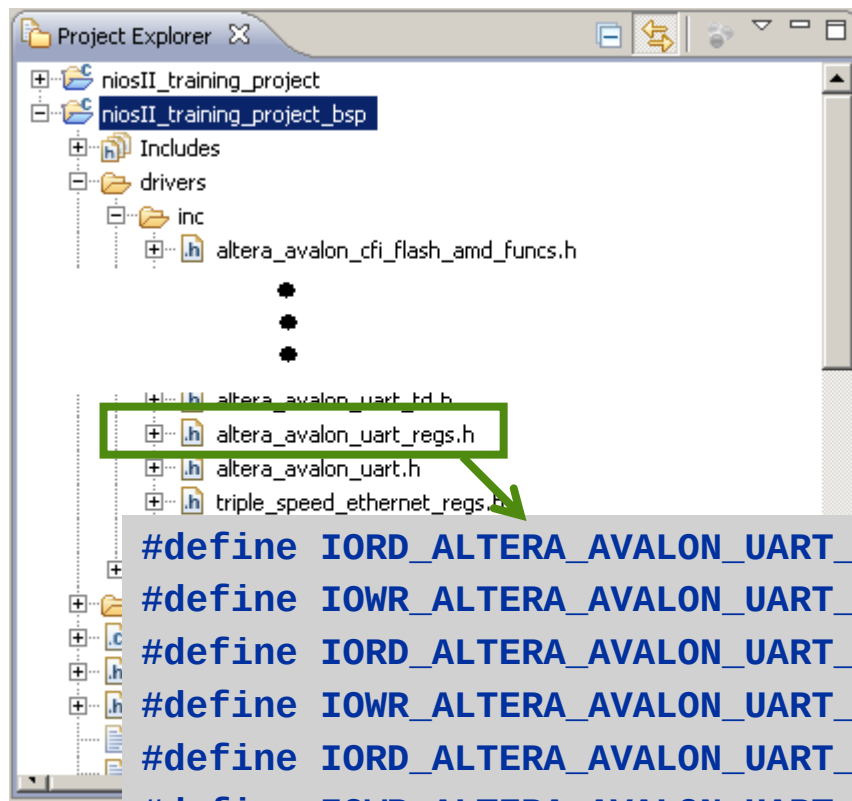
Reading/Writing Hardware in Nios II

- Use I/O macros to access hardware peripherals
 - I/O macros bypass the cache for hardware accesses
 - They use STxIO or LDxIO instructions
 - IORD(BASE, REGNUM)
 - IOWR(BASE, REGNUM, DATA)



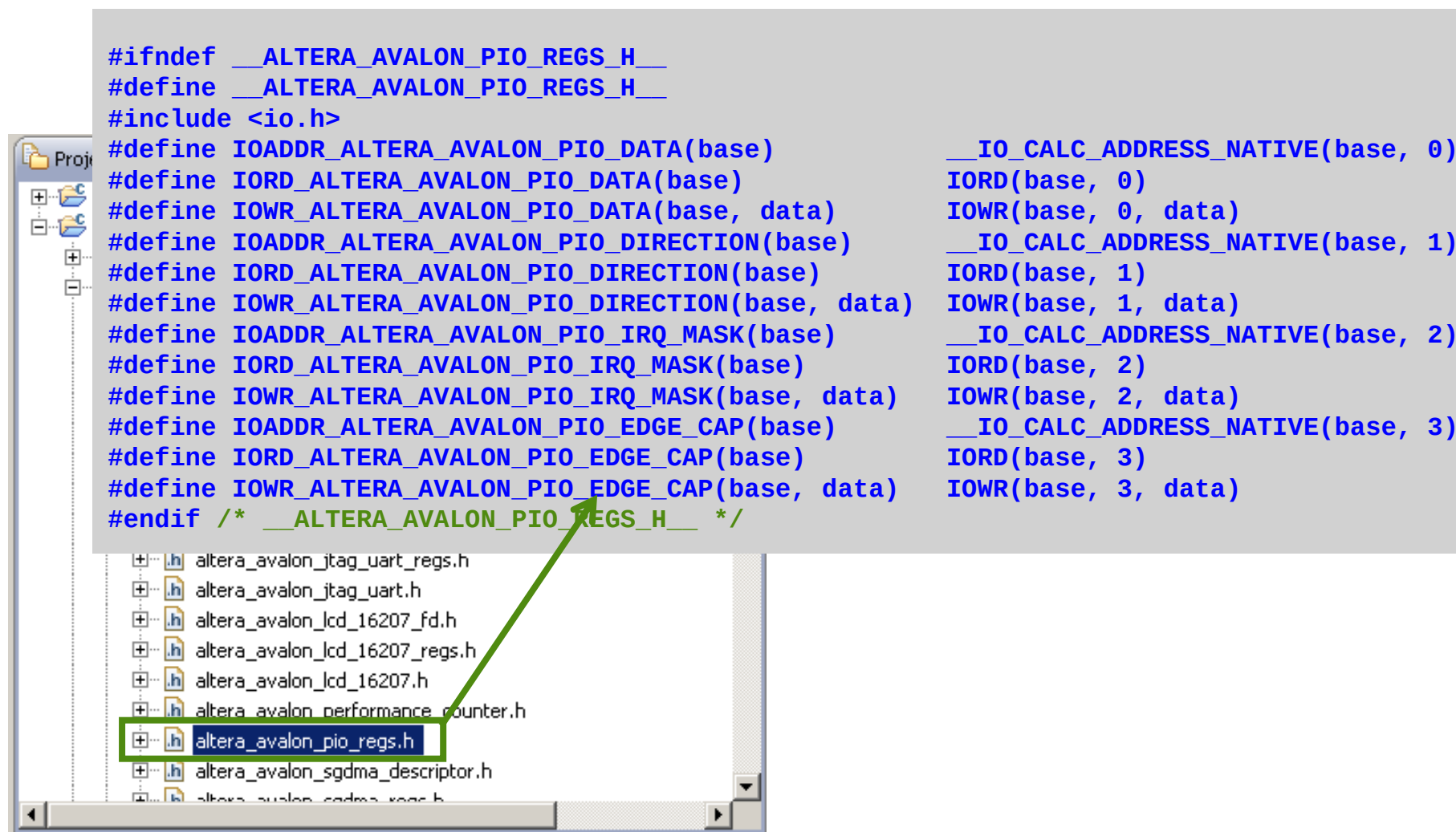
Header Files for Nios II Peripherals (1)

- Each Nios II peripheral has specific read/write macros for each register



```
#define IORD_ALTERA_AVALON_UART_RXDATA(base)      IORD(base, 0)
#define IOWR_ALTERA_AVALON_UART_RXDATA(base, data) IOWR(base, 0, data)
#define IORD_ALTERA_AVALON_UART_TXDATA(base)      IORD(base, 1)
#define IOWR_ALTERA_AVALON_UART_TXDATA(base, data) IOWR(base, 1, data)
#define IORD_ALTERA_AVALON_UART_STATUS(base)      IORD(base, 2)
#define IOWR_ALTERA_AVALON_UART_STATUS(base, data) IOWR(base, 2, data)
```

Header Files for Nios II Peripherals (2)



```
#ifndef __ALTERA_AVALON_PIO_REGS_H__
#define __ALTERA_AVALON_PIO_REGS_H__
#include <io.h>
#define IOADDR_ALTERA_AVALON_PIO_DATA(base)      __IO_CALC_ADDRESS_NATIVE(base, 0)
#define IORD_ALTERA_AVALON_PIO_DATA(base)        IORD(base, 0)
#define IOWR_ALTERA_AVALON_PIO_DATA(base, data)  IOWR(base, 0, data)
#define IOADDR_ALTERA_AVALON_PIO_DIRECTION(base) __IO_CALC_ADDRESS_NATIVE(base, 1)
#define IORD_ALTERA_AVALON_PIO_DIRECTION(base)  IORD(base, 1)
#define IOWR_ALTERA_AVALON_PIO_DIRECTION(base, data) IOWR(base, 1, data)
#define IOADDR_ALTERA_AVALON_PIO_IRQ_MASK(base) __IO_CALC_ADDRESS_NATIVE(base, 2)
#define IORD_ALTERA_AVALON_PIO_IRQ_MASK(base)   IORD(base, 2)
#define IOWR_ALTERA_AVALON_PIO_IRQ_MASK(base, data) IOWR(base, 2, data)
#define IOADDR_ALTERA_AVALON_PIO_EDGE_CAP(base) __IO_CALC_ADDRESS_NATIVE(base, 3)
#define IORD_ALTERA_AVALON_PIO_EDGE_CAP(base)   IORD(base, 3)
#define IOWR_ALTERA_AVALON_PIO_EDGE_CAP(base, data) IOWR(base, 3, data)
#endif /* __ALTERA_AVALON_PIO_REGS_H__ */
```

The screenshot shows an IDE window with the header file `altera_avalon_pio_regs.h` open. The file content is as follows:

```
#ifndef __ALTERA_AVALON_PIO_REGS_H__
#define __ALTERA_AVALON_PIO_REGS_H__
#include <io.h>
#define IOADDR_ALTERA_AVALON_PIO_DATA(base)      __IO_CALC_ADDRESS_NATIVE(base, 0)
#define IORD_ALTERA_AVALON_PIO_DATA(base)        IORD(base, 0)
#define IOWR_ALTERA_AVALON_PIO_DATA(base, data)  IOWR(base, 0, data)
#define IOADDR_ALTERA_AVALON_PIO_DIRECTION(base) __IO_CALC_ADDRESS_NATIVE(base, 1)
#define IORD_ALTERA_AVALON_PIO_DIRECTION(base)  IORD(base, 1)
#define IOWR_ALTERA_AVALON_PIO_DIRECTION(base, data) IOWR(base, 1, data)
#define IOADDR_ALTERA_AVALON_PIO_IRQ_MASK(base) __IO_CALC_ADDRESS_NATIVE(base, 2)
#define IORD_ALTERA_AVALON_PIO_IRQ_MASK(base)   IORD(base, 2)
#define IOWR_ALTERA_AVALON_PIO_IRQ_MASK(base, data) IOWR(base, 2, data)
#define IOADDR_ALTERA_AVALON_PIO_EDGE_CAP(base) __IO_CALC_ADDRESS_NATIVE(base, 3)
#define IORD_ALTERA_AVALON_PIO_EDGE_CAP(base)   IORD(base, 3)
#define IOWR_ALTERA_AVALON_PIO_EDGE_CAP(base, data) IOWR(base, 3, data)
#endif /* __ALTERA_AVALON_PIO_REGS_H__ */
```

The file explorer on the left shows the project structure, with `altera_avalon_pio_regs.h` highlighted. A green arrow points from the highlighted file in the explorer to the corresponding line in the code window.

Nios II Custom Peripherals

- Custom hardware also requires a header file
- Add header/driver code directly to Application Project

```
#ifndef __ALTERA_AVALON_PWM_REGS_H__
#define __ALTERA_AVALON_PWM_REGS_H__
#include <io.h>

#define IORD_ALTERA_AVALON_PWM_DIVIDER(base)      IORD(base, 0)
#define IOWR_ALTERA_AVALON_PWM_DIVIDER(base, data) IOWR(base, 0, data)

#define IORD_ALTERA_AVALON_PWM_DUTY(base)         IORD(base, 1)
#define IOWR_ALTERA_AVALON_PWM_DUTY(base, data)   IOWR(base, 1, data)

#endif /* __ALTERA_AVALON_PWM_REGS_H__ */
```

Example Nios II Program

```
#include "system.h"
#include "altera_avalon_pio_regs.h" // Write a program to shift LEDs right across the
#include <unistd.h>                  // dev board by pressing any of the 4 SW buttons
#define NONE_PRESSED 0xF           // no button press value
#define DEBOUNCE 30000             // Time in microseconds to wait for switch debounce

int main(void) {
    int buttons;                   // Use to hold button value
    int led = 0x01;                // Use to write to led

    while (1)
    {
        // Read buttons via pio
        buttons = IORD_ALTERA_AVALON_PIO_DATA(BUTTON_PIO_BASE);
        if (buttons != NONE_PRESSED) // if button pressed
        {
            if (led >= 0x80)         // if pattern is 00000001 on board
                led = 0x01;          // reset pattern
            else
                led = led << 1;       // shift right on board

            IOWR_ALTERA_AVALON_PIO_DATA(LED_PIO_BASE, led); // Write new value
            // Switch debounce routine not shown
        }
    }
}
```

From altera_avalon_pio_reg.h file

Peripheral name comes from system header file

Designing with the Nios II Processor and Qsys

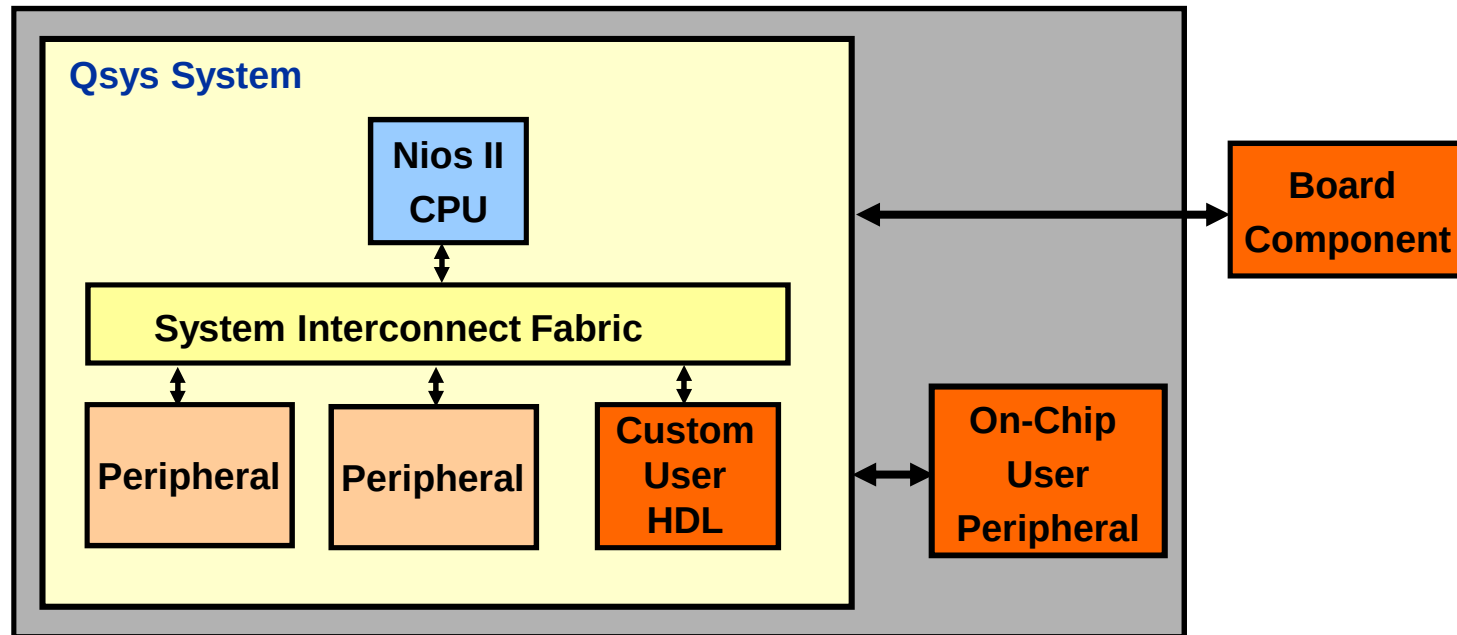
Importing User-Defined
Custom Peripherals into Qsys

Custom Peripherals

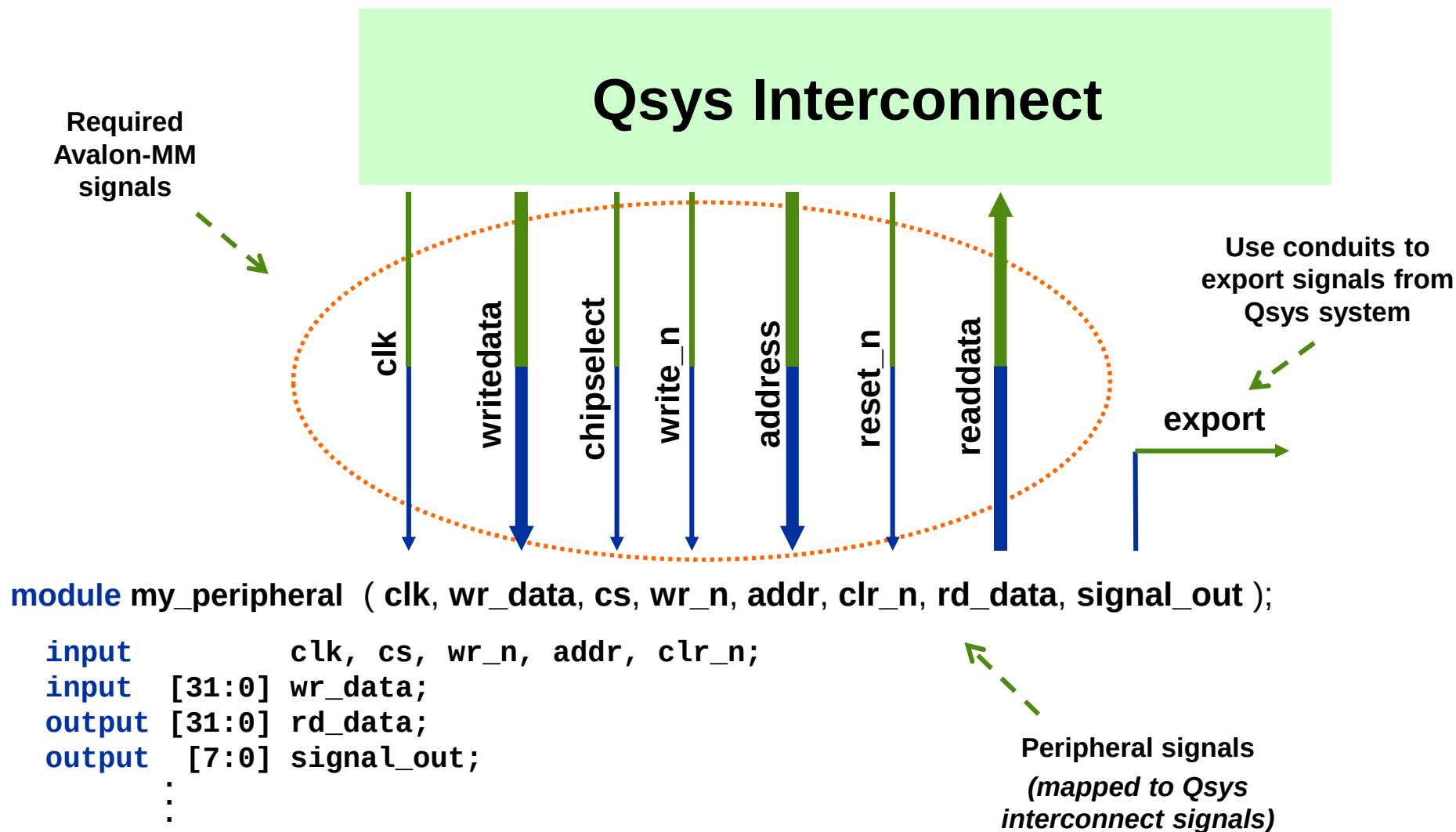
- You may wish to add a peripheral not included with Qsys
 - To perform some kind of proprietary function or perhaps a standard function that is not yet included as part of the kit
 - To expand or accelerate system capabilities
- Custom Peripherals connect directly to the system through the Qsys Interconnect
 - Signal mappings are defined in a component TCL file (`_hw.tcl`)
 - Edit by hand or create through Qsys Component Editor tool

Custom Peripherals

- Map into Nios II processor memory space
- Can be on-chip or off-chip
 - HDL code or an external component on your board
 - HDL can live inside Qsys system or out

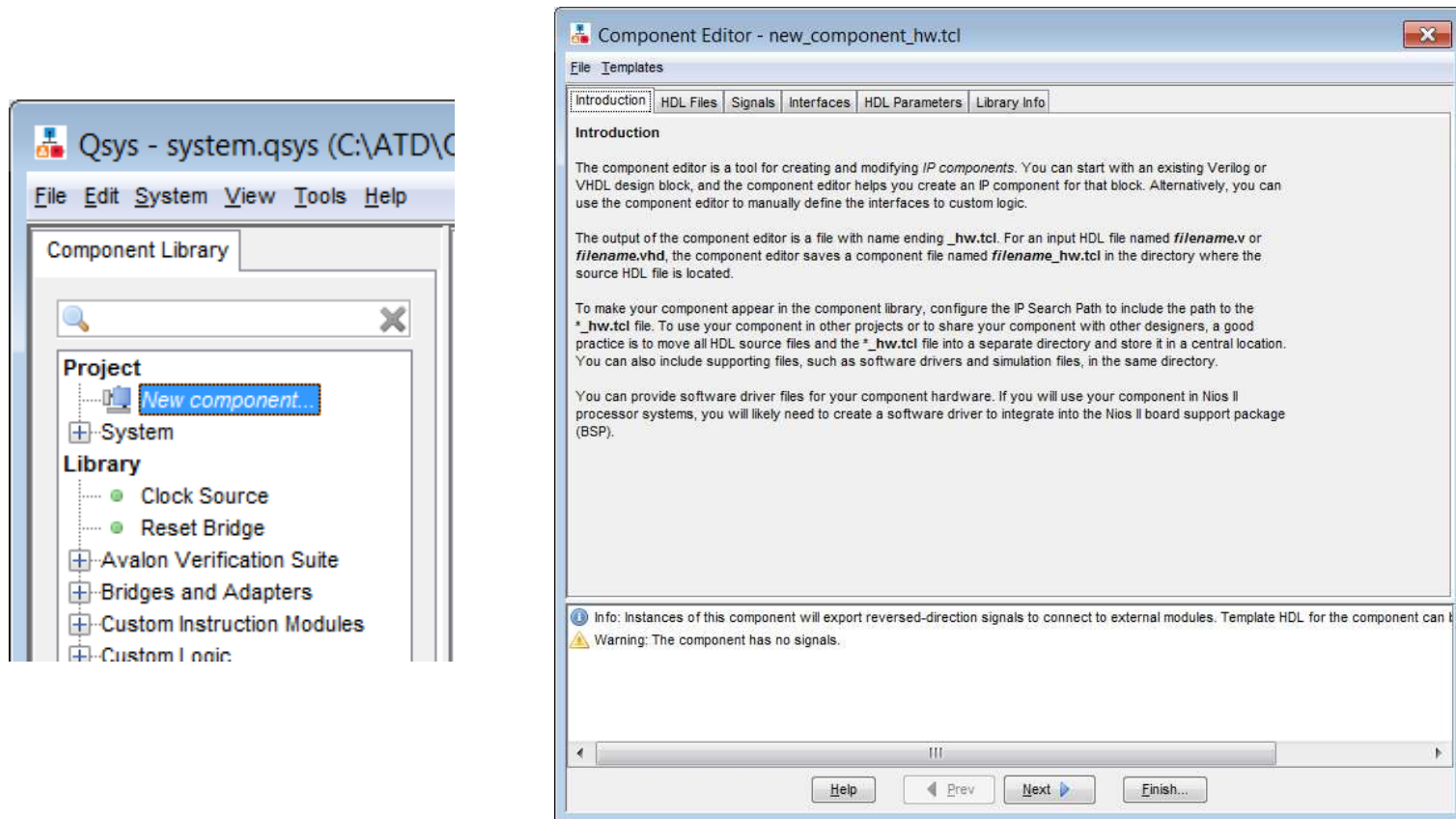


Example Peripheral Signal Mapping



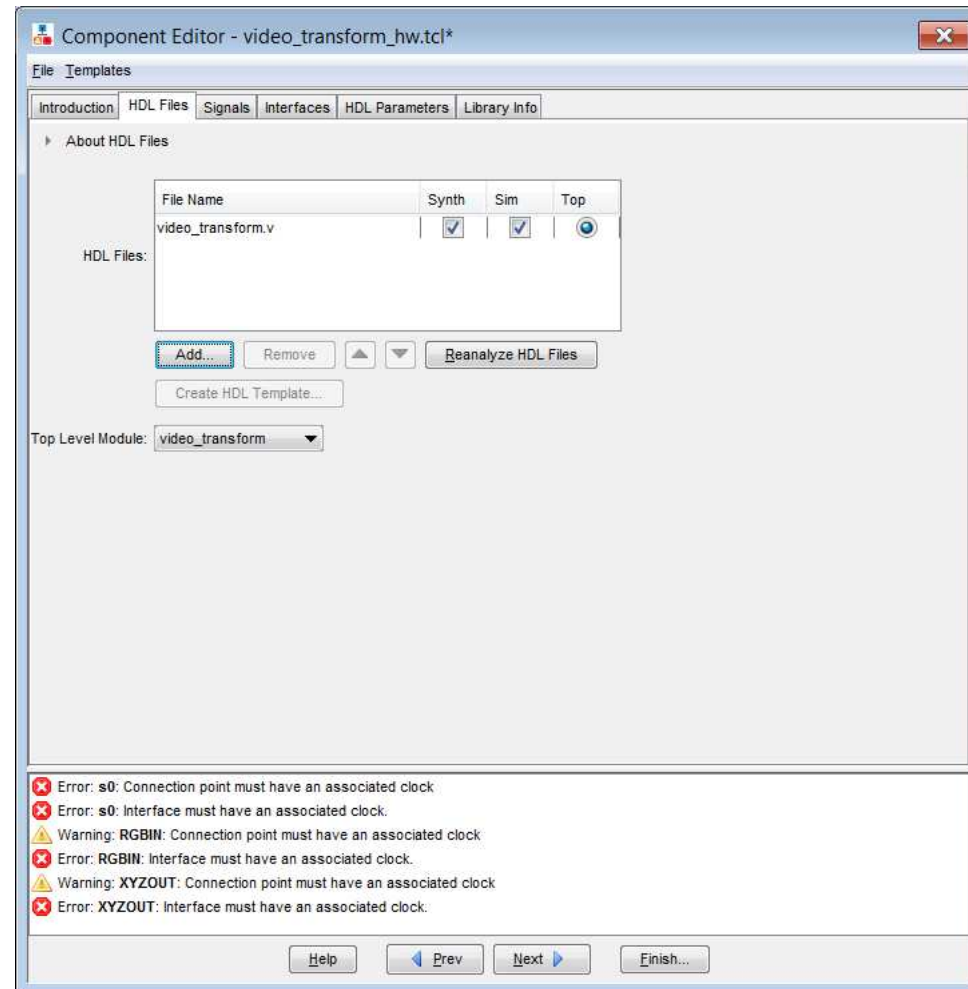
Component Editor

- Used to import peripherals into Qsys system
 - Launch from Qsys pick list or **File** menu



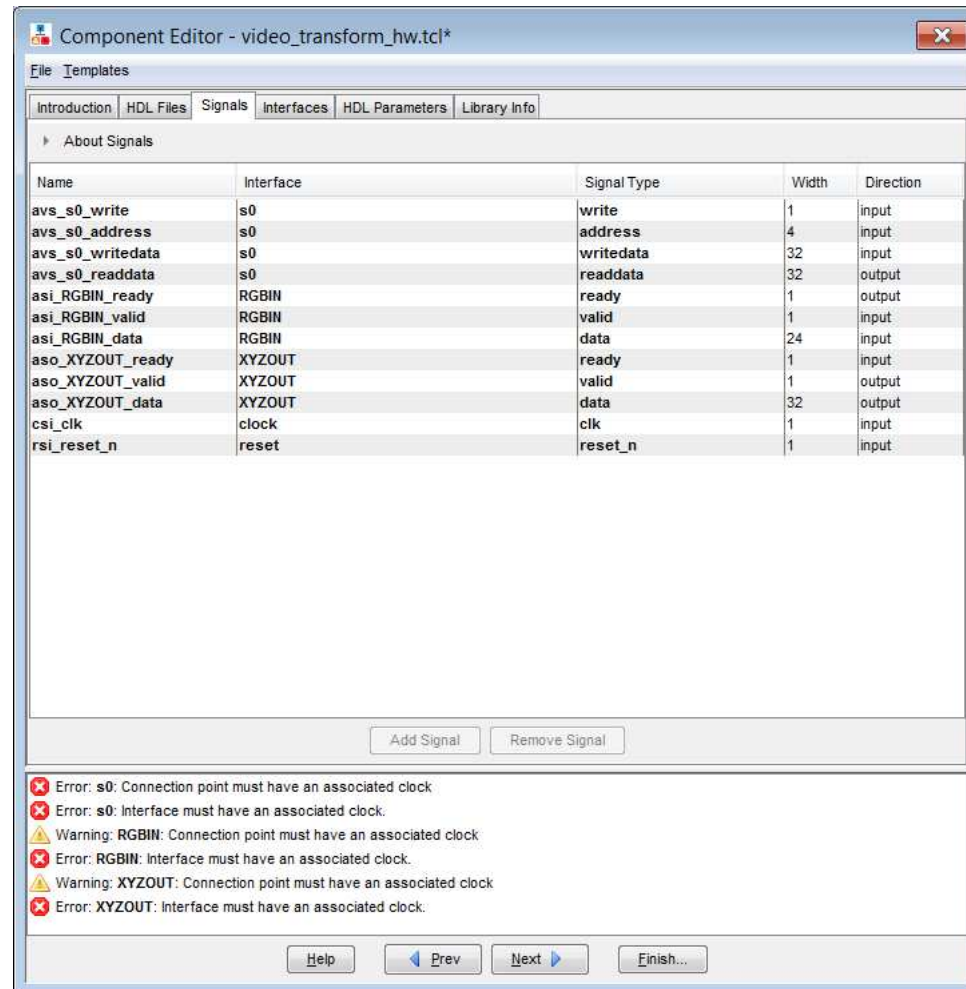
Component Editor (HDL Files Tab)

- Add HDL files for peripheral on the **HDL Files** tab
 - **Add...** then browse for peripheral code
- Specify top level module



Component Editor (Signals Tab)

- Map component signals to Qsys Interface types and Signal Names on the **Signals** tab



Auto-Port Mapping Naming Convention

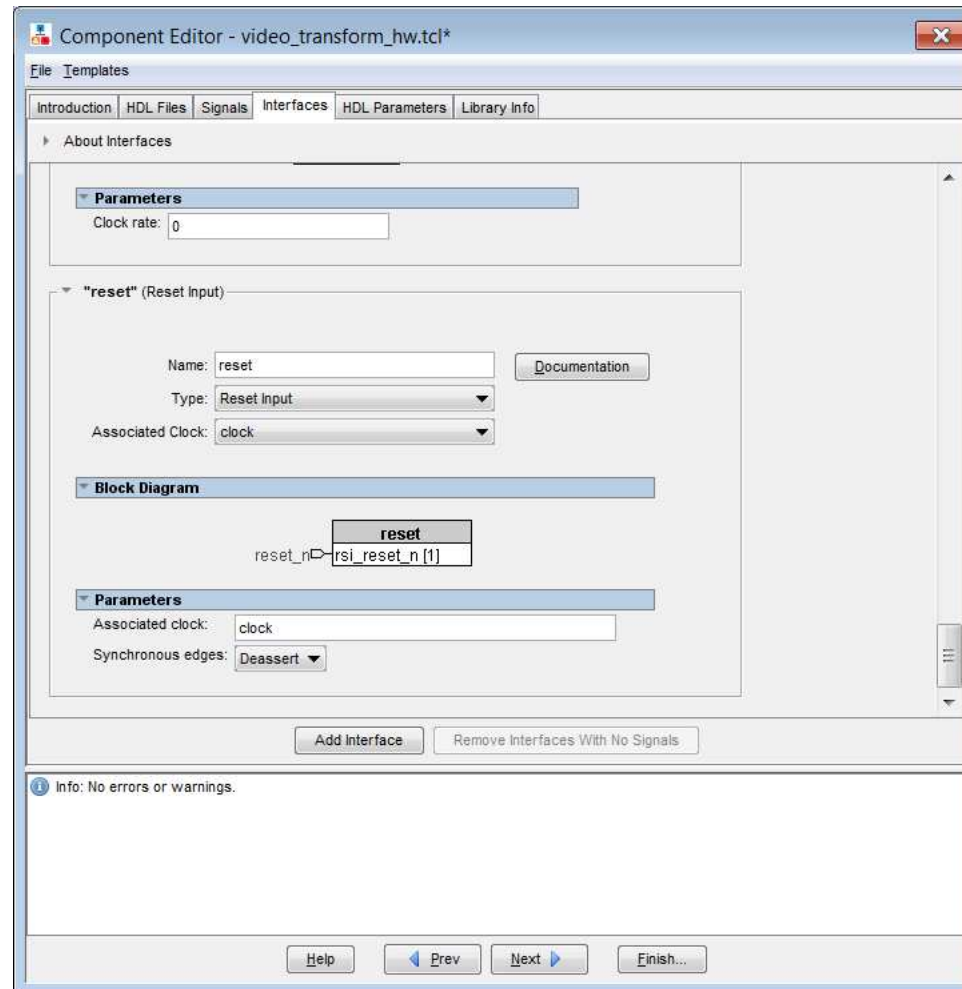
■ Can save time in Component Editor

- e.g. **avs_s0_readdata** auto-maps to an Avalon-MM slave interface (**avs**) called “**s0**” with signal type **readdata**

Value	Meaning
avs	Avalon-MM slave
avm	Avalon-MM master
aso	Avalon-ST source
asi	Avalon-ST sink
cs0	Clock output
csi	Clock input
coe	Conduit
inr	Interrupt receiver
ins	Interrupt sender
ncm	Nios II custom instruction master
ncs	Nios II custom instruction slave
rsi	Reset sink
rso	Reset source
tcm	Avalon-TC master
tcs	Avalon-TC slave

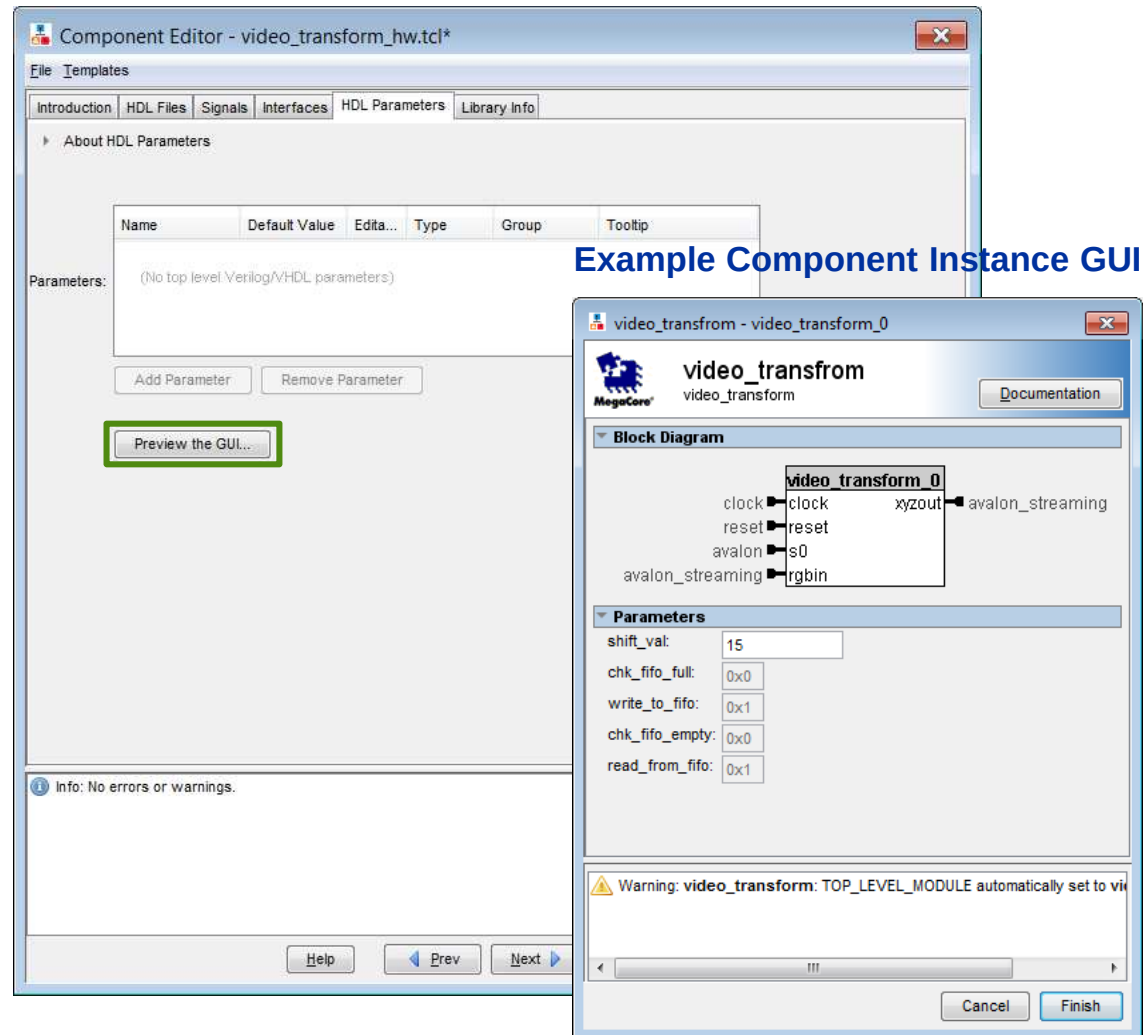
Component Editor (Interfaces Tab)

- Define properties for all component interfaces on the Interfaces tab
- Each Avalon Memory-Mapped interface must have an associated clock and reset
- Avalon-ST interfaces must have associated clocks
- Reset interfaces must also have associated clocks



Component Editor (HDL Parameters)

- If component has HDL parameters, they show up on **HDL Parameters** tab
- Select which parameters can be edited at instantiation time in Parameters window



Component Editor (Library Info)

- On **Library Info** tab, name the peripheral and select the Qsys Library picklist folder where it will live (i.e. the Group)
- Other misc. annotations can be added, as well

The screenshot shows the 'Component Editor - video_transform_hw.tcl*' window with the 'Library Info' tab selected. The 'Parameters' section contains the following fields:

- Name: video_transform
- Display name: video_transport
- Version: 1.0
- Group: Custom Logic
- Description: (empty)
- Created by: (empty)
- Icon: (empty)
- Documentation: (empty table with columns Title and URL)

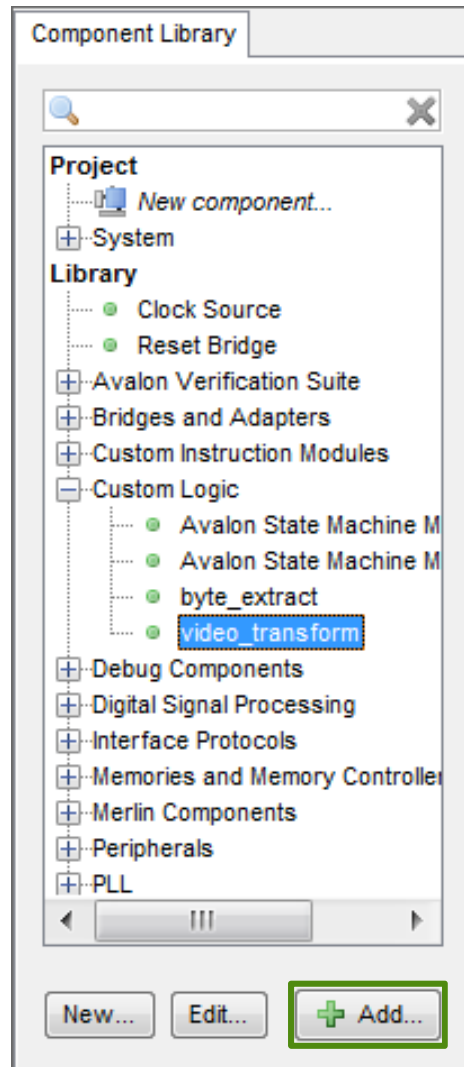
At the bottom of the window, there is a status bar with the message 'Info: No errors or warnings.' and navigation buttons: Help, Prev, Next, and Finish...

“_hw.tcl” File

- Generated by Component Editor
 - Describes component settings from Component Editor
- This file plus HDL code all you need to import a component into future projects
- Component appears in Library pick-list in the folder specified in the **Group** field on the Library Info tab



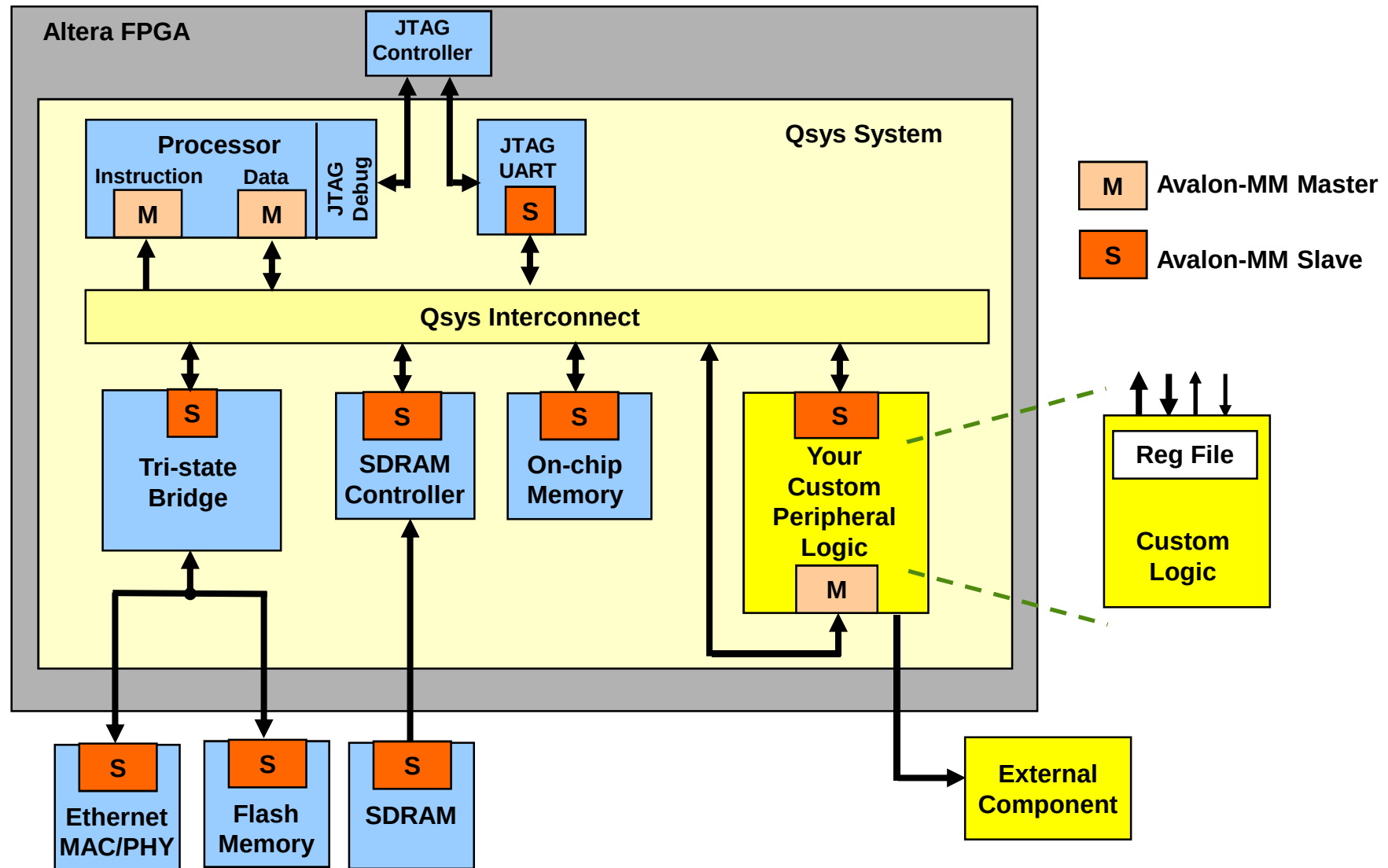
Import Peripheral Into Qsys System



- Find peripheral in “Custom Logic” folder and **Add** to system
- Wire up interfaces to appropriate blocks within system
- Export signals to connect outside of Qsys system if desired
 - Signals appear as inputs or outputs in Qsys HDL output block

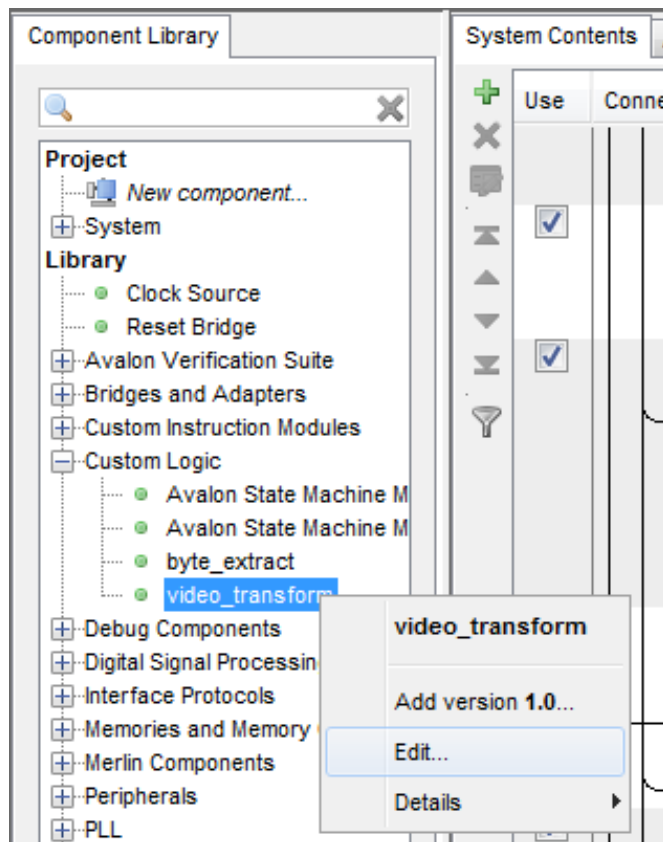
	☒ CHO_TRANSFORM	video_transform		
	RGBIN	Avalon Streaming Sink	Click to export	[clock]
	clock	Clock Input	Click to export	sys_clk
	s0	Avalon Memory Mapped Slave	Click to export	[clock]
	XYZOUT	Avalon Streaming Source	Click to export	[clock]
	reset	Reset Input	Click to export	[clock]

Custom Peripheral Integration Into SIF



Edit Component Parameters in Place

- **Right Click > Edit...**
 - Re-launches Component Editor

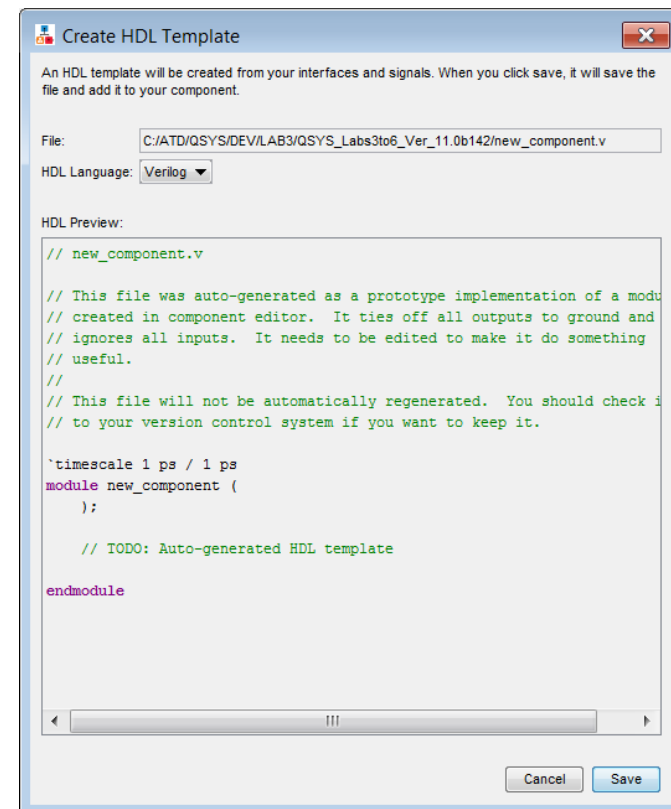
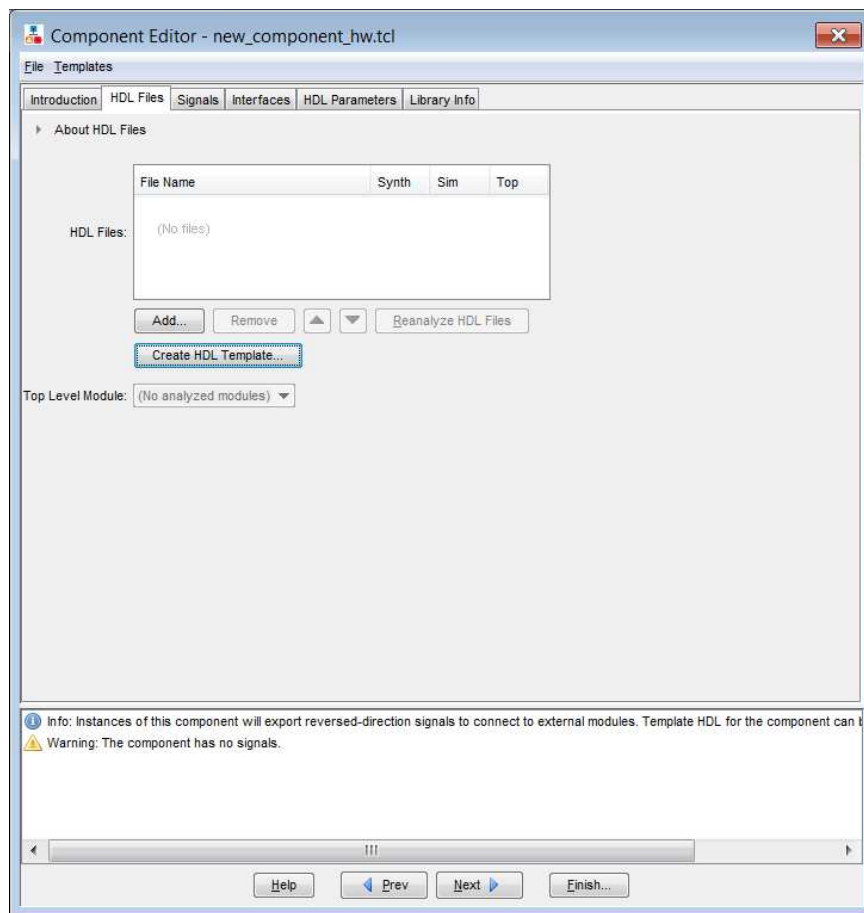


- Must refresh system after for changes to take effect
 - **File > Refresh System**

Qsys will search all search paths for `_hw.tcl` files and re-read them

Other Uses for Component Editor

- Create Custom Component HDL skeleton file
 - Select **Create HDL Template**



Several Templates Available

- e.g. Signals in Avalon-MM Master HDL template

The image displays two screenshots of the Altera Component Editor. The left screenshot shows the 'Component Editor - new_component_hw.tcl' window with the 'Templates' tab selected. A green box highlights the 'Add Typical Avalon-MM Master' option in the list. The right screenshot shows the 'Component Editor - new_component_hw.tcl' window with the 'Signals' tab selected, displaying a table of signals for the 'Add Typical Avalon-MM Master' template.

Name	Interface	Signal Type	Width	Direction
avm_m0_address	m0	address	8	output
avm_m0_read	m0	read	1	output
avm_m0_waitrequest	m0	waitrequest	1	input
avm_m0_readdata	m0	readdata	32	input
avm_m0_write	m0	write	1	output
avm_m0_writedata	m0	writedata	32	output
avm_m0_readdatavalid	m0	readdatavalid	1	input
clk	clock	clk	1	input
reset	reset	reset	1	input
avm_m1_address	m1	address	8	output
avm_m1_read	m1	read	1	output
avm_m1_waitrequest	m1	waitrequest	1	input
avm_m1_readdata	m1	readdata	32	input
avm_m1_write	m1	write	1	output
avm_m1_writedata	m1	writedata	32	output
avm_m1_readdatavalid	m1	readdatavalid	1	input

Please go to Exercise 3

Designing with the Nios II Processor and Qsys

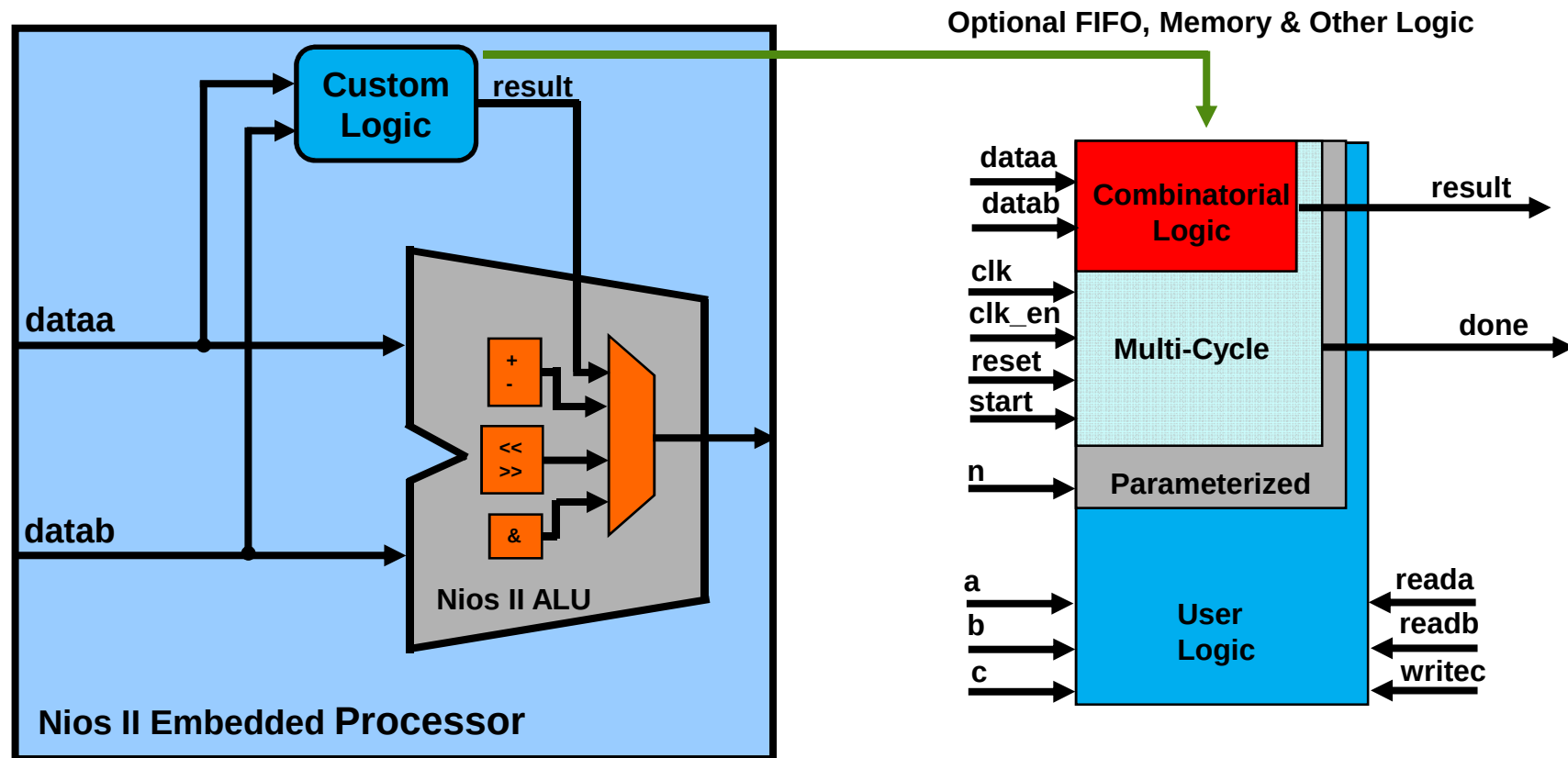
Nios II Processor Custom Instructions

Custom Instructions

- Add custom functionality to the Nios II processor design
 - To take full advantage of the flexibility of FPGA
- Dramatically boost processing performance
 - With no Increase in f_{MAX} required
- Application examples
 - Data stream processing (eg. network applications)
 - Application specific processing (eg. MP3 audio decode)
 - Software inner loop optimization

Custom Instructions

- Augment Nios II processor instruction set
 - Mux user logic into ALU path of processor pipeline

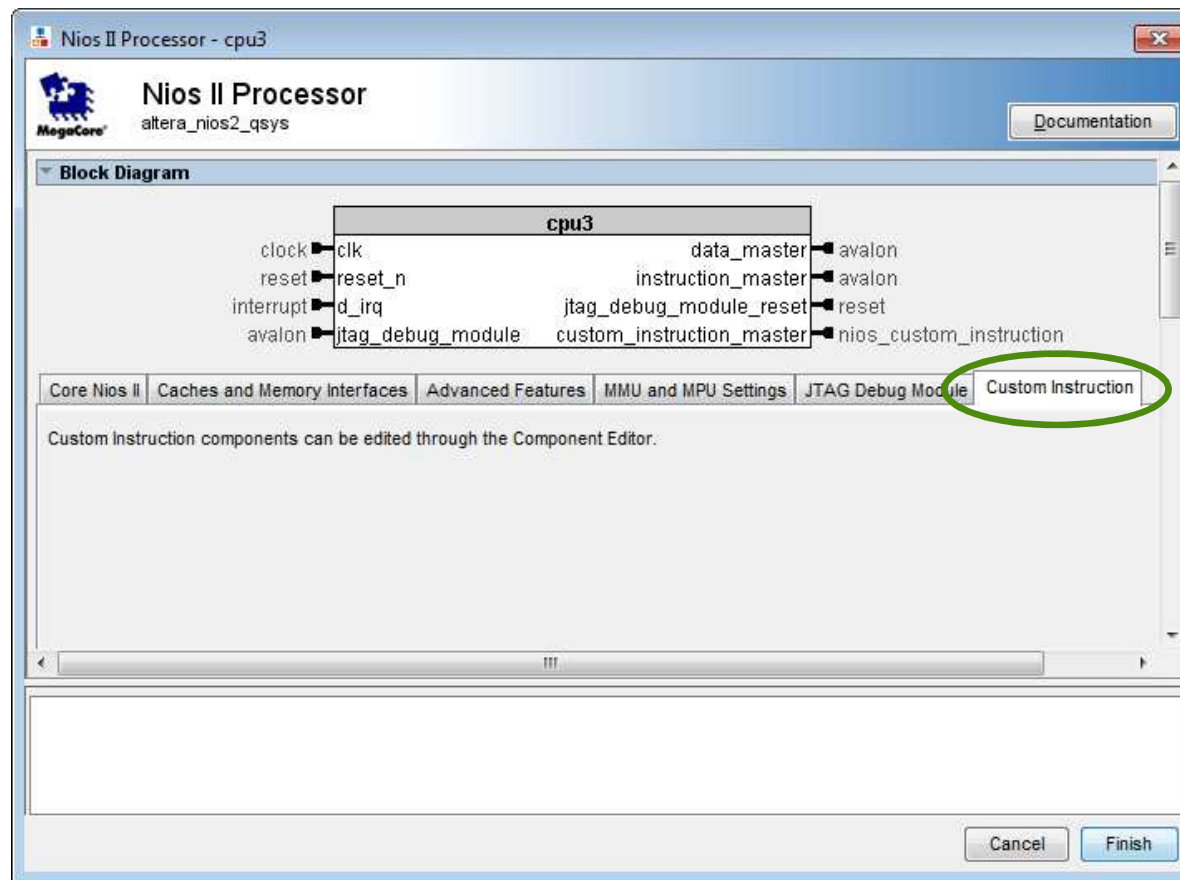


Custom Instructions

- Integrated Into Nios II Processor Development Tools
 - Qsys design tool handles op-code assignment
 - Generates C and assembly-language macros
 - Up to 256 different custom instructions possible
 - Multi-cycle instructions can have variable duration
 - Parameterization of custom instructions has changed

Nios II Processor Custom Instructions Tab

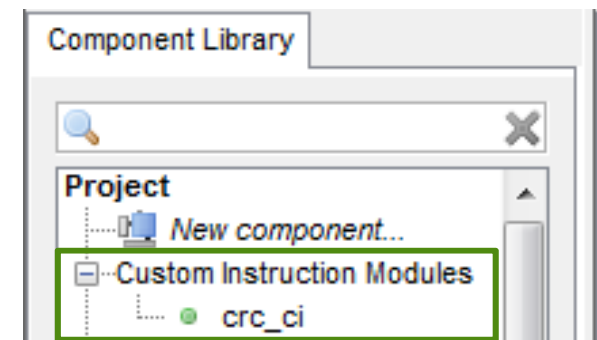
- No longer used to add Custom Instructions
 - Go to Component Editor instead to perform signal mapping



To Import Custom Instruction

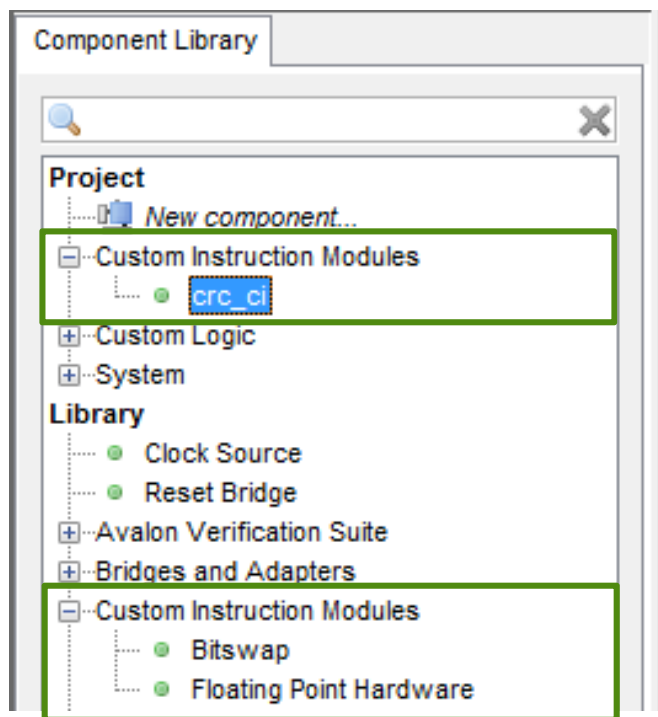
- Use Component Editor
 - Import HDL code
 - Map signals to **nios_custom_instruction** interface
 - Locate new instruction in **Custom Instruction Modules** folder in Qsys Library picklist
 - May have to refresh Qsys component list

...	Name	Interface	Signal Type	Width	Direction
reset	reset	nios_custom_instruction_slave	reset	1	input
clk	clk	nios_custom_instruction_slave	clk	1	input
start	start	nios_custom_instruction_slave	start	1	input
clk_en	clk_en	nios_custom_instruction_slave	clk_en	1	input
dataa	dataa	nios_custom_instruction_slave	dataa	32	input
datab	datab	nios_custom_instruction_slave	datab	32	input
result	result	nios_custom_instruction_slave	result	32	output



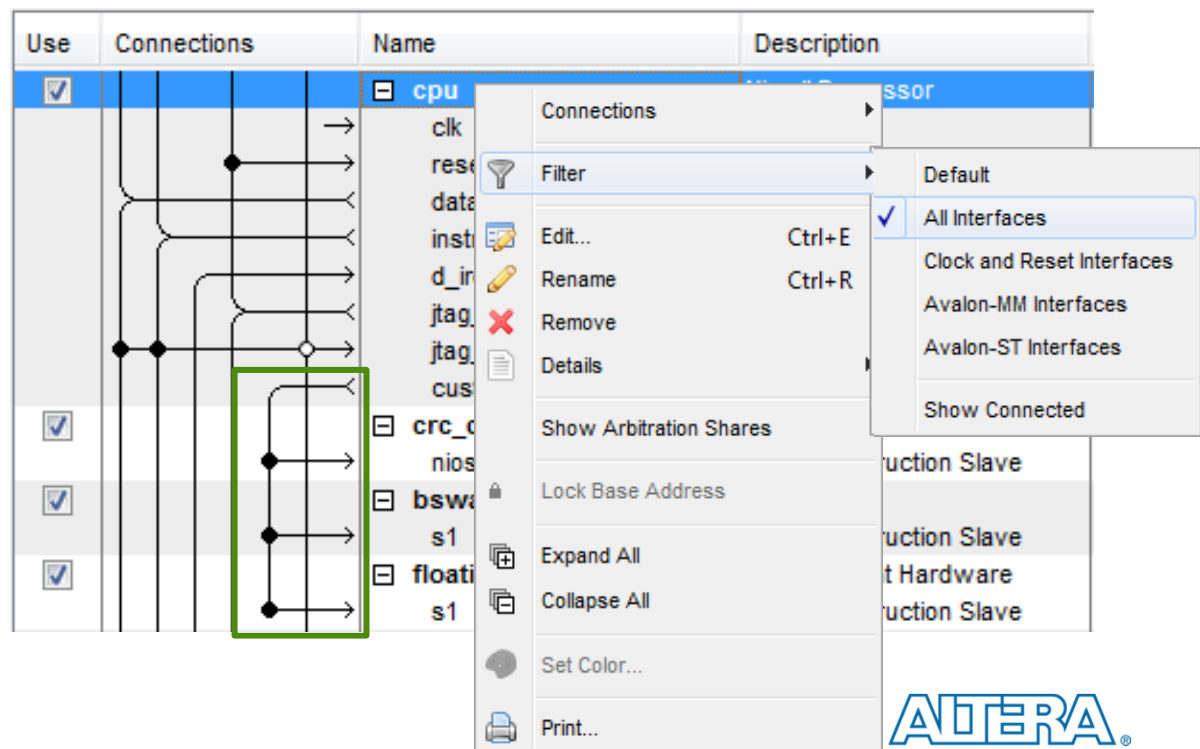
To Add Custom Instruction to System

- Add user-defined or Qsys library Custom Instructions to Nios II processor in Qsys system
 - Manually connect custom instruction interface to Nios II processor



Can rename them like
any other component

Not visible in Qsys system
unless filter turned on



To Remove Custom Instructions

- Manually delete `_hw.tcl` file from project
 - “<custom_instruction>_hw.tcl”
- Remove Custom Instruction from instantiated components in Qsys system

C Language Software Interface

- Nios II SBT for Eclipse generates macros automatically during software build process
- Macros defined in **system.h** file

```
#define ALT_CI_<your instruction_name>(instruction arguments)
```

- e.g. user C-code that references bitswap custom instruction

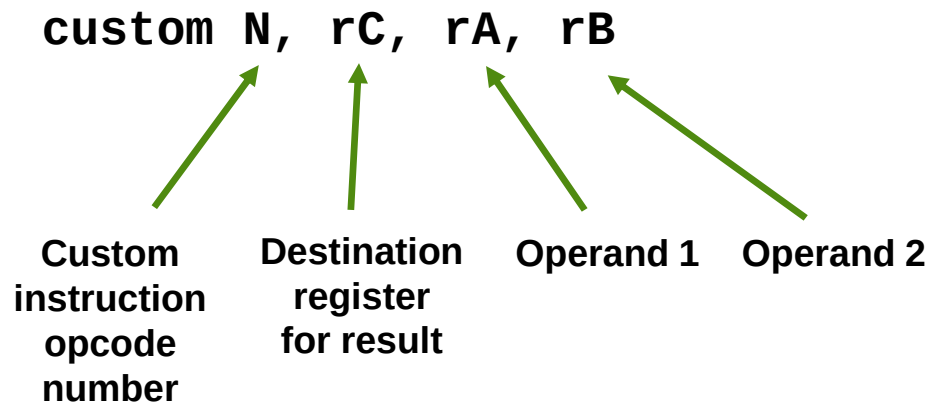
```
#include "system.h"
int main (void)
{
    int a = 0x12345678;
    int a_swap = 0;

    a_swap = ALT_CI_BSWAP(a);

    return 0;
}
```

Assembly Language Interface

- Assembler syntax for the custom instruction:



- Two Examples:

custom 0, r6, r7, r8
custom 3, c1, r2, c4

r = Nios II processor register
c = Custom Instruction internal register

Why Custom Instructions?

- Reduce complex sequence of instructions to few or one instruction
- Example: Floating point multiply (performed in 6 clock cycles)

Roughly 30x performance improvement

Significantly Faster!

- **Typical Flow**
 - Profile code
 - Identify critical inner loop
 - Create Custom Instruction logic
 - Replace one or all instructions in inner loop
 - Import Custom Instruction logic into design
 - Call Custom Instruction from C or assembly

Floating Point Custom Instructions

- Implement single precision floating-point arithmetic operations
 - Use custom instructions to accelerate floating-point operations in your application
- Available on every Nios II processor core
 - Includes single precision floating-point addition, subtraction, multiplication, and division
 - Floating-point division is available as an extension to the basic instruction set

Can You Use Integer Arithmetic Instead?

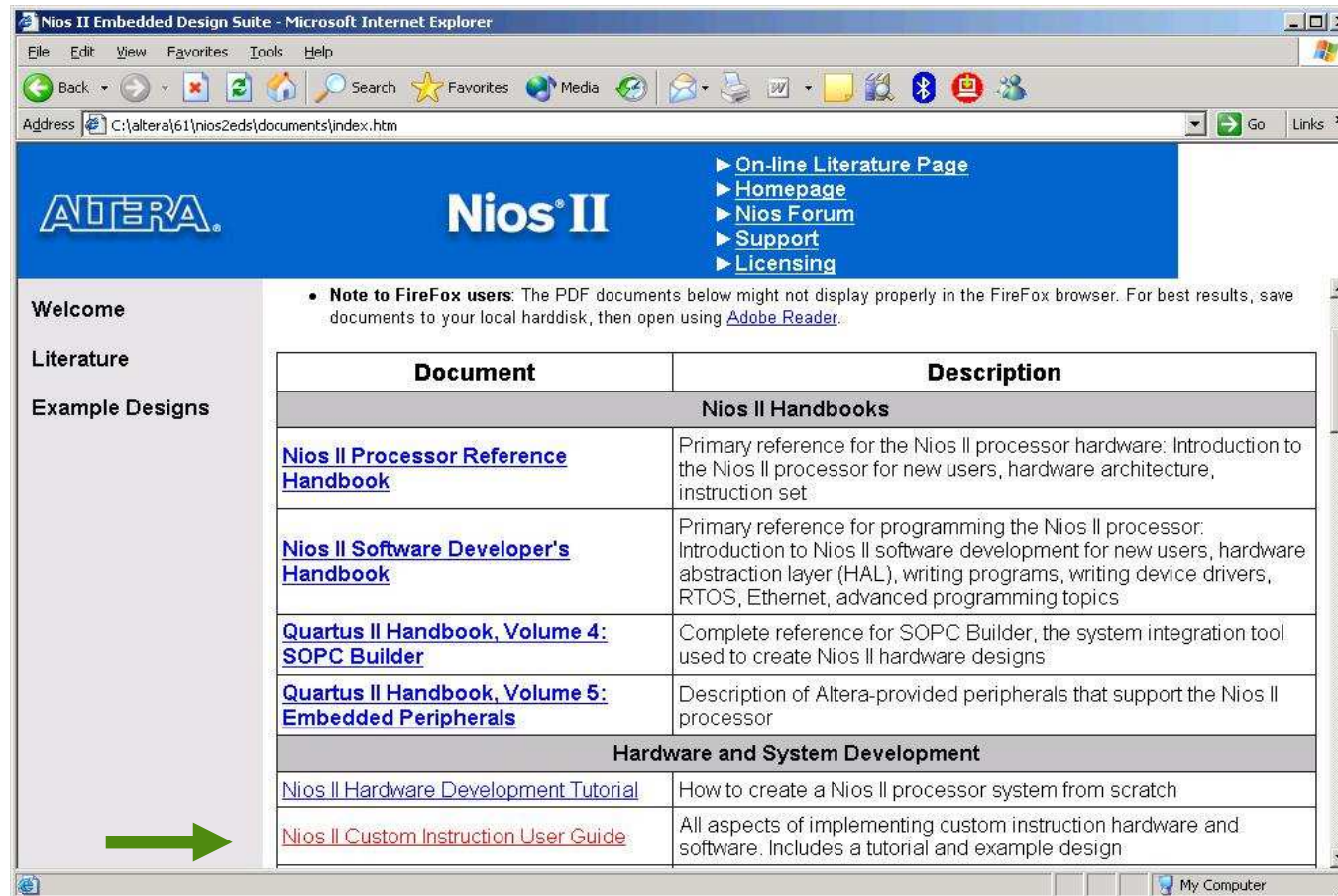
- While floating-point Custom Instructions are faster than software-implemented floating-point operations, they are slower than hardware-based integer (fixed point) math
- A common integer technique is to represent numerical values with an implicit scaling factor
 - As a simple example, if you are calculating **milliamps**, you might represent your values internally as **micro-amps** to eliminate decimals

Floating Point CI Macros

- Map to regular arithmetic symbols unless specific pragmas are included in C function
 - The following will force compiler to use software implementation of floating-point operations even if CI FP hardware exists in your system

Addition	<code>#pragma no_custom_fadds</code>
Subtraction	<code>#pragma no_custom_fsubs</code>
Multiplication	<code>#pragma no_custom_fmuls</code>
Division	<code>#pragma no_custom_fdivs</code>

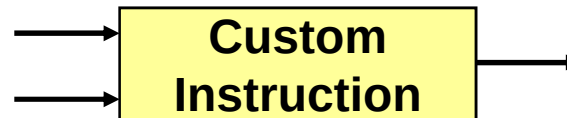
Nios II Custom Instruction User Guide



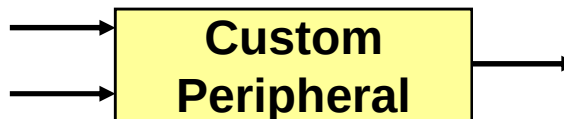
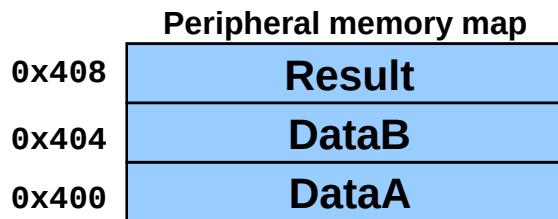
http://www.altera.com/literature/ug/ug_nios2_custom_instruction.pdf

Custom Instruction vs. Peripheral

- Custom Instruction can execute in a single cycle
 - No overhead for call to custom Hardware

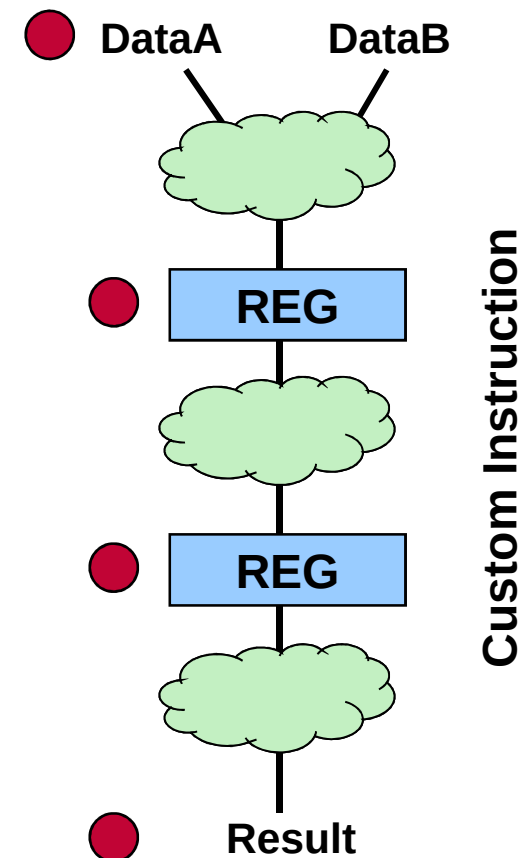
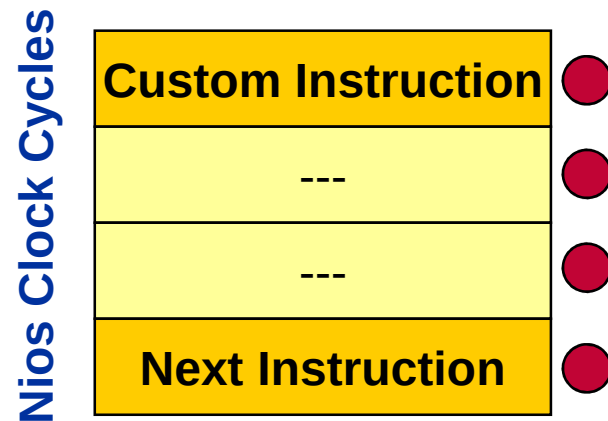


- Access to same Custom Peripheral would take multiple read/write cycles
 - Write DataA, then write DataB, and finally read Result



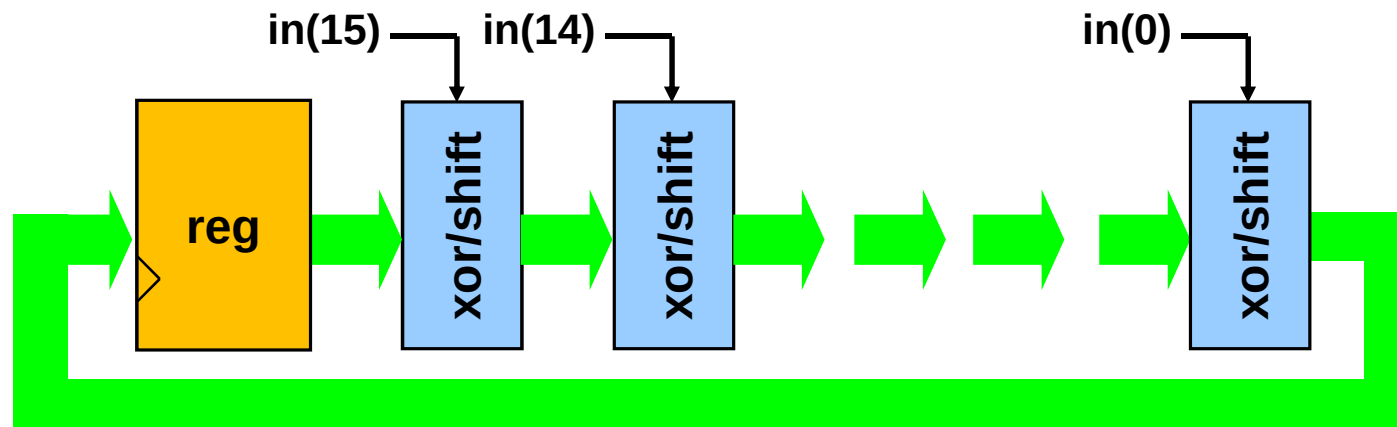
Multi-Cycle Custom Instructions

- Processor stalls while awaiting result
 - Clock cycles = 3



Accelerating CRC

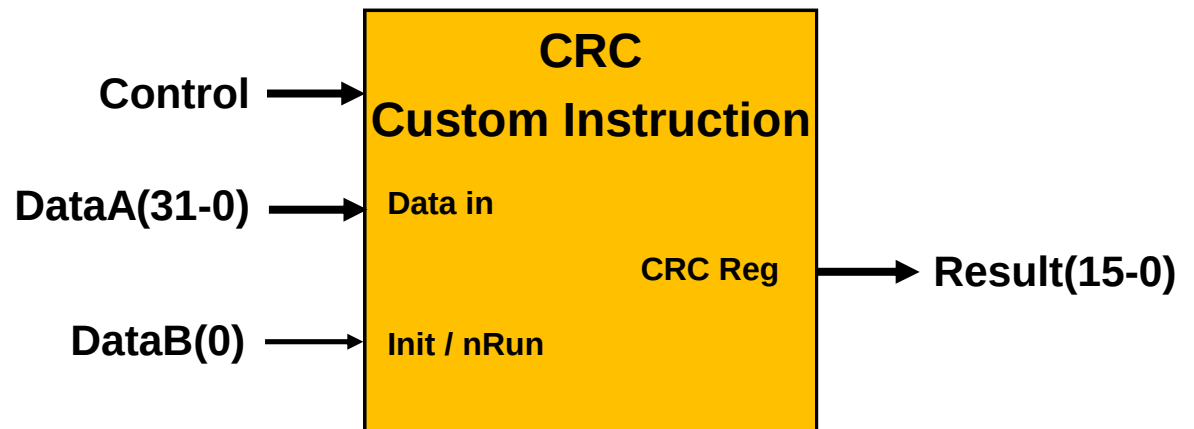
- Implementing the shift and XOR for each bit takes many clock cycles ~50
- Software algorithms tend to use look up tables to pre-compute each byte
- Parallel hardware is fastest



CRC Custom Instruction

- CRC16-CCITT needs to be preset to 0xFFFF at the start of each computation
- Can use Data B input to select between run and load
 - Use of prefix would waste a clock cycle

```
// reset crc  
ALT_CI_CRC(0xFFFF,1);  
  
// run crc  
ALT_CI_CRC(word,0);
```



Information for Exercise 4

- Add a crc Custom Instruction to the Nios II Processor and test with software
- Compare performance to software CRC function

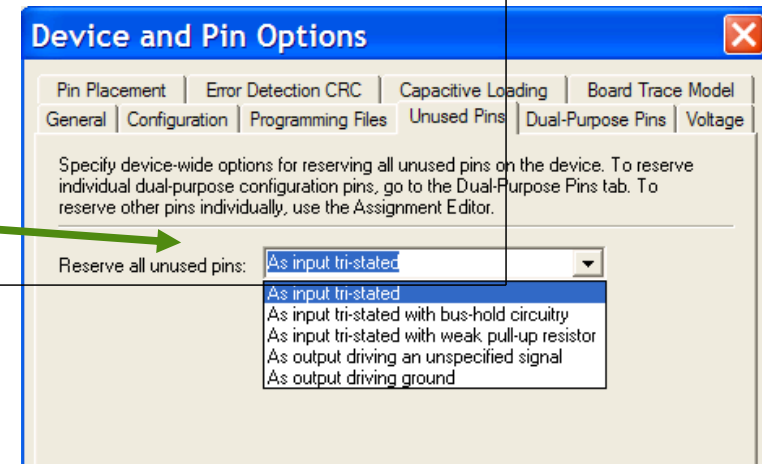
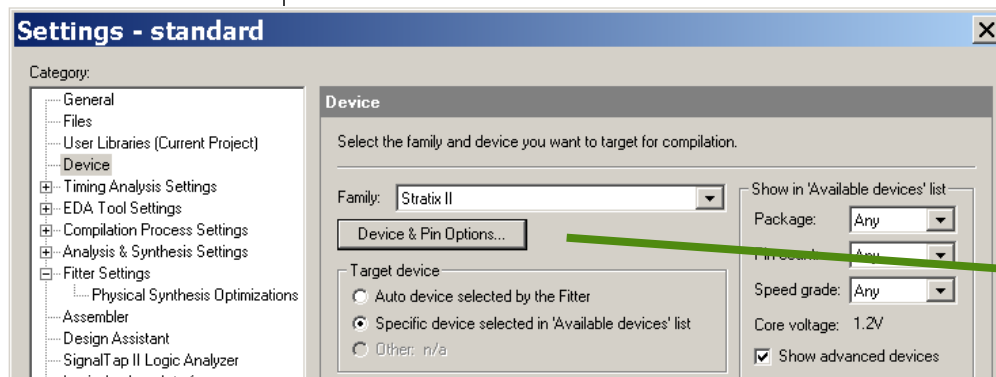
Please go to Exercise 4

Designing with the Nios II Processor and Qsys

Working with Altera Development Boards

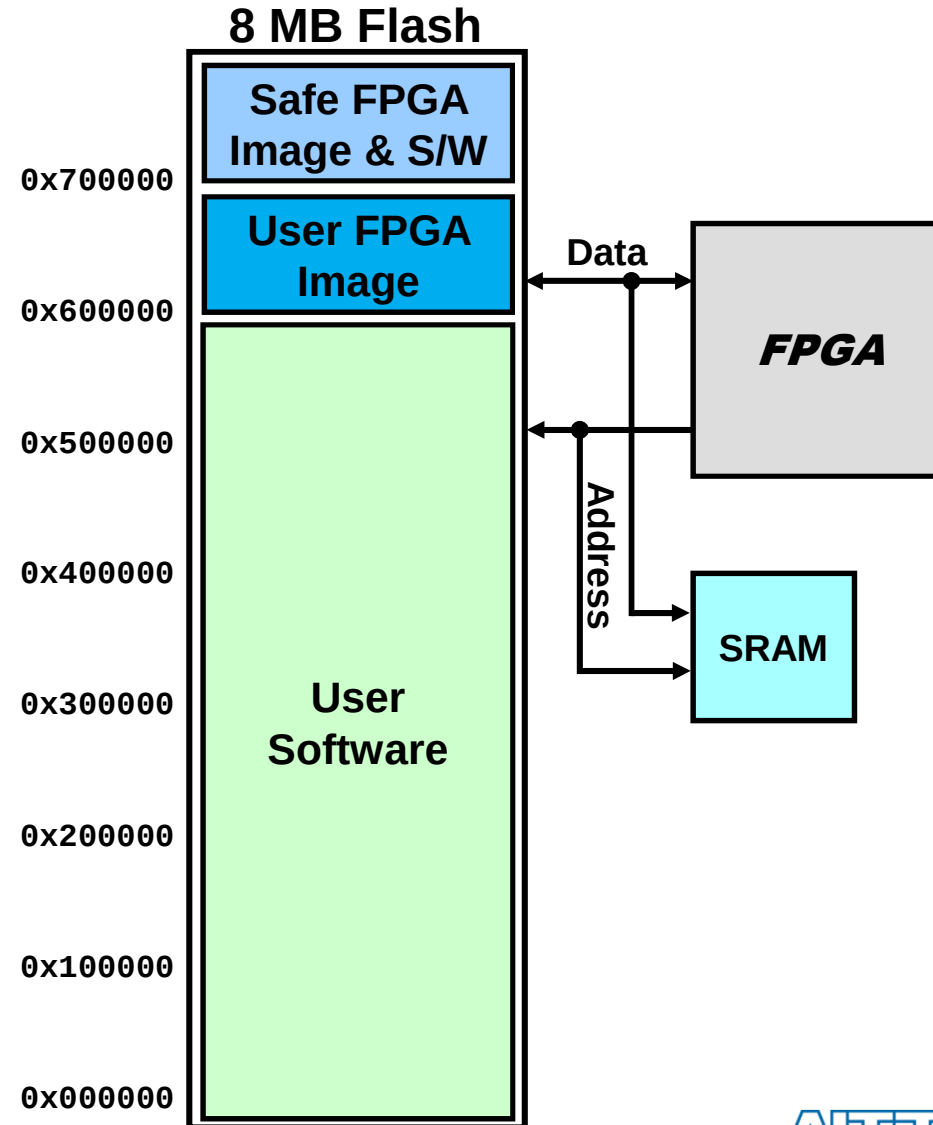
Ensure Unused I/O are Tri-State

- The FPGA may connect to components on the board not used by your design
- There is a connection between FPGA and MAX device on development kits to force reconfiguration
 - Active low, pulled high
 - **Assignments > Device ...**
- Be sure to set Dual Purpose pins to “Use as Regular IO”



Flash Memory Configuration

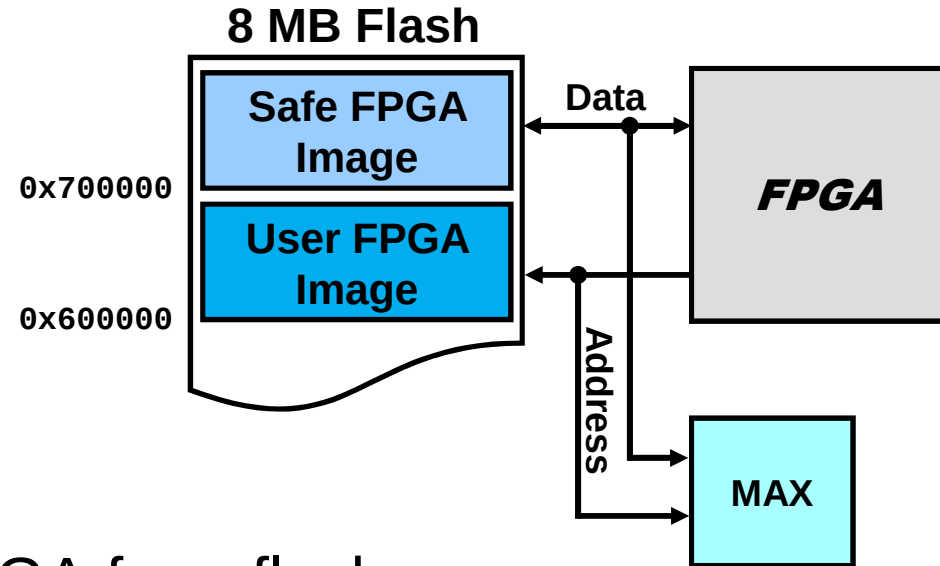
- Common on many Altera development kits



Hardware Configuration from Flash

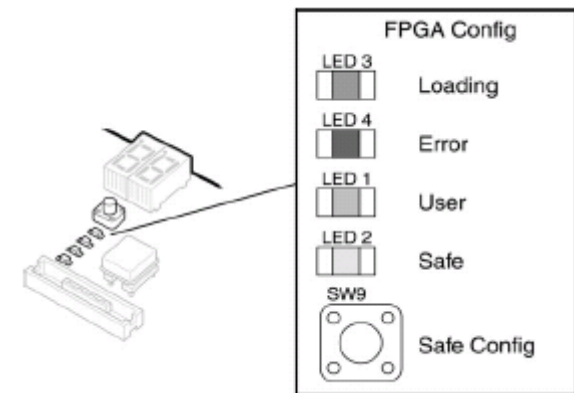
■ Flash Configuration

- Two FPGA images
 - Safe Image
 - User Image



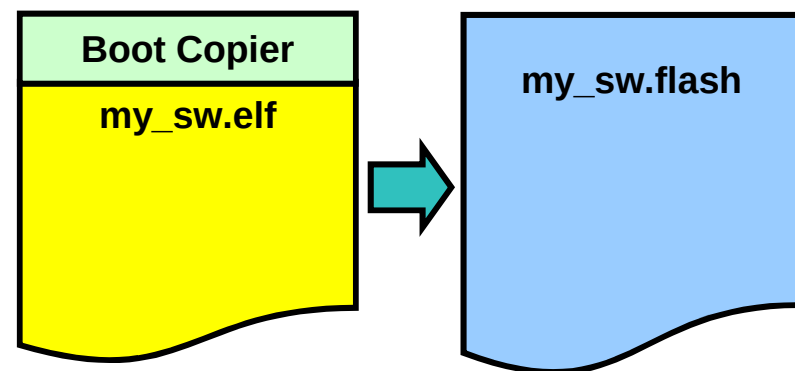
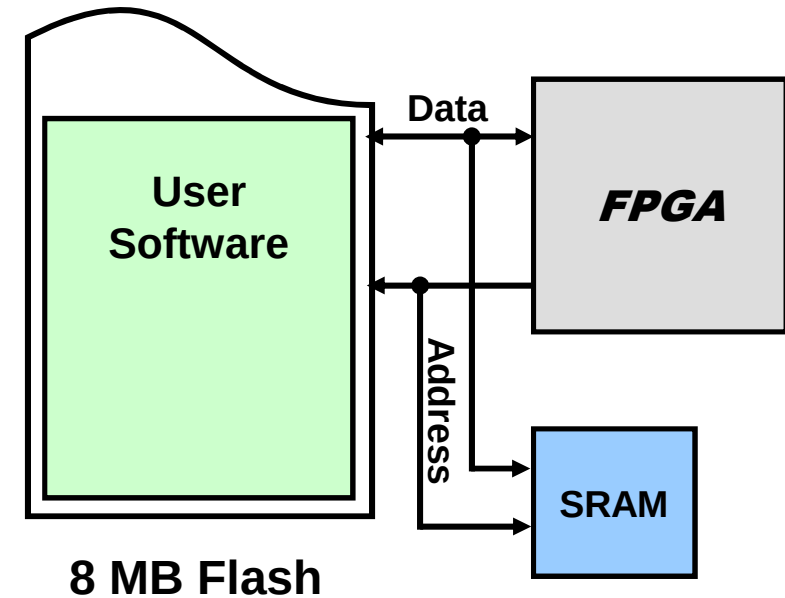
■ MAX[®] device configures FPGA from flash

- Upon power up or press of **Reset Config**
 - MAX Device Loads User Image into FPGA
 - If This Fails MAX Device Loads Safe Image
 - Failure includes no user image present
- Upon press of **Safe Config**
 - MAX Device Loads Safe Image into FPGA



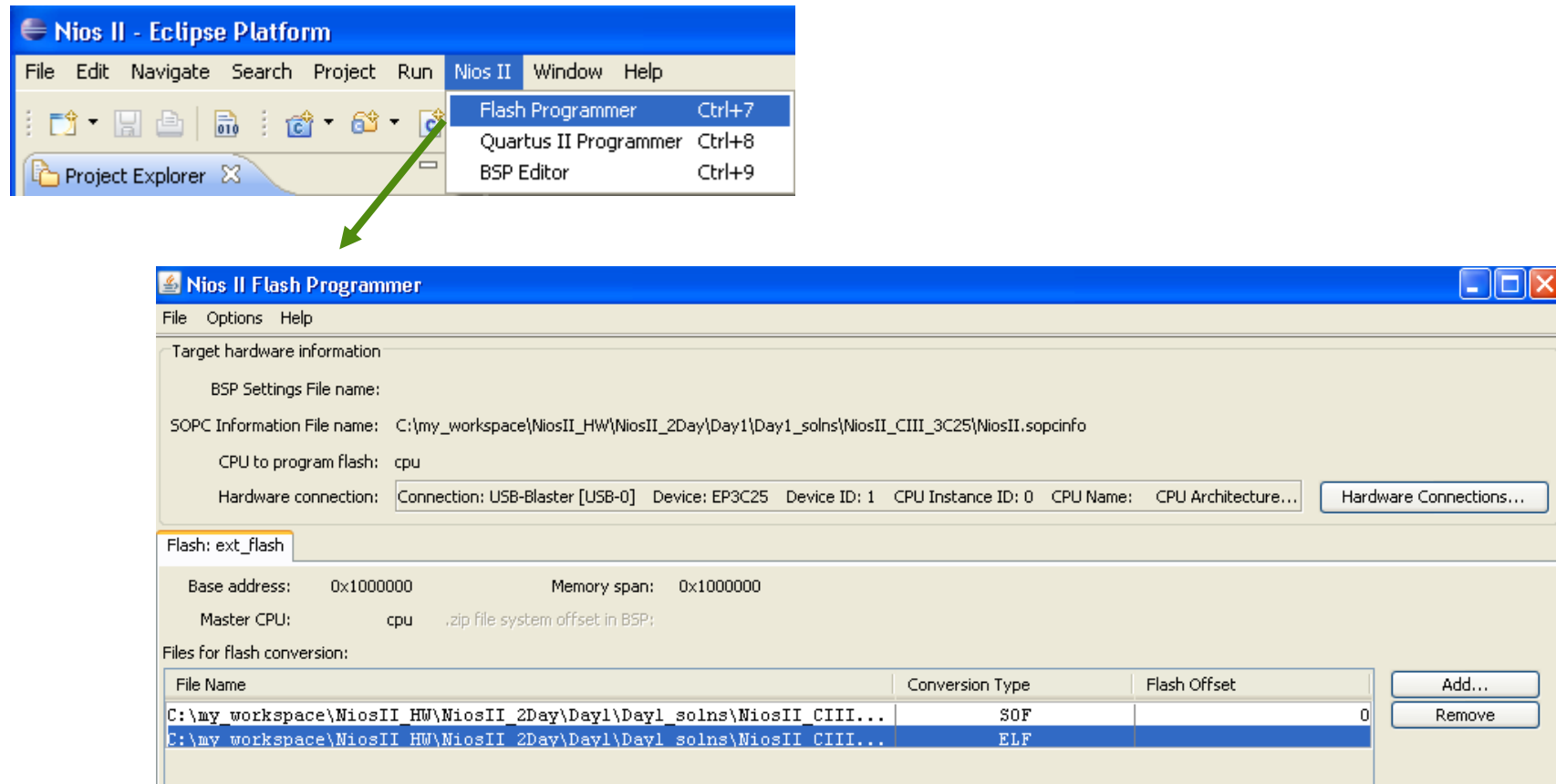
Boot Copier

- Use Flash for program storage
 - Running from Flash is slow
- Nios II flash programmer automatically prepends boot copier to program code
 - Assuming reset address and program memory are not in the same device

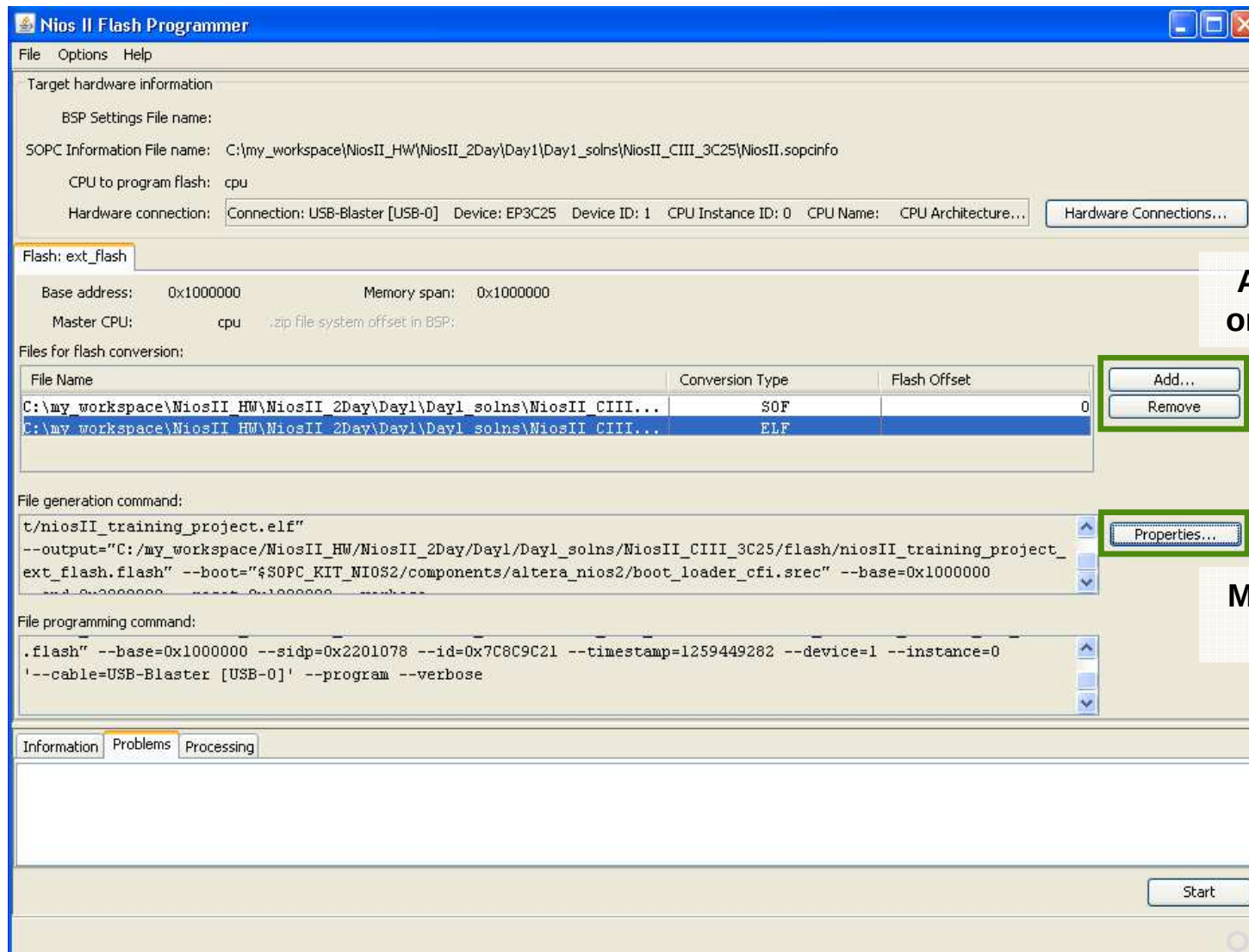


Nios II Flash Programmer

- Program Flash from Nios II SBT for Eclipse or command line



Extra Features



Add hardware (.sof) or software (.elf) files

Modify programming commands

Nios II Flash Programmer

■ Command Shell Utilities

- elf2flash
- sof2flash
- bin2flash
- nios2-flash-programmer

(see “*Nios II Flash Programmer User Guide*”)

■ Requirements

- Need CFI (Common Flash Interface) Flash Memory or...
- EPCS Serial Flash Controller required if booting from an EPCS device

What if You Have a Custom Board?

- Just ensure that your design has CFI flash peripheral and a CFI compliant flash chip on the board

Files for flash conversion:

File Name	Conversion Type	Flash Offset
C:\altera_trn\NiosII HW\NiosII 2Day\Day1\Solutions\NiosII CIII 3C...	SOF	0x60000
C:\altera_trn\NiosII HW\NiosII 2Day\Day1\Solutions\NiosII CIII 3C...	ELF	

- Target design also requires Nios II processor with at least Level 1 JTAG Debug core
 - Flash programming step utilizes this core
- And tri-state bridge peripheral to access the off-chip bus
 - See new *Nios II Flash Programmer User Guide* for details

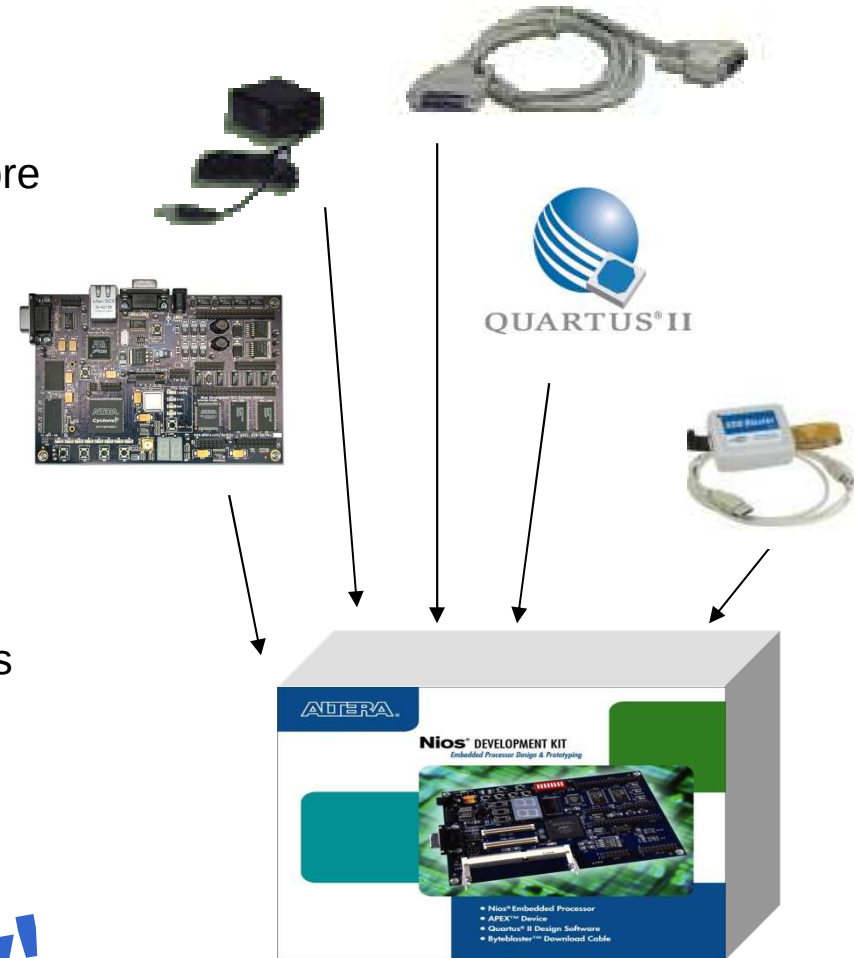
What If Factory Safe Flash Image Overwritten?

- Open Nios II Command Shell
 - Start > Programs > Altera > Nios II EDS *<version>*
> Nios II Command Shell
- Run flash-restoration script provided with the Nios II reference design
 - *./restore_my_flash*
- Follow the script's instructions

Diverse Portfolio of Altera Development Kits

- **Focus on quality & completeness**
 - All boards are fully tested and verified before shipment
 - Accompanied by accurate, technical documentation
- **Provide complete design environment**
 - Board w/featured Altera device
 - Quartus II software (DKE version)
 - “Kit” CD with reference designs and utilities
 - Cables and accessories as necessary
 - OOBE (out of Box experience)

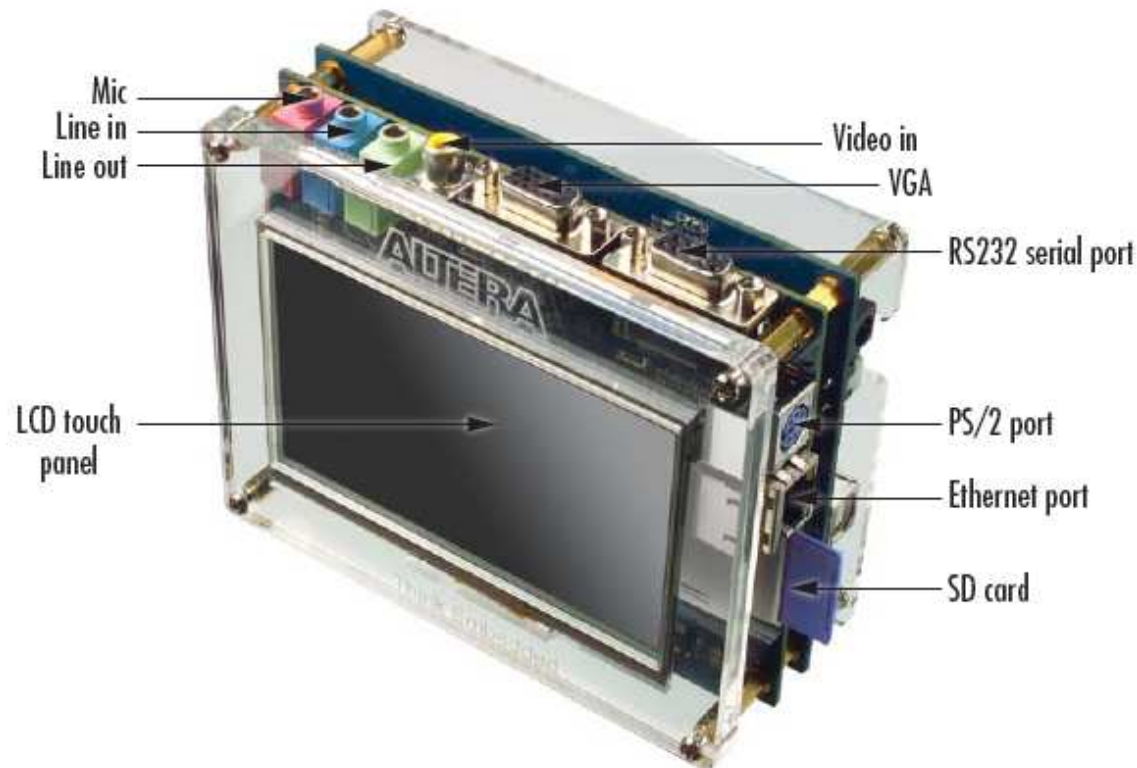
**Just Add
Electricity!**



It's all in the box!

Nios II Embedded Evaluation Kit (NEEK)

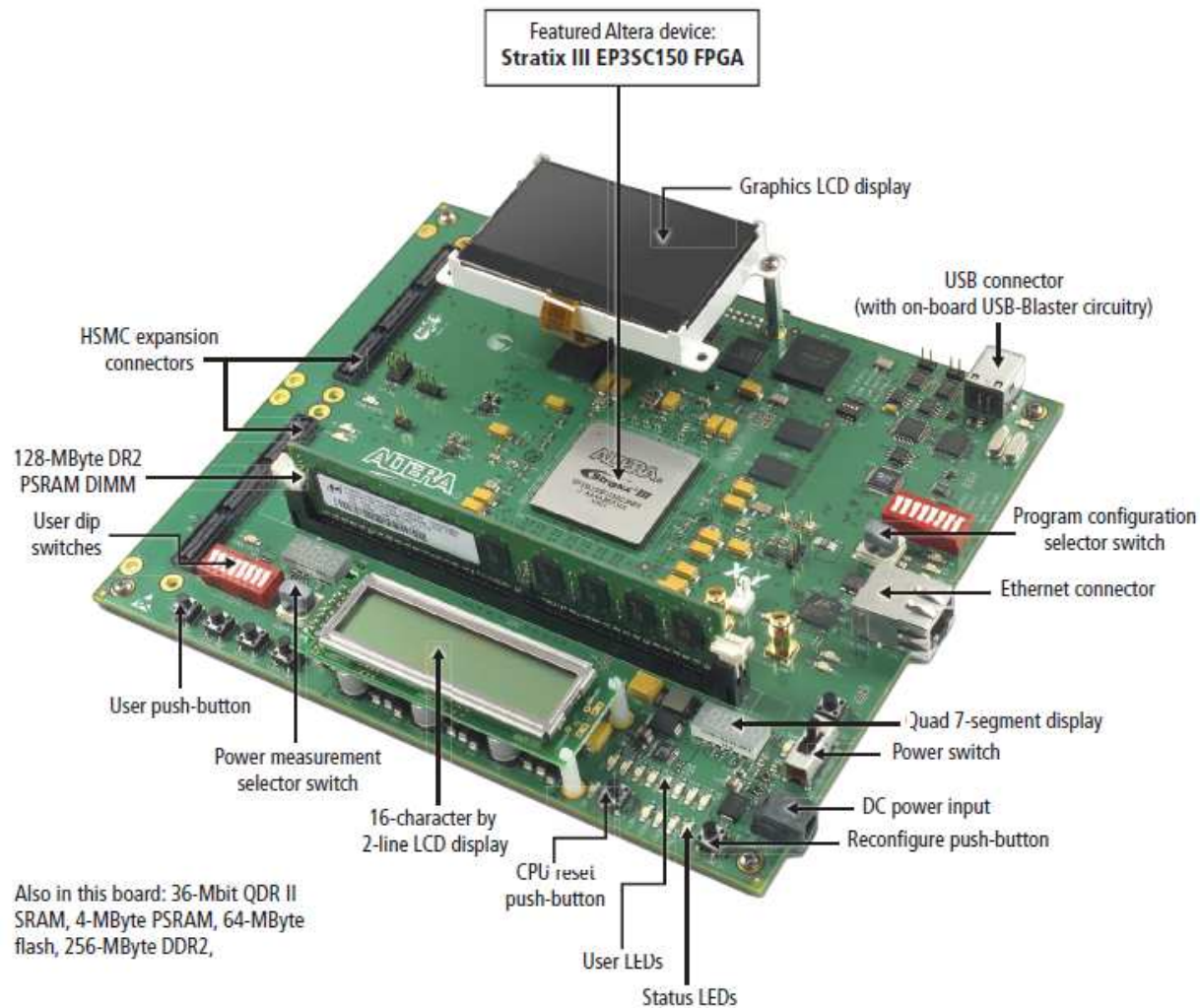
- Multi-board evaluation platform
 - “Cyclone III FPGA Starter Board”
 - Embedded LCD/VGA HSMC Daughter Card



Cyclone III Embedded Development Kit



Stratix III FPGA Development Kit



For Complete List of Dev Kits

■ Refer to www.altera.com:

> [Products](#) > [Dev Kits / Cables](#)

Product Name ▲▼	Device ▲▼	Price ▲▼
Arria GX Development Kit	Arria GX 1AGX60DF780	\$ 995
Audio Video Development Kit, Stratix II GX Edition	Stratix II GX 2SGX90	\$4,995
Cyclone II Starter Development Kit	Cyclone II 2C20	\$ 150
Cyclone III FPGA Development Kit	Cyclone III 3C120N	\$1,495
Cyclone III Starter FPGA Kit	Cyclone III 3C25	\$ 199
DSP Development Kit, Cyclone II Edition	Cyclone II 2C70	\$ 995
DSP Development Kit, Cyclone III Edition	Cyclone III EP3C120N	\$1,895
DSP Development Kit, Stratix II Edition	Stratix II 2S60	\$1,995
DSP Development Kit, Stratix II Professional Edition	Stratix II 2S180	\$5,995
DSP Development Kit, Stratix III Edition	Stratix III 3SL150	\$2,895
MAX II Development Kit	MAX II 1270	\$ 150
Nios II Development Kit, Cyclone II Edition (2C35)	Cyclone II 2C35	\$ 995
Nios II Development Kit, Stratix II Edition	Stratix II 2S60	\$ 995
Nios II Embedded Evaluation Kit, Cyclone III Edition	Cyclone III 3C25	\$ 449
PCI Development Kit, Cyclone II Edition	Cyclone II 2C35	\$ 995
PCI Express Development Kit, Stratix II GX Edition	Stratix II GX 2SGX90	\$2,995
Stratix III FPGA Development Kit	Stratix III 3SL150	\$2,495
Transceiver Signal Integrity Development Kit, Stratix II GX Edition	Stratix II GX 2SGX90	\$1,295
Video Development Kit, Cyclone II Edition	Cyclone II 2C70	\$1,095

Information for Exercise 5

- Program the hardware design and CRC software program to the Flash chip on your development board
- Boot system and run program stored in flash

Please go to Exercise 5

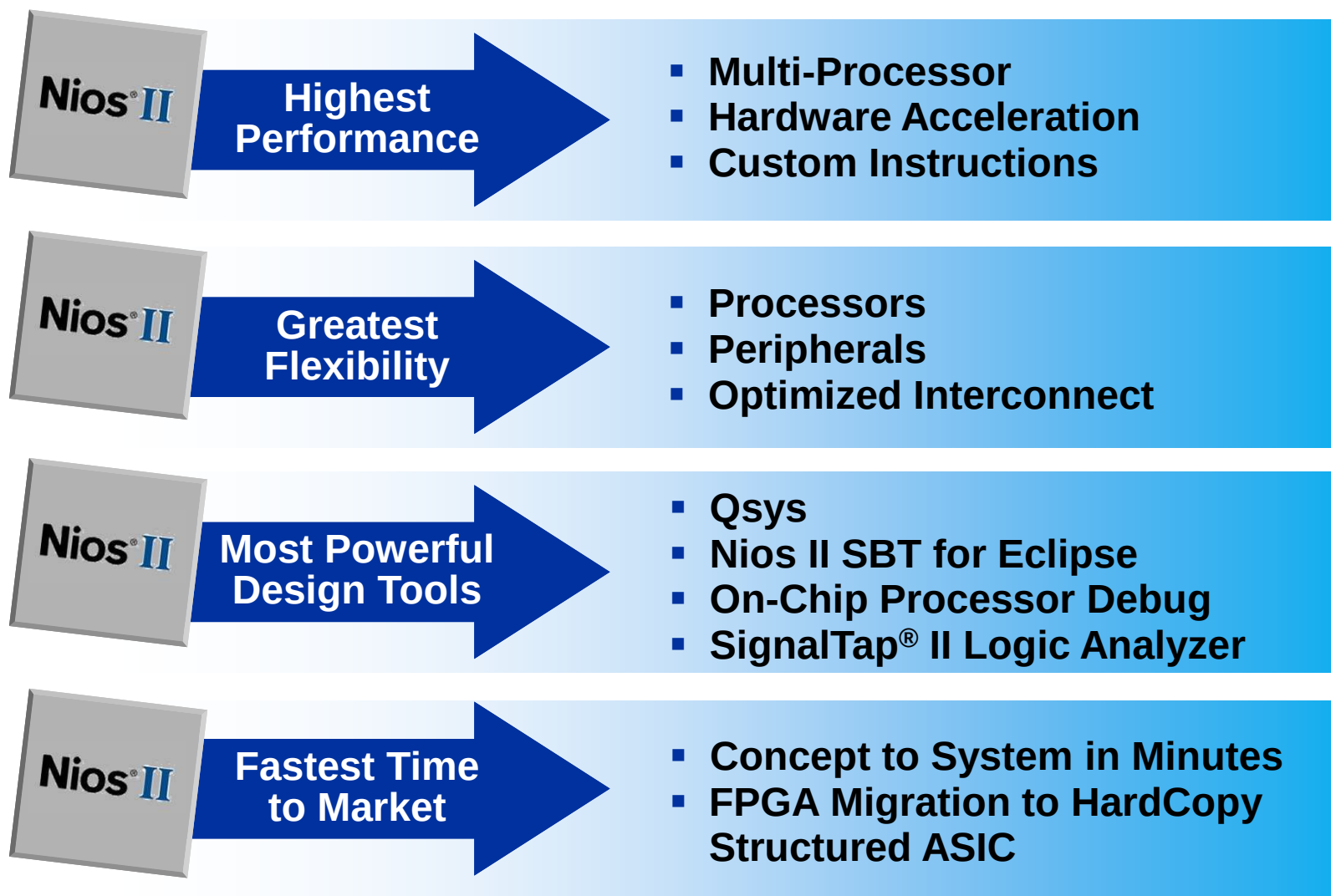
Designing with the Nios II Processor and Qsys

Class Summary

Class Summary

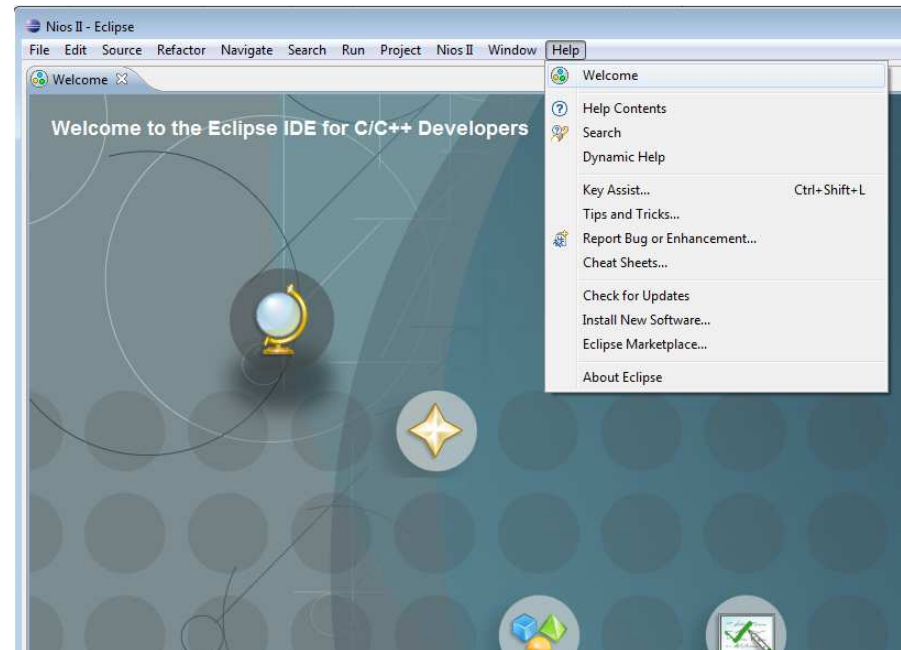
- Embedded Design Tools
 - Quartus II Software
 - Qsys
 - Nios II Software Build Tools for Eclipse
- Creating Systems on a Programmable Chip
- Applications for Qsys and Nios II Processor
- System Interconnect operation
 - Avalon-MM Interface
 - Avalon-ST Interface
- Adding Custom Hardware Blocks
- Creating and testing your own IP

Nios II Processor - Leads the Industry



Getting Help Resources

- Nios II Software Build Tools for Eclipse Help and Welcome pages, which provide
 - Overview
 - Tutorials + Cheat Sheets
 - What's new
 - Search capabilities
- Qsys Help menu
- Nios II Processor Hardware and Software Developers Handbooks
- Quartus II Handbook
- Embedded Design Handbook
- System Console User Guide
- Tutorials - Multiprocessor and Qsys
- One-Click Download
 - <http://www.altera.com/literature/lit-nio2.jsp>



Support Pages

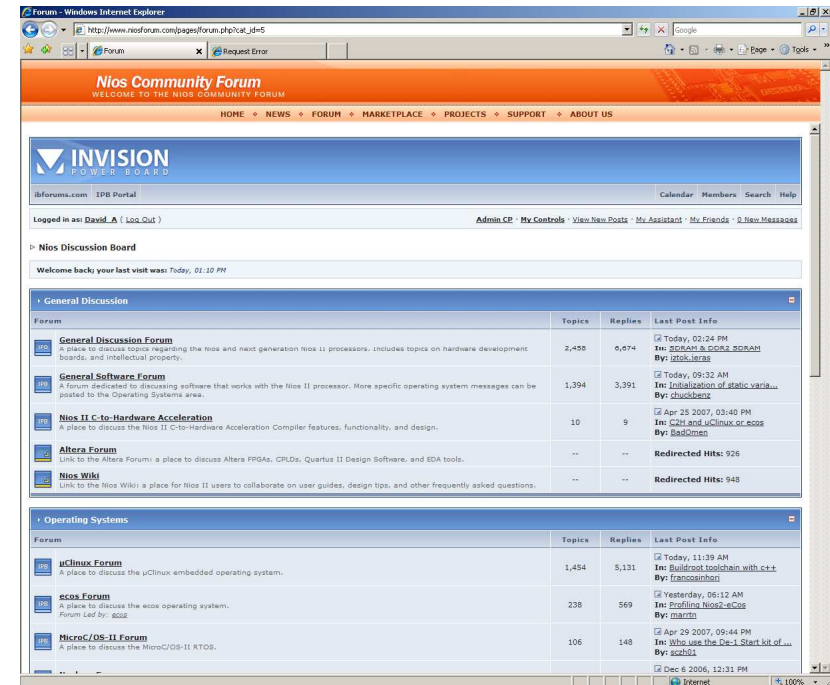
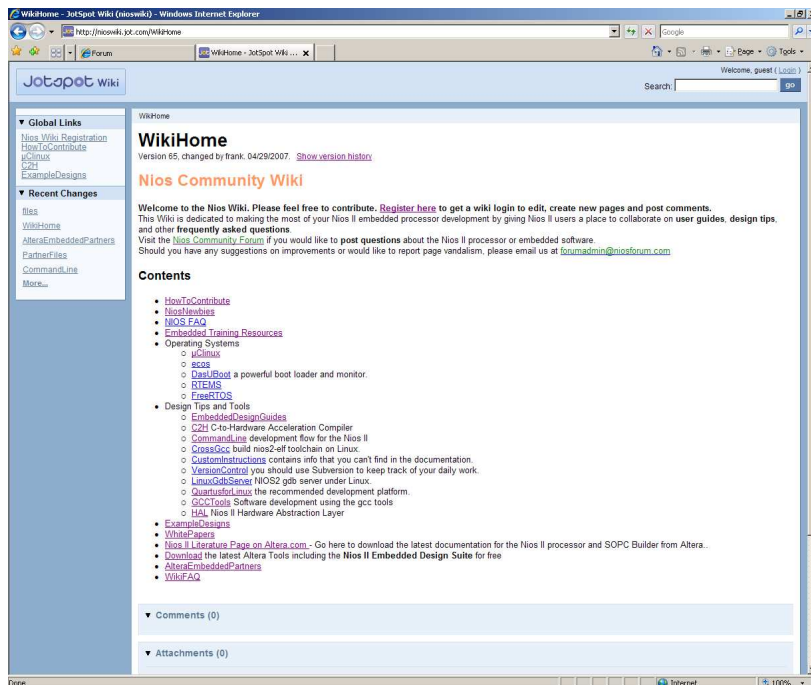
- <http://www.altera.com/support/ip/ips-index.html>
 - See Nios II Embedded Processor Support Pages
 - Release Notes
 - Errata
 - etc.



Nios II Processor User Community

Nios Forum (www.alteraforum.com)

- Thousands of registered Nios II users
- Millions of topic views



Nios Wiki (www.nioswiki.com)

- Hundreds of pages of Nios II user-generated processor documentation
- Hundreds of daily visitors on average

Altera Technical Support

- Reference Quartus II software on-line help
- [Quartus II Handbook](#)
- Consult Altera applications (factory applications engineers)
 - MySupport: <http://www.altera.com/mysupport>
 - Hotline: (800) 800-EPLD (7:00 a.m. - 5:00 p.m. PST)
- Field applications engineers: contact local Altera sales office
- Receive literature by mail: (888) 3-ALTERA
- Altera Forum: www.alteraforum.com
- FTP: [ftp.altera.com](ftp://ftp.altera.com)
- World-wide web: <http://www.altera.com>
 - Use solutions to search for answers to technical problems
 - View design examples

Learn More Through Technical Training

Instructor-Led Training



With Altera's instructor-led training courses, you can:

- Listen to a lecture from an Altera technical training engineer (instructor)
- Complete hands-on exercises with guidance from an Altera instructor
- Ask questions and receive real-time answers from an Altera instructor
- Each instructor-led class is one or two days in length (8 working hours per day).

Online Training



With Altera's online training courses, you can:

- Take a course at any time that is convenient for you
- Take a course from the comfort of your home or office (no need to travel as with instructor-led courses)
- Each online course will take approximate one to three hours to complete.

<http://www.altera.com/training>

View training class schedule and register for a class

Designing with the Nios II Processor and Qsys

Thank You!